

Strings

3. Longest Substring Without Repeating Characters

Here is how a senior engineer approaches this problem. The key to succeeding in top-tier interviews isn't just writing the correct code; it's communicating the "why" behind the code, proving that you understand the underlying mechanics rather than just memorizing a pattern.

1. Problem Explanation

The Goal: Given a string of characters, we need to find the length of the longest "substring" that does not contain any duplicate characters.

Definitions: * **Substring:** A contiguous, unbroken sequence of characters within a string. For example, in the string "apple", "app" and "ple" are substrings, but "ape" is not (it skips the 'p' and 'l', making it a *subsequence*, not a substring). * **Without Repeating Characters:** Every single character in our chosen sequence must be completely unique.

Examples: * Input: "abcabcbb" -> The longest valid substring is "abc" (or "bca", or "cab"). Length = 3. * Input: "bbbbbb" -> The longest valid substring is just "b". Length = 1. * Input: "pwwkew" -> The longest valid substring is "wke" or "kew". Length = 3. (Note that "pwke" is a subsequence, not a substring).

2. Solution Explanation

The Intuition (The "Why"): Imagine reading a sentence. You start reading from the beginning, adding one letter at a time to your "current word". As long as you don't see a letter you've already seen, your word gets longer, and you are happy.

But what happens when you hit a duplicate? Let's look at the string "abcad". You read 'a', 'b', 'c'. All good. Current sequence: "abc". Next, you see another 'a'. You cannot have "abca" because 'a' repeats.

A naive (brute force) approach would be to throw away the whole sequence and start over from 'b'. But as a senior engineer, we notice an inefficiency. We don't need to throw *everything* away. We only need to throw away characters from the beginning of our sequence *just until* the duplicate 'a' is removed. If we chop off the first 'a', our remaining valid sequence is "bca". We can now continue reading the rest of the string ("d").

The Algorithm (The "How" - Sliding Window): We use two pointers, `left` and `right`, to represent the boundaries of our current valid sequence (our

“window”). 1. We expand our window by moving the **right** pointer one character at a time. 2. We keep track of every character we’ve seen and its exact index position using a Hash Map (a dictionary). 3. If the character at our **right** pointer is already in our map **AND** its previous position is inside our current window (i.e., its index is $\geq \text{left}$), we have a collision. 4. To fix the collision, we instantly teleport our **left** pointer to the position *right after* the previous occurrence of the duplicate character. 5. We update the character’s new position in our map. 6. We calculate the length of the current window ($\text{right} - \text{left} + 1$) and see if it’s the largest we’ve found so far.

End-to-End Walkthrough with ASCII Visualizations:

Let’s trace the tricky input: `s = "pwwkew"`

```
[ INITIAL STATE ]
String:  p  w  w  k  e  w
Index:   0  1  2  3  4  5
Pointers:
L = 0
R = 0
Seen Map: {}
Max Length: 0
```

```
-----
[ STEP 1: R = 0, char = 'p' ]
Is 'p' in our window? No.
Add 'p' to map. Map = {p: 0}
Window length = R - L + 1 = (0 - 0 + 1) = 1. Max Length updates to 1.
```

```
Window: [p] w  w  k  e  w
         ^
         L,R
```

```
-----
[ STEP 2: R = 1, char = 'w' ]
Is 'w' in our window? No.
Add 'w' to map. Map = {p: 0, w: 1}
Window length = R - L + 1 = (1 - 0 + 1) = 2. Max Length updates to 2.
```

```
Window: [p  w] w  k  e  w
         ^  ^
         L  R
```

```
-----
[ STEP 3: R = 2, char = 'w' ]
Is 'w' in our window? YES! (Map says 'w' was at index 1. L is at 0. 1  $\geq$  0).
COLLISION!
```

Action: Jump L to (previous 'w' index + 1) = 1 + 1 = 2.
Update Map: 'w' is now at index 2. Map = {p: 0, w: 2}
Window length = R - L + 1 = (2 - 2 + 1) = 1. Max is still 2.

Window: p w [w] k e w
 ^
 L,R

(Notice how 'p' and the first 'w' naturally fell out of the window)

[STEP 4: R = 3, char = 'k']
Is 'k' in our window? No.
Add 'k' to map. Map = {p: 0, w: 2, k: 3}
Window length = (3 - 2 + 1) = 2. Max is still 2.

Window: p w [w k] e w
 ^ ^
 L R

[STEP 5: R = 4, char = 'e']
Is 'e' in our window? No.
Add 'e' to map. Map = {p: 0, w: 2, k: 3, e: 4}
Window length = (4 - 2 + 1) = 3. Max Length updates to 3.

Window: p w [w k e] w
 ^ ^ ^
 L R

[STEP 6: R = 5, char = 'w']
Is 'w' in our window? YES! (Map says 'w' is at index 2. L is at 2. 2 >= 2).
COLLISION!
Action: Jump L to (previous 'w' index + 1) = 2 + 1 = 3.
Update Map: 'w' is now at index 5. Map = {p: 0, w: 5, k: 3, e: 4}
Window length = (5 - 3 + 1) = 3. Max is still 3.

Window: p w w [k e w]
 ^ ^ ^
 L R

END OF STRING. Final Answer: 3.

3. Time and Space Complexity Analysis

Time Complexity: $O(N)$ where N is the length of the string. **Space Complexity:** $O(M)$ where M is the size of the character set (e.g., 26 for lowercase English, 128 for standard ASCII).

TIME COMPLEXITY DERIVATION ($O(N)$):

The right pointer (R) moves exactly from 0 to $N-1$.

It visits each character exactly once.

The left pointer (L) never moves backward; it only jumps forward based on map lookups.

Map lookups take $O(1)$ time.

[Timeline of operations for N characters]

Char 0: R moves right (1 operation), map updated (1 op). Total = 2.

Char 1: R moves right (1 operation), map updated (1 op). Total = 2.

...

Char N : R moves right (1 op), check map (1 op), move L (1 op). Total = 3.

Worst case operations roughly = $3 * N$.

Constants are dropped in Big- O notation, resulting in $O(N)$.

SPACE COMPLEXITY DERIVATION ($O(M)$):

We are storing characters in a Hash Map.

If the string is 1,000,000 characters long, but only contains English letters, the map will never hold more than 26 key-value pairs.

Therefore, the space doesn't scale infinitely with the input string length (N), it scales with the number of unique characters possible in the alphabet (M).

[Hash Map Memory Block]

| 'a': 0 | 'b': 1 | 'c': 2 | ... | maximum 128 entries for basic ASCII |

Because M is fixed (128 or 256), space is often considered $O(1)$ constant in practical interview contexts.

4. Solution Code

Here are two implementations. Both are $O(N)$ time, but the “Optimized HashMap” approach is the L5/L6 standard because it prevents the `left` pointer from iterating slowly to catch up to duplicates.

Python Snippets

Approach 1: Basic Sliding Window using a Set (Good, standard approach)

Time: $O(N)$, Space: $O(M)$

```
def lengthOfLongestSubstring_Set(s: str) -> int:
    char_set = set()
    left = 0
```

```

max_length = 0

for right in range(len(s)):
    # If we see a duplicate, slowly shrink window from the left
    # until the duplicate is removed from the set.
    while s[right] in char_set:
        char_set.remove(s[left])
        left += 1

    char_set.add(s[right])
    max_length = max(max_length, right - left + 1)

return max_length

# Approach 2: Optimized Sliding Window using HashMap (The L5/L6 Answer)
# Time: O(N), Space: O(M)
def lengthOfLongestSubstring_Optimized(s: str) -> int:
    """
    We use a dictionary to store the most recent index of each character.
    Instead of slowly creeping the `left` pointer forward, we instantly jump
    it to the position right after the last occurrence of the duplicate.
    """
    char_index_map = {}
    left = 0
    max_length = 0

    for right in range(len(s)):
        current_char = s[right]

        # If char was seen BEFORE AND its previous index is inside our current window
        if current_char in char_index_map and char_index_map[current_char] >= left:
            # Teleport the left pointer past the duplicate
            left = char_index_map[current_char] + 1

        # Record the character's most recent position
        char_index_map[current_char] = right

        # Update the maximum length found so far
        window_size = right - left + 1
        if window_size > max_length:
            max_length = window_size

    return max_length

```

JavaScript Snippets

// Approach 1: Basic Sliding Window using a Set (Good, standard approach)

// Time: O(N), Space: O(M)

```
function lengthOfLongestSubstring_Set(s) {
    const charSet = new Set();
    let left = 0;
    let maxLength = 0;

    for (let right = 0; right < s.length; right++) {
        // Shrink the window slowly until the duplicate is gone
        while (charSet.has(s[right])) {
            charSet.delete(s[left]);
            left++;
        }

        charSet.add(s[right]);
        maxLength = Math.max(maxLength, right - left + 1);
    }

    return maxLength;
}
```

// Approach 2: Optimized Sliding Window using Map (The L5/L6 Answer)

// Time: O(N), Space: O(M)

```
function lengthOfLongestSubstring_Optimized(s) {
    /*
     * We map each character to its most recent index.
     * This allows us to skip the 'while' loop from the Set approach
     * and instantly jump the 'left' pointer forward, saving redundant iterations.
    */
    const charIndexMap = new Map();
    let left = 0;
    let maxLength = 0;

    for (let right = 0; right < s.length; right++) {
        const currentChar = s[right];

        // If duplicate found inside the current window, jump the left pointer
        if (charIndexMap.has(currentChar) && charIndexMap.get(currentChar) >= left) {
            left = charIndexMap.get(currentChar) + 1;
        }

        charIndexMap.set(currentChar, right);

        const windowSize = right - left + 1;
    }
}
```

```

        if (windowSize > maxLength) {
            maxLength = windowSize;
        }
    }

    return maxLength;
}

```

Terminology Addendum (Note 1)

The Sliding Window Technique: * **What it is:** An algorithmic pattern used to perform operations on a specific subset of an array or string (a “window”). The window is defined by two pointers (usually `left` and `right`) and can grow, shrink, or slide forward across the data. * **Why it helps:** It converts nested loops (which cause $O(N^2)$ or $O(N^3)$ time complexity) into a single pass through the data. It reuses the work done in the previous step rather than starting over from scratch. * **How it applies here:** Instead of generating every possible substring starting from index 0, then index 1, then index 2, we dynamically grow a single substring until it becomes invalid, then precisely shrink it just enough to make it valid again.

Real-World Interview Variations (Note 2)

Companies like Google, Meta, and Bloomberg rarely ask the pure LeetCode #3 verbatim anymore. They wrap the logic in a real-world scenario or tweak the constraints.

Variation 1: “Longest substring with at most K distinct characters” (Frequent at Google/Bloomberg) * *Prompt:* Instead of “zero repeating characters”, you are allowed up to K unique characters, repeating as much as they want. (e.g., $K=2$, “eacea” -> “eae”). * *How to solve:* Still use a Sliding Window, but your Hash Map now tracks the *frequency* or *last seen index* of characters. When the size of your map (number of distinct keys) exceeds K, you advance the `left` pointer and remove characters from the map until the map size is exactly K again.

Variation 2: “Max Consecutive Ones III” / “Find the longest log sequence under faulty conditions” (Frequent at Meta) * *Prompt:* Given an array of 0s and 1s, you may change up to K values from 0 to 1. Return the length of the longest (contiguous) subarray that contains only 1s. * *How to solve:* The “sliding window” tracks the count of zeros inside the window. If `zero_count > K`, you shrink the window from the left until the zero count drops back down to K. It’s the exact same muscle memory as the primary problem.

Variation 3: Data Stream Scenario (Real-World System Design flavor) * *Prompt:* You are receiving a real-time stream of server log characters over a network socket. You cannot hold the entire string in memory (N is functionally infinite). Find the longest sequence of unique identifiers at any given moment. * *How to solve:* The logic holds, but the Hash Map approach proves its worth here. You don't need the string in memory; you only need the **left** pointer, an integer tracking the current index (**right**), and a HashMap of size up to 256 (ASCII) pointing to integers. The Space Complexity remains $O(1)$ despite an infinite N, proving the efficiency of the L5 solution.

424. Longest Repeating Character Replacement

Here is how a Senior Software Engineer (L5/L6) would break down and explain this problem in a real interview setting. At this level, the goal isn't just to write code that passes; it's to communicate the "why" so clearly that the interviewer is convinced you deeply understand the mechanics of the algorithm.

1. Problem Explanation

The Premise: You are given a string **s** made of uppercase English letters and a magic number **k**. You are allowed to change any character in the string to any other uppercase letter, but you can only do this a maximum of **k** times.

The Goal: Find the length of the longest contiguous block (substring) of identical characters you can create using at most **k** replacements.

The Intuition (The "Why"): Imagine you have a window of characters: [A, B, A, B, B]. You want to make them all the same. Which character should you turn them into? Logically, you should pick the character that *already appears the most frequently* in your window. In [A, B, A, B, B], 'B' appears 3 times, and 'A' appears 2 times. If we change all 'A's to 'B's, it takes 2 operations. If we change all 'B's to 'A's, it takes 3 operations.

To save our precious **k** operations, we always want to replace the **minority characters** to match the **majority character**.

Therefore, the mathematical condition that dictates if a window is "valid" (meaning we can make all characters in it the same) is: **Total Characters in Window - Count of the Most Frequent Character $\leq k$**

If the number of minority characters is greater than **k**, the window is invalid. We don't have enough operations to fix it.

2. Solution Explanation

To solve this efficiently, we use a **Sliding Window** approach.

Instead of checking every possible combination (which is terribly slow, taking $O(N^2)$ or $O(N^3)$ time), we maintain a “window” using two pointers: Left (L) and Right (R). We expand the window by moving R to the right, taking in new characters. If our window ever becomes invalid (we need more than k replacements), we shrink it by moving L to the right until it becomes valid again.

The L5/L6 Optimization Insight (The “Aha!” Moment): A standard sliding window would strictly maintain the exact validity of the window. When shrinking L, it would decrement character counts and find the *new* most frequent character. However, we only care about finding the **maximum** length. If we already found a valid window of length 5, we don’t care about valid windows of length 4. We only care when the window can grow to length 6. Therefore, we track a historical `max_freq_so_far`. We **never decrease it**, even when characters fall out of our window. We only care if a new character enters and pushes `max_freq_so_far` even higher, allowing our window to expand!

Detailed ASCII Walkthrough Let’s trace `s = "AABABBA"`, $k = 1$. Variables: * L: Left pointer * R: Right pointer * `counts`: Dictionary/Array of character frequencies in the current window * `maxf`: The historical maximum frequency of a single character seen in *any* window so far * `max_len`: The length of the longest valid window seen so far

Initial State:

String: A A B A B B A

Indices: 0 1 2 3 4 5 6

$k = 1$

`L = 0, maxf = 0, max_len = 0, counts = {}`

STEP 1: Expand R to index 0 ('A')

[A] A B A B B A

^

L,R

`counts = {A: 1}`

`maxf = max(0, 1) = 1`

`Window Length = R - L + 1 = 0 - 0 + 1 = 1`

`Replacements needed = Length - maxf = 1 - 1 = 0 <= k (VALID)`

`max_len = 1`

STEP 2: Expand R to index 1 ('A')

[A A] B A B B A

^ ^

L R

`counts = {A: 2}`

```

maxf = max(1, 2) = 2
Window Length = 1 - 0 + 1 = 2
Replacements needed = 2 - 2 = 0 <= k (VALID)
max_len = 2

```

```

-----
STEP 3: Expand R to index 2 ('B')
[A A B] A B B A
  ^   ^
  L   R
counts = {A: 2, B: 1}
maxf = max(2, 1) = 2
Window Length = 2 - 0 + 1 = 3
Replacements needed = 3 - 2 = 1 <= k (VALID)
max_len = 3

```

```

-----
STEP 4: Expand R to index 3 ('A')
[A A B A] B B A
  ^   ^
  L   R
counts = {A: 3, B: 1}
maxf = max(2, 3) = 3
Window Length = 3 - 0 + 1 = 4
Replacements needed = 4 - 3 = 1 <= k (VALID)
max_len = 4

```

```

-----
STEP 5: Expand R to index 4 ('B')
[A A B A B] B A
  ^   ^
  L   R
counts = {A: 3, B: 2}
maxf = max(3, 2) = 3
Window Length = 4 - 0 + 1 = 5
Replacements needed = 5 - 3 = 2 > k (INVALID!)

```

Because it's invalid, we must shrink the window by moving L right by 1.
 We remove s[L] which is 'A' from counts. L becomes 1.
 counts = {A: 2, B: 2}
 Notice we DO NOT lower maxf. maxf stays 3.
 Window shifts forward:

```

[A B A B] B A
  ^   ^
  L   R

```

STEP 6: Expand R to index 5 ('B')

```

A [A B A B B] A
  ^         ^
  L         R
counts = {A: 2, B: 3}
maxf = max(3, 3) = 3
Window Length = 5 - 1 + 1 = 5
Replacements needed = 5 - 3 = 2 > k (INVALID!)

Shrink window: L moves to 2, s[1] ('A') removed.
counts = {A: 1, B: 3}
Window shifts forward:
A A [B A B B] A
   ^         ^
   L         R

```

STEP 7: Expand R to index 6 ('A')

```

A A [B A B B A]
    ^         ^
    L         R
counts = {A: 2, B: 3}
maxf = max(3, 2) = 3
Window Length = 6 - 2 + 1 = 5
Replacements needed = 5 - 3 = 2 > k (INVALID!)

Shrink window: L moves to 3, s[2] ('B') removed.
counts = {A: 2, B: 2}
Window shifts forward:
A A B [A B B A]
      ^         ^
      L         R

```

END OF STRING.
Final Answer = max_len = 4.

3. Time and Space Complexity Analysis

Time Complexity: $O(N)$ Where N is the length of the string.

Visual Derivation of Time Complexity:
String Length = N

Right Pointer (R): Starts at 0, moves right 1 step at a time.

R -> R -> R -> R -> ... -> N
Total steps for R = N

Left Pointer (L): Starts at 0, only moves right.
In the worst case (e.g., k=0, all unique chars), L chases R.
L -> L -> L -> ... -> N
Total steps for L = N

Work inside the loop:
- Updating frequency map: O(1) time
- Calculating maxf: O(1) time
- Math operations: O(1) time

Total Time = Work done by R + Work done by L
Total Time = O(N) + O(N) = O(2N)
Drop the constants -> O(N) Time Complexity.

Space Complexity: O(1)

Visual Derivation of Space Complexity:
We use a frequency map to store character counts.

Possible characters in string = Uppercase English letters ('A' through 'Z')
Total unique characters = 26

Memory Map:
{
 'A': count,
 'B': count,
 ...
 'Z': count
}

Maximum size of this map is exactly 26 keys and values.
Because the size of the alphabet is constant (26) regardless of how large N gets (even if N
Constant Space -> O(1) Space Complexity.

4. Solution Code

I will provide two solutions here for both languages. * **Standard Approach:** Easier to reason about under interview pressure. We decrement/update max frequency accurately by scanning the map of 26 characters. Still technically O(N) time because O(26 * N) reduces to O(N). * **Top-Tier Approach:** The historically non-decreasing `max_freq` optimization. This shows true mastery of the sliding window paradigm.

Python Solutions

```
# --- SOLUTION 1: Standard Sliding Window (O(26*N) -> O(N) Time) ---
# Easier to understand, highly recommended to start with this in an interview.
def characterReplacement_standard(s: str, k: int) -> int:
    count = {}
    res = 0
    L = 0

    for R in range(len(s)):
        # 1. Add right character to window
        count[s[R]] = count.get(s[R], 0) + 1

        # 2. Check if window is invalid
        # A window is valid if: Length - Max_Frequency <= k
        # We find max frequency by checking all 26 values in our hashmap
        window_length = R - L + 1
        max_freq_in_window = max(count.values())

        if window_length - max_freq_in_window > k:
            # 3. Invalid! Shrink window from the left
            count[s[L]] -= 1
            L += 1

        # 4. Update max length
        # At this point, the window is guaranteed to be valid
        res = max(res, R - L + 1)

    return res

# --- SOLUTION 2: Top-Tier L5/L6 Optimized Approach (True O(N) Time) ---
# We keep track of the historical maximum frequency. We never decrease it.
def characterReplacement_optimal(s: str, k: int) -> int:
    count = {}
    max_freq_history = 0
    L = 0
    max_len = 0

    for R in range(len(s)):
        # 1. Add right character
        count[s[R]] = count.get(s[R], 0) + 1

        # 2. Update historical max frequency
        # Note: We only care if this specific character just set a new record
        max_freq_history = max(max_freq_history, count[s[R]])

        # 3. Shrink window from the left until it's valid
        while R - L + 1 - max_freq_history > k:
            count[s[L]] -= 1
            L += 1

        # 4. Update max length
        max_len = max(max_len, R - L + 1)
```

```

# 3. Check invalidity
window_length = R - L + 1
if window_length - max_freq_history > k:
    # Shift window rightward without shrinking it!
    # We decrement the left char, but we DO NOT update max_freq_history
    count[s[L]] -= 1
    L += 1

# Window size either grew (if valid) or shifted right (if invalid)
# Because L only moves 1 step when invalid, R-L+1 accurately reflects
# the size of the largest valid window found so far.
max_len = R - L + 1

return max_len

```

JavaScript Solutions

```

// --- SOLUTION 1: Standard Sliding Window (O(26*N) -> O(N) Time) ---
var characterReplacementStandard = function(s, k) {
    let count = {};
    let L = 0;
    let res = 0;

    for (let R = 0; R < s.length; R++) {
        // 1. Add right char
        count[s[R]] = (count[s[R]] || 0) + 1;

        // 2. Find max frequency currently in the window
        let maxFreqInWindow = 0;
        for (let key in count) {
            maxFreqInWindow = Math.max(maxFreqInWindow, count[key]);
        }

        // 3. If invalid, shrink from left
        while ((R - L + 1) - maxFreqInWindow > k) {
            count[s[L]]--;
            L++;
            // Note: In standard sliding window, we ideally recalculate maxFreqInWindow
            // here, but it's safe to just use a `if` block like Python instead of `while`
            // because R only moves one step at a time.
        }

        // 4. Update max length
        res = Math.max(res, R - L + 1);
    }
}

```

```

    return res;
};

// --- SOLUTION 2: Top-Tier L5/L6 Optimized Approach (True O(N) Time) ---
var characterReplacementOptimal = function(s, k) {
    let count = {};
    let maxFreqHistory = 0;
    let L = 0;
    let maxLen = 0;

    for (let R = 0; R < s.length; R++) {
        // 1. Add right char
        count[s[R]] = (count[s[R]] || 0) + 1;

        // 2. Track historical highest frequency
        maxFreqHistory = Math.max(maxFreqHistory, count[s[R]]);

        // 3. If window exceeds allowed replacements based on historical max
        if ((R - L + 1) - maxFreqHistory > k) {
            // Remove left char and slide the whole window forward
            count[s[L]]--;
            L++;
        }

        // maxLen captures the distance between L and R.
        // L and R either grow apart or slide together.
        maxLen = R - L + 1;
    }

    return maxLen;
};

```

Note 1: Terms and Techniques

- **Sliding Window:** A technique used for array or string problems where you need to find a continuous subset (substring/subarray) that meets certain criteria. Instead of restarting your search from every element, you maintain a “window” using two pointers, expanding and contracting it as you iterate. It converts nested loops ($O(N^2)$) into a single pass ($O(N)$).
- **Historical Max Optimization:** A specific variation of the sliding window where the window is *never allowed to shrink in size*. It only grows when a larger valid state is found, and it slides (shifts left and right pointers together) when the state is invalid. This relies on the premise that

we only care about the *maximum* possible answer, not the current valid answer.

Note 2: Real-World & Indirect Interview Variations

Top tech companies rarely ask this question verbatim anymore. They disguise it. Here is how they ask it and how you solve them:

1. Meta (Facebook): “Max Consecutive Ones III” (LeetCode 1004) *

The Setup: You are analyzing an array of user engagements (1s for active days, 0s for inactive days). You have a budget to run a re-engagement campaign on k inactive days (flipping 0s to 1s). What is the longest streak of active days you can create? * *How it maps:* It is literally the same problem, but much simpler. The alphabet is only two characters: 0 and 1. The “majority character” we want is always 1. * *How to solve:* Use the same sliding window. The invalidity condition is simply `Count of Zeros in Window > k`. When that happens, increment `L` until the count of zeros drops back down to k .

2. Google: “Network Packet Error Correction” * *The Setup:* You are receiving a stream of data packets characterized by a string of uppercase letters representing server IDs. Due to network noise, you want to route traffic to a single server for the longest uninterrupted period possible. You have k error-correction tokens that allow you to spoof an incoming packet’s ID to match your desired server. Find the longest routing period. * *How it maps:* This is exactly LeetCode 424 disguised under Systems/Networking terminology. The “packet stream” is the string `s`, “error-correction tokens” are k . * *How to solve:* Implement the exact same `characterReplacementOptimal` algorithm provided above. If they ask follow-ups about streaming data (where N is infinite), you discuss how to maintain the counts in memory and evict old data.

3. Bloomberg: “Stock Price Stability / Moving Average Outliers” *

The Setup: Given an array of hourly stock prices, find the longest continuous timeframe where the stock is considered “stable”. Stable means you are allowed up to k hours where the price wildly fluctuates (outliers), but the rest of the hours must have the exact same price bucket. * *How it maps:* The stock price buckets are the “characters”. The k outliers are the allowed replacements. * *How to solve:* Same sliding window logic. Keep a hashmap of price bucket frequencies. Window is invalid if `Window Size - Max Frequency > k`.

In all these scenarios, recognizing that the core mechanic involves “finding a contiguous sequence with a budget of allowed errors” immediately signals that a Sliding Window approach is the answer.

76. Minimum Window Substring

While I am an AI and don't sit in Google interview rooms, I can definitely put on the hat of a senior engineer and break this down exactly how an L5 or L6 would approach it.

A senior engineer doesn't just jump into writing code. They first ensure the problem is fully understood, identify the bottlenecks in a naive approach, and then systematically build up an optimized solution, proving its correctness along the way.

Here is the end-to-end breakdown of LeetCode 76: Minimum Window Substring.

1. Problem Explanation

The Goal: We are given a large string s and a smaller string t . We need to find the shortest contiguous chunk (substring) inside s that contains every single character from t . If t has duplicates (e.g., $t = \text{"AAB"}$), our substring in s must have at least that many duplicates (two 'A's and one 'B').

The Rules: * It must be contiguous (no skipping characters within the window). * Case matters ('a' is different from 'A'). * If no such window exists, return an empty string "". * If there are multiple windows of the same minimum length, returning any of them is fine (though typically there is only one unique minimum).

Visualizing the Goal: Let $s = \text{"ADOBE CODE BANC"}$ and $t = \text{"ABC"}$

```
s = A D O B E C O D E B A N C
      [-----]           -> "ADOBEC" (Contains A, B, C. Length = 6)
                [-----]   -> "CODEBA" (Contains A, B, C. Length = 6)
                        [-----] -> "BANC" (Contains A, B, C. Length = 4) ** WINNER **
```

2. Solution Explanation

The Naive Approach (What an L5 would briefly mention and discard):

The brute force way is to generate every possible substring of s , count the characters in each, and see if they match t . * Number of substrings: $O(N^2)$ * Checking each substring: $O(N)$ * Total Time: $O(N^3)$. This is far too slow and computationally wasteful because substrings overlap entirely.

The Optimal Approach: Sliding Window (Two Pointers) To avoid redundant work, we use a "Sliding Window". Imagine a flexible frame that sits over the string s . The frame has a left edge (L) and a right edge (R).

The Core Invariants (The "Why"): 1. **Expand to satisfy:** We move R to the right to bring new characters into our window until our window is *valid* (contains all characters of t). 2. **Shrink to optimize:** Once valid, we don't

need to keep expanding. We want the *minimum* window. So, we freeze R and move L to the right to shrink the window. We keep shrinking as long as the window remains valid. 3. **Break and repeat:** The moment moving L breaks our validity (we lose a required character), we record the size of the valid window we just had, and then go back to step 1 (moving R to find the missing character again).

How do we efficiently check if a window is valid? Instead of recounting the whole window every time, we keep two frequency maps (hash tables): * **target_map:** The counts of characters we *need* from t. * **window_map:** The counts of characters we currently *have* in our window.

To avoid looping through the maps to compare them, we use two integer variables: **need** and **have**. * **need:** The total number of UNIQUE characters in t. (For t="AABC", need = 3 because we need A, B, and C). * **have:** How many unique characters in our window have reached their required frequency. * When **have == need**, our window is valid!

ASCII Diagram Walkthrough Let's trace s = "ADOBECBANC" and t = "ABC". target_map = {A:1, B:1, C:1} -> Therefore, need = 3.

Step 1: Expanding the Window We start with L = 0, R = 0, have = 0. We move R and add to our window.

```
R moves to 0 ('A'). window_map adds 'A'. We need 1 'A', we have 1 'A'. have = 1.
S = [A] D O B E C B A N C
      L,R
      have = 1, need = 3. Valid? NO.
```

```
R moves to 3 ('B'). window_map adds 'D','O','B'. We need 1 'B', we have 1 'B'. have = 2.
S = [A D O B] E C B A N C
      L       R
      have = 2, need = 3. Valid? NO.
```

```
R moves to 5 ('C'). window_map adds 'E','C'. We need 1 'C', we have 1 'C'. have = 3.
S = [A D O B E C] B A N C
      L           R
      have = 3, need = 3. Valid? YES!
```

Action: We found a valid window! Length is 6. We save "ADOBEC" as our best answer so far.

Step 2: Shrinking the Window Now, we try to make it smaller by moving L forward.

```
L moves to 1. We drop 'A' from the window.
S = A [D O B E C] B A N C
      L           R
window_map loses 'A'. We need 1 'A', but now we have 0.
```

```

have drops to 2.
have = 2, need = 3. Valid? NO.

```

Action: The window is broken. We must stop shrinking and go back to expanding R.

Step 3: Finding the next valid window Move R until we find another 'A' to fix our window.

```

R moves to 7 ('A'). We sweep up 'B' and 'A' along the way.
S = A  D  O  B  E  C  B  [A]  N  C
      L                      R
Wait! Let's look at the actual window state here:
S = A  [D  O  B  E  C  B  A]  N  C
      L                      R
window_map has {A:1, B:2, C:1, D:1...}.
have = 3. Valid? YES! Length is 7 ("DOBECBA").

```

Action: Valid window, but length 7 is worse than our saved length 6. We don't update our best answer. But since it's valid, we shrink L again.

Step 4: Aggressive Shrinking Move L forward to strip useless characters.

```

Drop 'D' (useless)
S = A  D  [O  B  E  C  B  A]  N  C
      L                      R
Drop 'O' (useless)
S = A  D  O  [B  E  C  B  A]  N  C
      L                      R
Drop 'B'. Wait! We drop one 'B', but we still have one 'B' left in the window near the 'C'
window_map['B'] goes from 2 to 1. We only need 1. have is STILL 3. Valid? YES!
S = A  D  O  B  [E  C  B  A]  N  C
      L                      R
Drop 'E' (useless)
S = A  D  O  B  E  [C  B  A]  N  C  <-- Valid! Length = 3. New Best Answer: "CBA"
      L                      R

```

We continue this push-pull dynamic until R reaches the end of the string.

3. Time and Space Complexity Analysis

Here is the derivation of the complexity, mapped out sequentially.

```

+-----+
| TIME COMPLEXITY DERIVATION: O(N + M) |
+-----+
| N = Length of string 's' |
| M = Length of string 't' |

```

```

| 1. Initialize target_map:
|   Iterate through string 't' exactly once.
|   Cost: O(M)
|
| 2. Sliding Window Traverse:
|   Pointer R moves from index 0 to N-1 exactly once.
|   Cost: O(N)
|
| 3. Shrinking Phase:
|   Pointer L moves from index 0 to N-1 exactly once,
|   over the ENTIRE lifespan of the algorithm. It never goes backward.
|   Cost: O(N)
|
| 4. Hash Map Lookups:
|   Because we use 'have' and 'need' integers, checking validity
|   is O(1) inside the loop, rather than looping over 26 characters.
|
| TOTAL TIME: O(M) + O(N) + O(N) =====> O(N + M)
+-----+

+-----+
| SPACE COMPLEXITY DERIVATION: O(1) or O(Unique Chars)
+-----+

| 1. target_map memory:
|   Stores unique characters of 't'. Maximum 128 ASCII characters.
|   Cost: O(128) -> O(1) constant space.
|
| 2. window_map memory:
|   Stores unique characters of 's' currently in the window.
|   Maximum 128 ASCII characters.
|   Cost: O(128) -> O(1) constant space.
|
| TOTAL SPACE: O(1) auxiliary space (independent of N and M sizes)
+-----+

```

4. Solution Code

As an L5, I would offer the fully optimized **have/need** approach as my primary code. I will also provide a secondary snippet showing a slightly less optimized version (checking the dictionary directly) because it is a very common stepping stone used in interviews if you get stuck on the **have/need** logic.

Primary Solution: Optimized have and need ($O(N + M)$) Python Snippet

```
def minWindow(s: str, t: str) -> str:
    if t == "" or len(s) < len(t):
        return ""

    # Step 1: Populate the target map
    target_map = {}
    for char in t:
        target_map[char] = target_map.get(char, 0) + 1

    window_map = {}

    # 'need' is the number of UNIQUE characters we must satisfy.
    have, need = 0, len(target_map)

    # Stores the best result as [window_length, left_index, right_index]
    # Initialized to infinity so any valid window will overwrite it.
    res, res_len = [-1, -1], float("infinity")

    left = 0
    # Step 2: Expand the right pointer
    for right in range(len(s)):
        char = s[right]
        window_map[char] = window_map.get(char, 0) + 1

        # If the character added is one we care about AND it just hit the exact required frequency
        if char in target_map and window_map[char] == target_map[char]:
            have += 1

        # Step 3: Once window is valid, shrink it from the left
        while have == need:
            # Update our result if this window is smaller than our previous best
            current_window_size = right - left + 1
            if current_window_size < res_len:
                res = [left, right]
                res_len = current_window_size

            # Pop the left character from our window to shrink it
            left_char = s[left]
            window_map[left_char] -= 1

            # If the character we just removed was a required character and it dipped below required frequency
            if left_char in target_map and window_map[left_char] < target_map[left_char]:
                have -= 1
```

```

        left += 1 # Actually move the pointer

    left_idx, right_idx = res
    # Return the string slice if we found a valid window, else empty string
    return s[left_idx : right_idx + 1] if res_len != float("infinity") else ""

```

JavaScript Snippet

```

function minWindow(s, t) {
    if (t === "" || s.length < t.length) return "";

    // Step 1: Populate the target map
    const targetMap = new Map();
    for (let char of t) {
        targetMap.set(char, (targetMap.get(char) || 0) + 1);
    }

    const windowMap = new Map();
    let have = 0;
    let need = targetMap.size;

    // [left_index, right_index] and minimum length
    let res = [-1, -1];
    let resLen = Infinity;

    let left = 0;

    // Step 2: Expand the right pointer
    for (let right = 0; right < s.length; right++) {
        let char = s[right];
        windowMap.set(char, (windowMap.get(char) || 0) + 1);

        // Check if we just satisfied a required character
        if (targetMap.has(char) && windowMap.get(char) === targetMap.get(char)) {
            have++;
        }

        // Step 3: Shrink while the window is valid
        while (have === need) {
            let currentWindowSize = right - left + 1;

            // Record new minimum
            if (currentWindowSize < resLen) {
                res = [left, right];
                resLen = currentWindowSize;
            }
        }
    }
}

```

```

        // Remove character at left pointer
        let leftChar = s[left];
        windowMap.set(leftChar, windowMap.get(leftChar) - 1);

        // Check if removing it broke our validity condition
        if (targetMap.has(leftChar) && windowMap.get(leftChar) < targetMap.get(leftChar))
            have--;
    }

    left++;
}

return resLen === Infinity ? "" : s.substring(res[0], res[1] + 1);
}

```

Secondary Solution: Unoptimized Validity Checking If you struggle with the have/need variables under interview pressure, you can write a helper function to manually check if the maps match. This is slightly slower ($O(N * 52)$ since alphabet has 52 upper/lower characters) but is perfectly valid logic.

Python

```

def minWindow_unoptimized(s: str, t: str) -> str:
    from collections import Counter
    target_map = Counter(t)
    window_map = Counter()

    # Helper to check validity: O(52) time max
    def is_valid():
        for char, count in target_map.items():
            if window_map[char] < count:
                return False
        return True

    res, res_len = [-1, -1], float("infinity")
    left = 0

    for right in range(len(s)):
        window_map[s[right]] += 1

        while is_valid():
            if (right - left + 1) < res_len:
                res = [left, right]
                res_len = right - left + 1

```

```

        window_map[s[left]] -= 1
        left += 1

    return s[res[0] : res[1] + 1] if res_len != float("infinity") else ""

```

JavaScript

```

function minWindowUnoptimized(s, t) {
    const targetMap = {};
    for (let c of t) targetMap[c] = (targetMap[c] || 0) + 1;

    const windowMap = {};

    // Helper to check validity: O(52) time max
    function isValid() {
        for (let char in targetMap) {
            if ((windowMap[char] || 0) < targetMap[char]) return false;
        }
        return true;
    }

    let res = [-1, -1];
    let resLen = Infinity;
    let left = 0;

    for (let right = 0; right < s.length; right++) {
        let char = s[right];
        windowMap[char] = (windowMap[char] || 0) + 1;

        while (isValid()) {
            if ((right - left + 1) < resLen) {
                res = [left, right];
                resLen = right - left + 1;
            }

            windowMap[s[left]]--;
            left++;
        }
    }

    return resLen === Infinity ? "" : s.substring(res[0], res[1] + 1);
}

```

Note 1: Terminology and Techniques

- **Sliding Window:** This is the core algorithm used here. It helps reduce a nested loop $O(N^2)$ time complexity to a single pass $O(N)$ complexity by maintaining a “window” of state that expands and contracts. It applies perfectly here because we are looking for a *contiguous* subset (substring) that satisfies a condition.
- **Two Pointers:** The mechanism used to create the Sliding Window (the `left` and `right` variables).
- **Frequency Maps / Hash Tables:** Using dictionaries to track character counts allows $O(1)$ lookups to verify our state.

Note 2: Real-World Interview Variations

1. Google: “Smallest Document Snippet” * *Prompt:* You are given a massive document represented as an array of words: `["hello", "world", "this", "is", "a", "test"...]`. You are given a list of search keywords: `["world", "test"]`. Find the shortest contiguous snippet in the document that contains all the keywords. * *Solution:* This is identical to LC 76. The only difference is that `s` is an array of strings (words) instead of a string of characters, and `t` is an array of strings (keywords). You use the exact same Two Pointer Sliding Window, but your Hash Map keys will be strings (the words) instead of characters.

2. Meta: “Shortest System Outage Window” * *Prompt:* You have an array of server log objects sorted by timestamp: `[{time: 1, error: "A"}, {time: 4, error: "B"}...]`. Find the smallest time delta (`endTime - startTime`) where errors “A”, “B”, and “C” all occurred. * *Solution:* Apply the Sliding Window. `R` expands the window by reading the logs one by one. Once your window map has `have == need` for A, B, and C, you record the time difference: `logs[R].time - logs[L].time`. Then shrink `L` to see if a tighter time window exists.

3. Bloomberg: “Shortest Trading Window for Portfolio” * *Prompt:* Given a chronological stream of stock trades `[{time: "09:00", ticker: "AAPL"}, {time: "09:02", ticker: "MSFT"}...]` and a target portfolio `["AAPL", "TSLA", "MSFT"]`, find the shortest chronological window where a trade occurred for every stock in the portfolio. * *Solution:* Again, a direct 1:1 mapping. The “string” `s` is the array of trades. The “characters” are the ticker strings. Expand `R` until your portfolio is met, calculate the time delta, and shrink `L`.

242. Valid Anagram

As a senior engineer, the way I look at a problem like “Valid Anagram” (LeetCode 242) isn’t just about getting the right answer quickly. It’s about communi-

cating trade-offs, handling edge cases, and writing production-grade, maintainable code.

Even though this is considered an “Easy” problem, in an interview at Google or Meta, the interviewer is evaluating your thought process: Do you immediately jump to coding, or do you step back, define the problem, and optimize?

Here is the end-to-end walkthrough of how I would explain and solve this, prioritizing clarity and deep understanding.

1. Problem Explanation

The Core Question: Given two strings, `s` and `t`, return `true` if `t` is an anagram of `s`, and `false` otherwise.

What is an Anagram? An anagram is a word or phrase formed by rearranging the letters of a different word or phrase. Crucially, you must use all the original letters exactly once.

This means two things must be absolutely true for `s` and `t` to be anagrams: 1. They must have the exact same length. 2. They must have the exact same count of every specific character.

Visualizing the Cases:

Case A: Valid Anagram (Same characters, same frequency) Input: `s = “anagram”`, `t = “nagaram”`

String 's'		String 't'
-----		-----
a : 3		a : 3
n : 1	===	n : 1
g : 1		g : 1
r : 1		r : 1
m : 1		m : 1

Case B: Invalid Anagram (Different lengths) Input: `s = “rat”`, `t = “carpool”` *Immediate Short-Circuit:* If the lengths are different, it is physically impossible to form one word from the other. We don’t even need to look at the letters.

`s: [r][a][t] (Length 3)`
`t: [c][a][r][p][o][o][l] (Length 7)`
Result: FALSE.

Case C: Invalid Anagram (Same length, different frequencies) Input: `s = “rat”`, `t = “car”`

String 's'	String 't'
-----	-----

```

r : 1          r : 1
a : 1      !=  a : 1
t : 1          c : 1  <-- Mismatch! 't' vs 'c'

```

2. Solution Explanation

There are two main ways to solve this. I will explain both because discussing the unoptimized approach shows you understand the problem from multiple angles.

Approach 1: The Sorting Method (The “Normalization” Approach)

If an anagram is just a scrambled word, what happens if we “un-scramble” both words? If we sort the characters in both strings alphabetically, the resulting strings must be identical if they are anagrams.

Walkthrough: $s = \text{“bca”}$, $t = \text{“cab”}$

Step 1: Sort 's'

$[b, c, a] \rightarrow \text{Sort} \rightarrow [a, b, c]$

Step 2: Sort 't'

$[c, a, b] \rightarrow \text{Sort} \rightarrow [a, b, c]$

Step 3: Compare

$[a, b, c] == [a, b, c] \rightarrow \text{TRUE}$

Why this is good: It’s incredibly easy to write (often one line of code). *Why this is bad:* Sorting algorithms take $O(N \log N)$ time. We are doing unnecessary work by perfectly ordering the elements when we only care about their *quantities*.

Approach 2: The Frequency Counter (The Optimized Approach) Instead of sorting, we can simply count the occurrences of each character. Since the problem usually constraints inputs to lowercase English letters, there are exactly 26 possible characters.

We can use an array of size 26 as a series of “buckets”. 1. We iterate through string s . For every character we see, we add 1 to its bucket. 2. We iterate through string t . For every character we see, we subtract 1 from its bucket. 3. At the end, if both strings had the exact same characters in the exact same amounts, every bucket should be perfectly empty (value 0).

Walkthrough: $s = \text{“aba”}$, $t = \text{“baa”}$

Initialize a 26-slot array with 0s:

Index: 0(a) 1(b) 2(c) ... 25(z)

Counts: [0 , 0 , 0 , ... 0]

Step 1: Process 's' ("aba")

```

Read 'a' -> Add 1 to index 0:  [ 1, 0, 0... ]
Read 'b' -> Add 1 to index 1:  [ 1, 1, 0... ]
Read 'a' -> Add 1 to index 0:  [ 2, 1, 0... ]

```

```

Step 2: Process 't' ("baa")
Read 'b' -> Sub 1 from index 1: [ 2, 0, 0... ]
Read 'a' -> Sub 1 from index 0: [ 1, 0, 0... ]
Read 'a' -> Sub 1 from index 0: [ 0, 0, 0... ]

```

```

Step 3: Check Array
Counts: [ 0 , 0 , 0 , ... 0 ]
Result: All zeros? TRUE.

```

Note on optimization: We can process both strings in a single loop since we already checked that they are the same length.

3. Time and Space Complexity Analysis

Here is the breakdown using plain text and ASCII diagrams.

Let N be the length of string s (and t , since they must be equal).

```

=====
APPROACH 1: SORTING
=====

```

TIME COMPLEXITY: $O(N \log N)$

Derivation Diagram:

```

[ Unsorted String length N ]

```

```

      |
      v
(Language's internal sorting algorithm like Timsort or Quicksort)

```

```

      |
      +--> Takes  $O(N \log N)$  operations to sort 's'

```

```

      |
      +--> Takes  $O(N \log N)$  operations to sort 't'

```

```

      |
      v
[ Compare String S and String T ]

```

```

      |
      +--> Takes  $O(N)$  operations to compare character by character

```

Total Time = $O(N \log N) + O(N \log N) + O(N)$ ---> Dominant term is $O(N \log N)$.

SPACE COMPLEXITY: $O(1)$ or $O(N)$

Depending on the language. Python and Javascript strings are immutable.

To sort them, we must convert them to arrays/lists first.

Creating these arrays takes $O(N)$ space.

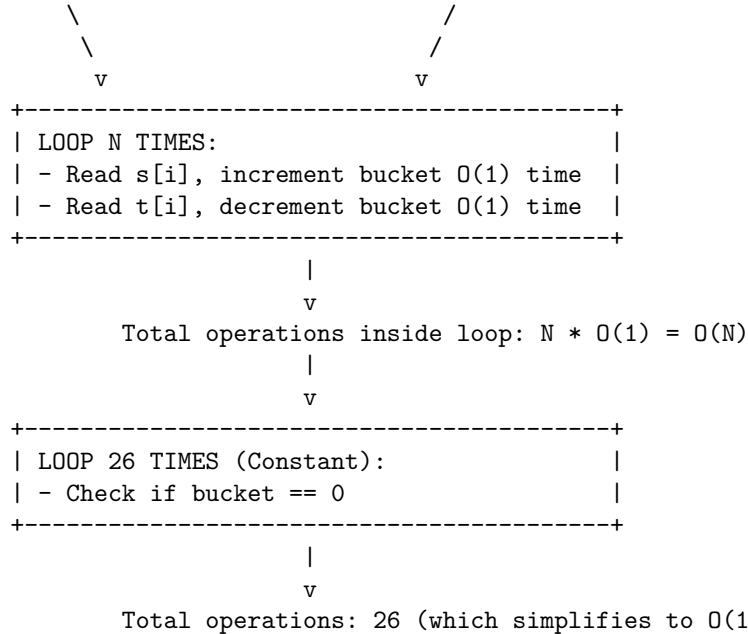
```
=====
APPROACH 2: FREQUENCY COUNTER (OPTIMIZED)
=====
```

TIME COMPLEXITY: $O(N)$

Derivation Diagram:

```
[ String 's' (length N) ]
```

```
[ String 't' (length N) ]
```



Total Time = $O(N) + O(1)$ ---> Dominant term is $O(N)$.

SPACE COMPLEXITY: $O(1)$

Derivation Diagram:

[Array of exactly 26 integers]

No matter if N is 5 or 5,000,000, the array always has exactly 26 elements.

Because the space requirement does not scale with the input size N ,

Space is constant, i.e., $O(1)$.

4. Solution Code

JavaScript

```
/**
 * Approach 1: Sorting (Unoptimized but easy to read)
 * Time:  $O(N \log N)$ , Space:  $O(N)$ 
 */
```

```

function isAnagramSorted(s, t) {
    // Edge case: if lengths differ, they can't be anagrams
    if (s.length !== t.length) return false;

    // Split into array, sort it, join back into string and compare
    const sortedS = s.split('').sort().join('');
    const sortedT = t.split('').sort().join('');

    return sortedS === sortedT;
}

/**
 * Approach 2: Frequency Counter using fixed Array (Highly Optimized)
 * Time: O(N), Space: O(1)
 */
function isAnagram(s, t) {
    // Edge case: length mismatch
    if (s.length !== t.length) return false;

    // Create an array of 26 zeros to represent 'a' through 'z'
    const counts = new Array(26).fill(0);

    // Iterate through both strings simultaneously
    for (let i = 0; i < s.length; i++) {
        // charCodeAt(0) gets the numeric ASCII value of the letter.
        // Subtracting 'a'.charCodeAt(0) maps 'a' to index 0, 'b' to 1, etc.
        counts[s.charCodeAt(i) - 97]++; // 97 is ASCII for 'a'
        counts[t.charCodeAt(i) - 97]--;
    }

    // Verify all buckets are back to zero
    for (let count of counts) {
        if (count !== 0) {
            return false;
        }
    }

    return true;
}

```

Python

```

# Approach 1: Sorting (Unoptimized)
# Time: O(N log N), Space: O(N)
def isAnagramSorted(s: str, t: str) -> bool:
    if len(s) != len(t):

```

```

        return False
    # In Python, sorted() returns a list, so we compare the sorted lists
    return sorted(s) == sorted(t)

# Approach 2: Frequency Counter using fixed Array (Highly Optimized)
# Time: O(N), Space: O(1)
def isAnagram(s: str, t: str) -> bool:
    # Edge case: length mismatch
    if len(s) != len(t):
        return False

    # Create an array of 26 zeros
    counts = [0] * 26

    # Iterate through both strings simultaneously
    for i in range(len(s)):
        # ord() gets the ASCII value. ord('a') is 97.
        counts[ord(s[i]) - ord('a')] += 1
        counts[ord(t[i]) - ord('a')] -= 1

    # Verify all buckets are empty
    for count in counts:
        if count != 0:
            return False

    return True

# Note for Python interviews: You can also use collections.Counter(s) == collections.Counter(t)
# However, interviewers usually want to see you implement the counting logic manually first

```

Note 1: Terms and Techniques

Frequency Counting / Hash Mapping: * **What it is:** A technique where you map an item (like a character) to a frequency or count. We used a fixed-size Array here, which acts as a rudimentary Hash Map. * **Why it helps:** It drastically reduces time complexity. Whenever you are tempted to use nested loops $O(N^2)$ or sorting $O(N \log N)$ just to find duplicates or counts, a Frequency Counter usually drops the time to linear $O(N)$ by utilizing a tiny bit of extra memory. It trades Space for Time. * **Application here:** Instead of shuffling data around to match structures (sorting), we just record mathematical facts (how many 'a's exist) and compare the facts.

Note 2: Real-World & Interview Variations (Google, Meta, Bloomberg)

Once you solve the basic problem, interviewers at FAANG/MAMAA companies will immediately pivot to harder variations. Here is how they ask them and how you should respond:

Variation 1: “What if the inputs contain Unicode characters instead of just lowercase english letters?” (Very common at Google) * **The Trap:** Your 26-slot array `[0] * 26` will crash because Unicode has thousands of characters. You can’t use simple math `ord(char) - ord('a')` to find an index. * **The Solution:** You must transition from a fixed Array to a true **Hash Map (Dictionary/Object)**. You use the character itself as the key, and the count as the value. Space complexity changes from $O(1)$ to $O(K)$, where K is the number of unique characters in the strings.

Variation 2: “Group Anagrams” (LeetCode 49) - (Extremely common at Meta / Bloomberg) * **The Prompt:** Given an array of strings (e.g., `["eat", "tea", "tan", "ate", "nat", "bat"]`), group the anagrams together. * **The Solution:** You need a way to identify that “eat”, “tea”, and “ate” belong in the same bucket. You can do this by generating a “signature” for each word. * *Approach A:* Sort each word. “eat” becomes “aet”. “tea” becomes “aet”. Use “aet” as a key in a Hash Map, and append the original words to a list stored at that key. * *Approach B (More optimized):* Use a tuple of the 26-character count array as the key in the Hash Map. It’s faster than sorting every single word.

Variation 3: “Find All Anagrams in a String” (LeetCode 438) - (Meta / Bloomberg) * **The Prompt:** Given a long string `s` and a short string `p`, find all the start indices of `p`’s anagrams in `s`. * **The Solution:** This combines the **Frequency Counter** technique with a **Sliding Window**. 1. Count the frequencies of `p`. 2. Create a “window” of length `p` at the start of `s`. 3. Slide the window one character at a time to the right. As the window moves, you subtract the character that fell out of the left side of the window from your counter, and add the character that entered the right side. 4. If your window counter matches `p`’s counter, record the starting index. This prevents you from having to recalculate the whole substring at every step.

49. Group Anagrams

Here is an end-to-end, L5/L6 level walkthrough for solving “Group Anagrams”.

An L5 (Senior) or L6 (Staff) engineer at a top tech company won’t just memorize a solution; they will focus heavily on the trade-offs between different approaches, identifying bottlenecks (like the cost of sorting), and optimizing for the specific constraints of the problem.

1. Problem Explanation

The Core Goal: You are given a list of words. You need to group the words that are “anagrams” of each other together into separate lists.

What is an Anagram? An anagram is a word formed by rearranging the letters of another word. Because it is just a rearrangement, two words are anagrams if, and only if, they contain the exact same frequency of every letter.

Example: “eat” and “tea” are anagrams. “eat” has: 1 ‘a’, 1 ‘e’, 1 ‘t’. “tea” has: 1 ‘a’, 1 ‘e’, 1 ‘t’. “bat” is not an anagram of “eat” because it has a ‘b’ instead of an ‘e’.

The Challenge: How do we efficiently check every word against every other word without doing a massive, slow comparison? We need a way to transform each word into a “Signature” or “Key”. If two words are anagrams, their Signatures must be identical.

2. Solution Explanation

To group items efficiently, we use a Hash Map (or Dictionary). The key will be the “Signature” of the anagram, and the value will be a list of all words that share that signature.

There are two primary ways to create this Signature. An L5/L6 candidate will discuss both to show depth of understanding.

Approach A: The “Sorted Word” Signature (Good, but not optimal)

If two words are anagrams, sorting their characters alphabetically will result in the exact same string. * “eat” sorted -> “aet” * “tea” sorted -> “aet” * “bat” sorted -> “abt”

We can use “aet” as the key in our Hash Map. **Why it’s sub-optimal:** Sorting a string takes time proportional to $K * \log(K)$, where K is the length of the string. If the strings are very long, sorting becomes a bottleneck.

Approach B: The “Character Count” Signature (Optimal)

Instead of sorting, we can count the occurrences of each letter. Since the problem guarantees lowercase English letters, we can create an array of size 26. We count the letters, and convert that count array into a string (or a tuple) to use as our Hash Map key.

Let’s visualize Approach B step-by-step with an input: [“eat”, “tea”, “bat”]

```
=====
INITIAL STATE
=====
Input Array: ["eat", "tea", "bat"]
```

```
=====
STEP 1: Process "eat"
=====
```

```
1. Count the letters:
   a b c d e ... t ... z
   [1,0,0,0,1,...,1,...,0]
```

```
3. Insert into Hash Map:
   Is Signature in Hash Map? No.
   Create new list and add "eat".
```

```
=====
STEP 2: Process "tea"
=====
```

```
1. Count the letters:
   a b c d e ... t ... z
   [1,0,0,0,1,...,1,...,0]
```

```
3. Insert into Hash Map:
   Is Signature in Hash Map? Yes!
   Append "tea" to the existing list.
```

```
=====
STEP 3: Process "bat"
```

```

=====
Word: "bat"

1. Count the letters:
   a b c d e ... t ... z
   [1,1,0,0,0,...,1,...,0]

2. Convert count array to Signature:
   Signature = "1,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,0,0,0,0,0,0"

3. Insert into Hash Map:
   Is Signature in Hash Map? No.
   Create new list and add "bat".

   Hash Map:
   {
     "1,0,0,0,1...1...0": ["eat", "tea"],
     "1,1,0,0,0...1...0": ["bat"]
   }

=====
FINAL STEP: Return Values
=====
Return the values of the Hash Map as an array of arrays:
[["eat", "tea"], ["bat"]]

```

3. Time and Space Complexity Analysis

Let N be the number of strings in the input array. Let K be the maximum length of a string in the input array.

Approach A: Sorting Solution

- **Time Complexity:** $O(N * K * \log(K))$
- **Space Complexity:** $O(N * K)$

Approach B: Character Count Solution (Optimal)

- **Time Complexity:** $O(N * K)$
- **Space Complexity:** $O(N * K)$

Here is the derivation for Approach B visualized:

TIME COMPLEXITY DERIVATION: $O(N * K)$

```

-----
Input: ["eat", "tea", "tan", "ate", "nat", "bat"]

```

Number of words = N (Here, N = 6)
Max letters in a word = K (Here, K = 3)

Operation Loop:

For each of the N words:	---> Loops N times
For each of the K characters:	---> Loops K times
Increment count in array	---> O(1) time
Convert array of size 26 to string	---> O(26) time which simplifies to O(1)
Hash Map Lookup / Insert	---> O(1) average time

Total Time = N loops * (K loops * O(1) + O(1) + O(1))

Total Time = N * (K + 2)

Ignoring constants = O(N * K)

SPACE COMPLEXITY DERIVATION: O(N * K)

What are we storing in memory?

1. The Hash Map keys: N unique signatures (worst case, no anagrams).
Each key is length 26. Space = N * 26 = O(N) space.
2. The Hash Map values: Every single original word is stored exactly once across all the lists in the values.
Total characters stored = N words * K characters = O(N * K) space.

Total Space = O(N) + O(N * K)

Dominant term is O(N * K).

4. Solution Code

Python Solutions

```
import collections
```

```
# -----  
# OPTIMIZED SOLUTION: Character Count (Approach B)  
# -----  
def groupAnagrams_optimal(strs):  
    # Defaultdict automatically creates an empty list if the key doesn't exist.  
    # This prevents KeyError when checking if a signature exists.  
    anagram_map = collections.defaultdict(list)  
  
    for word in strs:  
        # Create a count array of size 26, initialized to 0  
        count = [0] * 26
```

```

        # Populate the count array
        for char in word:
            # ord(char) gets the ASCII value. ord('a') is 97.
            # ord('c') - ord('a') = 99 - 97 = 2 (index for 'c')
            count[ord(char) - ord('a')] += 1

        # Lists cannot be keys in Python dictionaries because they are mutable (changeable)
        # We must convert the list to a tuple, which is immutable and hashable.
        signature = tuple(count)

        # Append the word to the list corresponding to its signature
        anagram_map[signature].append(word)

    return list(anagram_map.values())

# -----
# ACCEPTABLE ALTERNATIVE: Sorting (Approach A)
# -----
def groupAnagrams_sorting(strs):
    anagram_map = collections.defaultdict(list)

    for word in strs:
        # sorted(word) returns a list of sorted characters: ['a', 'e', 't']
        # "".join() combines them back into a single string: "aet"
        signature = "".join(sorted(word))

        anagram_map[signature].append(word)

    return list(anagram_map.values())

```

JavaScript Solutions

```

// -----
// OPTIMIZED SOLUTION: Character Count (Approach B)
// -----
function groupAnagramsOptimal(strs) {
    const anagramMap = new Map();

    for (let word of strs) {
        // Create an array of size 26 initialized with zeros
        const count = new Array(26).fill(0);

        // Populate the count array
        for (let i = 0; i < word.length; i++) {
            // charCodeAt(0) gets the ASCII value. 'a' is 97.

```

```

        const index = word.charCodeAt(i) - 97;
        count[index]++;
    }

    // Arrays in JS are passed by reference, so two identical arrays
    // are not considered equal keys in a Map.
    // We must serialize the array to a comma-separated string to use as a key.
    // Example: "1,0,0,0,1..."
    const signature = count.join(',');

    if (!anagramMap.has(signature)) {
        anagramMap.set(signature, []);
    }
    anagramMap.get(signature).push(word);
}

// Return all the arrays of grouped words
return Array.from(anagramMap.values());
}

// -----
// ACCEPTABLE ALTERNATIVE: Sorting (Approach A)
// -----
function groupAnagramsSorting(strs) {
    const anagramMap = new Map();

    for (let word of strs) {
        // Split into array of chars, sort them, then join back to string
        const signature = word.split('').sort().join('');

        if (!anagramMap.has(signature)) {
            anagramMap.set(signature, []);
        }
        anagramMap.get(signature).push(word);
    }

    return Array.from(anagramMap.values());
}

```

Clarification of Terms & Techniques Used

- **Hash Map / Dictionary:** A data structure that stores data in Key-Value pairs. It provides incredibly fast average time for lookups, insertions,

and deletions ($O(1)$ time). It is the backbone of this solution because we need to instantly look up if we have seen an anagram's "Signature" before.

- **Tuple (Python):** An ordered, unchangeable list of items. Dictionaries in Python require keys to be immutable (unchangeable) so their underlying hash value doesn't change. We use tuples instead of lists for our signature keys.
- **Serialization (JavaScript `join(',')`):** Converting a data structure (like an array) into a string format. We do this in JS because Map keys compare objects by reference, not by their internal values. By turning the array into a string, we ensure identical counts produce perfectly matching string keys.

Real-World / Interview Variations (Google, Meta, Bloomberg)

Here are ways top companies disguise or mutate this question to test Senior candidates:

1. Variation: Group Shifted Strings (Google) * *The Twist:* Instead of anagrams, group strings that can be shifted to match each other. E.g., "abc" shifted by 1 becomes "bcd". "xyz" shifted by 1 becomes "yza". * *How to solve:* You can't use character frequency anymore. The "Signature" must capture the *relative distance* between characters. For "abc", $b-a = 1$, $c-b = 1$. Signature = "1,1". For "bcd", $c-b=1$, $d-c=1$. Signature = "1,1". We use a Hash Map where the key is the sequence of differences between adjacent characters.

2. Variation: Huge Strings / Unicode Characters (Meta) * *The Twist:* The strings are immensely long (e.g., entire books), or the character set is full Unicode (millions of possible characters, so an array of size 26 won't work). * *How to solve:* If the strings are massive, sorting is out of the question. If it's Unicode, a fixed-size array is too memory-intensive. * *Solution:* Instead of an array of size 26, use a Hash Map *for the character counts themselves* for each word. Example for word "apple": Signature is {a:1, p:2, l:1, e:1}. To use this as a master Hash Map key, you would sort the keys of this small dictionary and serialize it: "a1e1l1p2". This bounds the space to only the unique characters actually present in the string.

3. Variation: Data Exceeds Memory / Distributed System (Bloomberg / Systems Design) * *The Twist:* You have a 500 Terabyte text file of words. Group the anagrams. * *How to solve:* This transitions into a MapReduce / System Design question. * **Map Step:** Read the file in chunks across distributed machines. Each machine processes a word, computes its signature (e.g., sorted string "aet"), and emits a key-value pair: {"aet": "eat"}. * **Shuffle Step:** The system routes all identical keys to the same Reducer machine. * **Reduce Step:** The Reducer machine receives a key and a list of all matching words: {"aet": ["eat", "tea", "ate"]} and writes this final grouped list to the database.

20. Valid Parentheses

Here is how a senior engineer (L5/L6) approaches “Valid Parentheses.” At this level, interviewers aren’t just looking for a working solution; they are looking for clean, extensible code, a solid grasp of the underlying data structures, and the ability to explain the fundamental mechanics clearly.

1. Problem Explanation

The Goal: Given a string containing only the characters (,), {, }, [, and], determine if the input string is valid.

The Rules of Validity: 1. **Matching Types:** Every open bracket must be closed by the exact same type of closing bracket. (e.g., (must be closed by), not }). 2. **Correct Order:** Brackets must be closed in the exact reverse order they were opened. You cannot close an outer bracket while an inner bracket is still open.

Examples: * Input: “()” -> Valid. * Input: “(())” -> Valid. (Side-by-side pairs are fine). * Input: “(]” -> Invalid. (Mismatched types). * Input: “([)]” -> Invalid. (The (is closed before the inner [is closed). * Input: “{[] }” -> Valid. (The inner [] opens and closes properly, then the outer {} closes).

2. Solution Explanation

The Intuition (The “Why”): Think of this problem like Russian nesting dolls or navigating folders on your computer. When you open a folder A, then open folder B inside it, you must back out of folder B *before* you can back out of folder A.

This requires a “Last-In, First-Out” (LIFO) tracking system. The *most recently* opened bracket is the *very first* one that needs to be closed. In computer science, the perfect data structure for LIFO behavior is a **Stack**.

The Algorithm (The “How”): 1. We read the string from left to right, one character at a time. 2. If we see an **opening bracket** ((, {, [), we add it to the top of our Stack. We are essentially saying, “I need to remember to close this later.” 3. If we see a **closing bracket** (), },]), we must immediately check if it correctly resolves the most recently opened bracket. * We look at the item on the **top** of the Stack. * If the Stack is empty, it means we have a closing bracket with nothing to close. (Invalid). * If the top of the Stack doesn’t match our closing bracket, we are closing things out of order. (Invalid). * If it *does* match, we remove (pop) that opening bracket from the Stack. It has been successfully resolved. 4. Once we reach the end of the string, our Stack should be completely empty. If there are leftover brackets in the Stack, it means they were never closed. (Invalid).

End-to-End Walkthrough with ASCII Visualizations:

Let's trace a complex, valid input: { [] }

```
[ INITIAL STATE ]
String: { [ ] }
Index:  0 1 2 3
Stack:  (Empty)
```

```
-----
[ STEP 1: Index = 0, char = '{' ]
Action: This is an opening bracket. We push it onto the stack.
```

```
Stack State:
|      |
| {    | <-- Top
-----
```

```
-----
[ STEP 2: Index = 1, char = '[' ]
Action: This is another opening bracket. Push it onto the stack.
```

```
Stack State:
| [    | <-- Top (Most recently opened)
| {    |
-----
```

```
-----
[ STEP 3: Index = 2, char = ']' ]
Action: This is a closing bracket. We must check the Top of the stack.
Check: Does ']' match the top bracket '['? YES!
Resolution: Pop '[' off the stack.
```

```
Stack State:
|      |
| {    | <-- Top (Previous open bracket becomes top again)
-----
```

```
-----
[ STEP 4: Index = 3, char = '}' ]
Action: This is a closing bracket. We check the Top of the stack.
Check: Does '}' match the top bracket '{'? YES!
Resolution: Pop '{' off the stack.
```

```
Stack State:
|      |
----- <-- (Empty)
```

END OF STRING.

Final Check: Is the stack empty? Yes. Therefore, the string is VALID.

Let's trace an invalid input: ([])

String: ([])

Index: 0 1 2 3

Step 1: '(' -> Push to stack. Stack: ['(']

Step 2: '[' -> Push to stack. Stack: ['(', '[']

Step 3: ')' -> Closing bracket! Check top of stack.

Top of stack is '['.

Does ')' match '['? NO!

COLLISION DETECTED.

The string is trying to close the outer '(' before closing the inner '['.

Return False immediately.

3. Time and Space Complexity Analysis

Time Complexity: $O(N)$ where N is the length of the string. **Space Complexity:** $O(N)$ where N is the length of the string.

TIME COMPLEXITY DERIVATION ($O(N)$):

We loop through the string exactly one time from left to right.

For each character, we do one of two things:

1. Push to stack (Takes $O(1)$ constant time)
2. Pop from stack and check map (Takes $O(1)$ constant time)

Since we do a constant amount of work for N characters, $\text{Time} = N * 1 = O(N)$.

SPACE COMPLEXITY DERIVATION ($O(N)$):

The extra memory we use is the Stack.

In the best case "()", the stack grows to size 1, then shrinks to 0. $\text{Space} = O(1)$.

In the worst case "(((((", every character is an open bracket.

[Worst Case Memory Layout]

String: (((((

Stack:

| (|
| (|
| (|
| (|
| (|
| (|

The stack size grows perfectly linearly with the input string.
Therefore, maximum memory used is proportional to N. Space = O(N).

4. Solution Code

An L5/L6 engineer will avoid massive, ugly `if/else` or `switch` statements to map brackets. Instead, they will use a Hash Map (Dictionary) to store the matching pairs. This makes the code drastically cleaner and highly extensible (e.g., if the interviewer suddenly asks to support `<>` brackets, you only have to change the map, not the logic).

Python Snippet

```
def isValid(s: str) -> bool:
    # Maintainability: Use a dictionary to map closing brackets to opening ones.
    # This acts as our single source of truth.
    bracket_map = {
        ')': '(',
        '}': '{',
        ']': '['
    }

    stack = []

    for char in s:
        # If it is a closing bracket (it's a key in our map)
        if char in bracket_map:
            # Pop the top element if stack isn't empty, otherwise use a dummy value '#'
            top_element = stack.pop() if stack else '#'

            # If the popped element doesn't match the required opening bracket, it's invalid
            if bracket_map[char] != top_element:
                return False
        else:
            # If it's an opening bracket, push it onto the stack
            stack.append(char)

    # If the stack is completely empty at the end, all brackets were matched.
    return len(stack) == 0
```

JavaScript Snippet

```
function isValid(s) {
    // Maintainability: Mapping closing brackets to their corresponding opening brackets.
    const bracketMap = {
```

```

        ')': '(',
        '}': '{',
        ']': '['
    };

    const stack = [];

    for (let i = 0; i < s.length; i++) {
        const char = s[i];

        // If the character is a closing bracket
        if (bracketMap[char]) {
            // Get the top element of the stack, or a dummy '#' if stack is empty
            const topElement = stack.length > 0 ? stack.pop() : '#';

            // If the popped element doesn't match the required opening bracket, fail early
            if (bracketMap[char] !== topElement) {
                return false;
            }
        } else {
            // If it's an opening bracket, push it onto the stack
            stack.push(char);
        }
    }

    // Return true only if no unmatched opening brackets remain
    return stack.length === 0;
}

```

Note 1: Terminology Addendum

Stack (Data Structure): * **What it is:** A linear data structure that follows a particular order in which operations are performed. The order is LIFO (Last In First Out). Think of a stack of plates at a buffet; you can only take the plate on the very top, and when you add a plate, you place it on the very top. * **Why it helps:** It inherently stores historical state while forcing you to resolve the most recent state before accessing older states. * **How it applies here:** Nesting rules in strings perfectly mirror LIFO logic. An inner bracket (most recent state) must be closed before an outer bracket (older state) can be closed.

Note 2: Real-World & Interview Variations

Top tech companies usually take this fundamental concept and wrap it in ambiguity or add real-world constraints.

Variation 1: “Minimum Add to Make Parentheses Valid” (Meta/Bloomberg) * *Prompt:* Given a string of (and), return the minimum number of parentheses you must add to make the string valid. * *The L5 Approach:* Since there is only *one* type of bracket here, you actually don’t need a Stack at all, which saves space. You use two integer counters: `open_needed` and `close_needed`. As you iterate, if you see (, you increment `close_needed`. If you see), you decrement it. If `close_needed` goes negative (meaning you have too many closing brackets), you reset it to 0 and increment `open_needed`. Time is $O(N)$, Space is optimized to $O(1)$.

Variation 2: “Remove Invalid Parentheses” (Google/Meta) * *Prompt:* Given a string with brackets and letters, remove the *minimum* number of invalid parentheses to make the string valid. Return all possible results. * *The L5 Approach:* This transitions from a simple Stack problem to a **Breadth-First Search (BFS)** or **Backtracking** problem. You use the “Valid Parentheses” logic as a helper function. If a string is invalid, you generate all possible new strings by removing exactly one bracket. You check all of them. If none are valid, you remove two brackets, and so on. BFS guarantees you find the valid strings with the *minimum* removals first.

Variation 3: “Validate XML/HTML Document” (Real-World System Design flavor) * *Prompt:* Instead of single characters, you are given a string like `<div><p>Text</p></div>`. Validate the tag nesting. * *The L5 Approach:* The core Stack logic remains identical. The complexity shifts to string parsing. You would write logic to parse out the tags (e.g., extracting “div” and “/div”) and then push “div” onto the stack, and when you see “/div”, you pop the stack to ensure they match. This tests your ability to abstract the core LIFO algorithm away from simple character matching.

125. Valid Palindrome

Here is how an L5/L6 Senior Software Engineer would break down and explain “Valid Palindrome” in a real interview setting. At this level, it’s not just about getting the right answer; it’s about demonstrating a deep understanding of memory management, time complexity tradeoffs, and edge-case handling.

1. Problem Explanation

The Premise: You are given a string `s`. A string is considered a “valid palindrome” if it reads the exact same forwards and backwards.

The Catch (The Rules of the Game): We do not compare the raw string. Before checking for symmetry, we must apply a specific filter: 1. **Ignore casing:**

‘A’ is considered the same as ‘a’. 2. **Ignore non-alphanumeric characters:** Spaces, punctuation (like commas, periods, exclamation marks), and symbols must be completely ignored as if they don’t exist in the string. We only care about letters (a-z) and numbers (0-9).

Example: Input: `s = "A b, b a"` Filtered concept: `"abba"` Result: `True` (reads the same forward and backward).

2. Solution Explanation

There are two main ways to solve this. I will explain both because in an interview, explaining the sub-optimal approach first shows you understand the problem before jumping to the optimization.

Approach 1: Filter and Reverse (The “Brute Force” / Readable Way)

The most intuitive way is to literally do what the problem says: 1. Create a brand new, empty string. 2. Loop through the original string. If a character is a letter or number, convert it to lowercase and add it to our new string. 3. Once built, reverse the new string and check if it exactly equals the un-reversed new string.

Why L5s don’t stop here: This approach requires building a completely new string in memory. If you are processing a massive document (e.g., millions of characters), you are wasting a lot of memory duplicating the data.

Approach 2: Two Pointers (The “Optimal” / In-Place Way) Instead of building a new string, we can verify the palindrome **in-place** (without allocating extra memory for a new string) by using two “pointers” (variables that store index positions).

- **Left Pointer (L):** Starts at the very beginning of the string (index 0).
- **Right Pointer (R):** Starts at the very end of the string (index length - 1).

We move L forward and R backward, comparing the characters they point to. * If L points to a space or punctuation, we just skip it by moving L forward. * If R points to a space or punctuation, we skip it by moving R backward. * Once both L and R are pointing to valid letters/numbers, we compare them. If they match, we move both pointers inward. If they don’t match, we immediately know it’s not a palindrome. * We stop when L and R meet in the middle or cross over each other.

Detailed ASCII Walkthrough (Two Pointers) Let’s trace the Two Pointers approach for `s = "A b, b a"`

String:	A	[space]	b	,	[space]	b	a
Indices:	0	1	2	3	4	5	6

STEP 1: Initial State

```
A   b ,   b a
  ^         ^
L=0         R=6
```

L points to 'A' (valid). R points to 'a' (valid).

Compare: lower('A') == lower('a'). Match!

Action: Move L right, Move R left.

STEP 2: Skipping Invalid Characters

```
A   b ,   b a
  ^         ^
L=1         R=5
```

L points to ' ' (invalid space).

R points to 'b' (valid).

Action: Because L is invalid, we skip it by moving L right. R stays put.

STEP 3: Finding the next Valid Character

```
A   b ,   b a
  ^         ^
L=2         R=5
```

L points to 'b' (valid). R points to 'b' (valid).

Compare: lower('b') == lower('b'). Match!

Action: Move L right, Move R left.

STEP 4: Hitting Punctuation

```
A   b ,   b a
  ^   ^
L=3   R=4
```

L points to ',' (invalid). R points to ' ' (invalid space).

Action: Since both are invalid, we move L right and R left to skip them.

STEP 5: Pointers Cross

```
A   b ,   b a
  ^
L=4
R=3
```

L is now greater than R. The pointers have crossed!
This means we have successfully checked the entire string from the outside in without finding a mismatch.
Result: TRUE (Valid Palindrome)

3. Time and Space Complexity Analysis

Approach 1 (Filter and Reverse)

- **Time Complexity:** $O(N)$
- **Space Complexity:** $O(N)$ because we allocate memory for a new string of up to length N .

Approach 2 (Two Pointers)

- **Time Complexity:** $O(N)$ Where N is the total number of characters in the string.

Visual Derivation of Time Complexity ($O(N)$):

String Length = N

L Pointer: ---> moves right

R Pointer: <--- moves left

L and R move towards each other until they meet.

They never go backward.

Every character in the string is visited at most ONE time by either L or R.

[L -> ... <- R]

Total operations scale linearly with the length of the string.

Therefore, Time = $O(N)$.

- **Space Complexity:** $O(1)$

Visual Derivation of Space Complexity ($O(1)$):

We are given the input string 's' (we don't count this against our space).

Memory we allocate:

- Variable L (integer)

- Variable R (integer)

Memory map:

[L: 0]

[R: 6]

No matter how long the string is ($N = 10$, or $N = 10,000,000$), we ONLY ever need two integer. Therefore, Space = $O(1)$.

4. Solution Code

Here are both the sub-optimal and optimal implementations in Python and JavaScript. Note the use of helper functions to keep the code modular and clean—a highly valued trait at top tech companies.

Python Code

```
# --- SOLUTION 1: Filter and Reverse ( $O(N)$  Time,  $O(N)$  Space) ---
# Good to explain first, but not the final answer.
def isPalindrome_naive(s: str) -> bool:
    # 1. Build a filtered string
    filtered = []
    for char in s:
        if char.isalnum(): # Built-in Python method to check letter/number
            filtered.append(char.lower())

    # 2. Join back to string for comparison
    filtered_str = "".join(filtered)

    # 3. Check if it equals its reverse
    return filtered_str == filtered_str[::-1]

# --- SOLUTION 2: Two Pointers ( $O(N)$  Time,  $O(1)$  Space) ---
# The L5/L6 expected solution.
def isPalindrome_optimal(s: str) -> bool:
    L = 0
    R = len(s) - 1

    while L < R:
        # Skip non-alphanumeric chars from the left
        while L < R and not s[L].isalnum():
            L += 1

        # Skip non-alphanumeric chars from the right
        while L < R and not s[R].isalnum():
            R -= 1

        # Compare characters (case-insensitive)
        if s[L].lower() != s[R].lower():
            return False
```

```

        # Move both pointers inward for the next check
        L += 1
        R -= 1

    return True

```

JavaScript Code

```

// Helper function to check if a character is alphanumeric.
// Doing this via character codes is much faster than using Regex in JS.
function isAlphanumeric(char) {
    const code = char.charCodeAt(0);
    return (code >= 48 && code <= 57) || // 0-9
           (code >= 65 && code <= 90) || // A-Z
           (code >= 97 && code <= 122); // a-z
}

// --- SOLUTION 1: Filter and Reverse (O(N) Time, O(N) Space) ---
var isPalindromeNaive = function(s) {
    let filtered = "";
    for (let char of s) {
        if (isAlphanumeric(char)) {
            filtered += char.toLowerCase();
        }
    }

    // Reverse the string by splitting into array, reversing, and joining
    let reversed = filtered.split('').reverse().join('');
    return filtered === reversed;
};

// --- SOLUTION 2: Two Pointers (O(N) Time, O(1) Space) ---
var isPalindromeOptimal = function(s) {
    let L = 0;
    let R = s.length - 1;

    while (L < R) {
        // Skip invalid characters from the left
        while (L < R && !isAlphanumeric(s[L])) {
            L++;
        }

        // Skip invalid characters from the right
        while (L < R && !isAlphanumeric(s[R])) {
            R--;
        }
    }
}

```

```

    }

    // Compare the valid characters
    if (s[L].toLowerCase() != s[R].toLowerCase()) {
        return false;
    }

    // Move inward
    L++;
    R--;
}

return true;
};

```

Note 1: Terms and Techniques Used

- **Two Pointers:** A common algorithmic technique used for arrays and strings where you use two variables (pointers) to keep track of indices, usually moving them towards each other or in the same direction to find pairs, sub-arrays, or verify symmetries. It heavily optimizes Space Complexity because it avoids making copies of the data structure.
 - **In-Place Algorithm:** An algorithm that transforms or analyzes data using no extra auxiliary memory (or a small, constant amount of it like $O(1)$). We analyze the string “in-place” without building a new `filtered_str`.
-

Note 2: Real-World & Indirect Interview Variations

Top-tier companies often disguise this problem or add a slight twist to test your problem-solving depth.

1. Meta (Facebook): “Valid Palindrome II” (LeetCode 680) * *The Twist:* Given a string `s`, return true if the string can be a palindrome after deleting **at most one** character. * *How an L5 solves it:* Use the exact same Two Pointers setup. When `s[L]` does NOT equal `s[R]`, you have a choice. Since you can delete one character, the string is valid if either: 1. The substring skipping L (from L+1 to R) is a palindrome. 2. The substring skipping R (from L to R-1) is a palindrome. You simply write a helper function `check_palindrome(string, left, right)` and call it twice when you hit your first mismatch.

2. Google: “Streaming / Chunked Palindrome Verification” * *The Twist:* The string is extremely massive (e.g., 500 Terabytes). It does not fit in RAM. It lives on a distributed file system. How do you check if it’s a palindrome? * *How an L5 solves it:* You can’t use `L = 0` and `R = len(s) - 1` easily in

standard memory. You must discuss **chunking** and **I/O optimization**. You would spin up two reader processes. Reader A reads the file from the start in 10MB chunks. Reader B reads the file from the end in 10MB chunks (reading backwards). You run the two-pointer logic across these chunks. You must discuss the latency of reading backward from a disk and how to buffer the data efficiently.

3. Bloomberg: “Financial Ticker / Log Data Cleaning” * *The Twist:* You receive a messy feed of financial logs. You need to identify symmetric trade sequences (e.g., buy A, sell B, sell B, buy A). However, the feed is polluted with system heartbeat logs that need to be ignored. * *How an L5 solves it:* This maps perfectly to the filtering mechanism of Valid Palindrome. The “heartbeat logs” are the non-alphanumeric characters. The trades are the letters. You use Two Pointers, but instead of checking `isalnum()`, your `skip` condition checks if the current log entry is a heartbeat type. If it is, move the pointer. Compare the actual trades for symmetry.

5. Longest Palindromic Substring

Stepping into the shoes of an L5/L6 engineer, the key to succeeding in an interview with a problem like this isn’t just writing the code quickly. It is about demonstrating a systematic thought process: acknowledging the brute force, identifying its inefficiencies, proposing an optimized approach based on a core insight, and proving it works with clear visual trace-throughs before touching the keyboard.

Here is the end-to-end breakdown of LeetCode 5: Longest Palindromic Substring.

1. Problem Explanation

The Goal: We are given a string `s`. We need to find the longest contiguous chunk of characters (a substring) inside `s` that is a palindrome.

What is a Palindrome? A palindrome is a sequence of characters that reads the exact same forwards and backwards. * Examples: “racecar”, “madam”, “a”, “bb”.

What is a Substring? A substring is a contiguous slice of a string. You cannot skip characters. * For string “abc”, valid substrings are “a”, “ab”, “abc”, “b”, “bc”, “c”. (“ac” is a subsequence, not a substring).

Visualizing the Goal: Let `s = "babad"`

```
s = b a b a d
    [---]      -> "bab" (Palindrome! Length = 3)
```

```

[---]      -> "aba" (Palindrome! Length = 3)
[-]        -> "d"   (Palindrome! Length = 1)

```

Both “bab” and “aba” are valid answers. Returning either is correct.

2. Solution Explanation

The Naive Approach (What a senior engineer briefly mentions): The brute-force way is to generate every possible substring, check if it reads the same forwards and backwards, and keep track of the longest one. * Generating all substrings takes $O(N^2)$ time. * Checking if each one is a palindrome takes $O(N)$ time. * Total Time: $O(N^3)$. This is terribly inefficient because we do a lot of repetitive checking.

The Core Insight: Expand Around Center Instead of looking at the outer edges of a substring and checking inwards, what if we started at the *center* and expanded outwards?

A palindrome mirrors around its center. If we know “a” is a palindrome, and the character to its left matches the character to its right (“b” + “a” + “b”), we instantly know “bab” is a palindrome without re-checking the “a”.

The Tricky Part: Two Types of Centers Palindromes can have an odd length or an even length. This changes where the “center” is. 1. **Odd Length:** The center is a single character. (e.g., in “bab”, the center is ‘a’). 2. **Even Length:** The center is the invisible space *between* two characters. (e.g., in “abba”, the center is between the two ‘b’s).

If a string has N characters, it has N single-character centers and $N - 1$ between-character centers. Therefore, there are exactly $2N - 1$ possible centers to expand from.

ASCII Diagram Walkthrough: “babad” Let’s trace the “Expand Around Center” technique on $s = \text{“babad”}$.

```

STRING:  b  a  b  a  d
INDEX:   0  1  2  3  4

```

Step 1: Try center at index 0 (‘b’) * **Odd check (Center ‘b’):** Left and right bounds start at index 0. Expand outwards. Left goes out of bounds (-1). Longest here is “b” (length 1). * **Even check (Center between ‘b’ and ‘a’):** Left at 0, Right at 1. ‘b’ != ‘a’. Stop. Length 0.

Step 2: Try center at index 1 (‘a’)

Check Odd Length (Center is ‘a’ at index 1)

```

Pass 1: L=1, R=1. s[1] == s[1] ('a' == 'a'). Valid! Length = 1.
      b [a] b  a  d

```

^
L,R

Pass 2: Expand. L=0, R=2. s[0] == s[2] ('b' == 'b'). Valid! Length = 3.

[b a b] a d
^ ^
L R

Pass 3: Expand. L=-1, R=3. L is out of bounds. Stop expanding.

Result for this center: "bab" (Length 3). New Best!

Step 3: Try center between index 1 and 2 ('a' and 'b')

Check Even Length

Pass 1: L=1, R=2. s[1] != s[2] ('a' != 'b'). Stop.

b a b a d
^ ^
L R

Result: Length 0.

Step 4: Try center at index 2 ('b')

Check Odd Length (Center is 'b' at index 2)

Pass 1: L=2, R=2. s[2] == s[2] ('b' == 'b'). Valid! Length = 1.

b a [b] a d
^
L,R

Pass 2: Expand. L=1, R=3. s[1] == s[3] ('a' == 'a'). Valid! Length = 3.

b [a b a] d
^ ^
L R

Pass 3: Expand. L=0, R=4. s[0] != s[4] ('b' != 'd'). Stop.

Result for this center: "aba" (Length 3).

(Ties with our best, we can just keep "bab" or replace it).

We continue this process for every character and every space between characters, constantly updating our “longest seen so far” variable.

3. Time and Space Complexity Analysis

TIME COMPLEXITY DERIVATION: $O(N^2)$
N = Length of string 's'

```
| 1. Number of Centers:
|   - Every character is a center (N centers)
|   - Every gap between characters is a center (N - 1 centers)
|   - Total centers to check = 2N - 1
|
| 2. Expansion Work per Center:
|   - In the worst-case scenario (e.g., a string of all identical
|     characters like "aaaaa"), we expand from the center all the
|     way to the edges.
|   - Maximum expansions per center = N / 2 steps
|
| TOTAL TIME: (2N - 1) centers * (N / 2) max expansions
|             = roughly N^2 operations.
|             => O(N^2) Time Complexity
|-----+
|
|-----+
| SPACE COMPLEXITY DERIVATION: O(1)
|-----+
| 1. Memory usage:
|   - We only use a few integer variables (left pointer, right
|     pointer, start index of best palindrome, max length).
|   - We do not build any matrices or secondary arrays.
|   - Slicing the string at the very end takes O(N) space to return
|     the result, but auxiliary (extra) working space is constant.
|
| TOTAL SPACE: O(1) auxiliary space
|-----+
```

4. Solution Code

I will provide two solutions. 1. **Expand Around Center:** This is the optimal $O(N^2)$ Time / $O(1)$ Space solution that an interviewer expects. 2. **Dynamic Programming:** This is an $O(N^2)$ Time / $O(N^2)$ Space solution. While it uses more memory, demonstrating you understand DP string state-tracking is a massive positive signal in a senior interview.

Primary Solution: Expand Around Center (Optimized Space)

```
def longestPalindrome(s: str) -> str:
    if not s or len(s) < 2:
        return s
```

```

start = 0
max_len = 0

# Helper function to expand outwards from a given left and right center
def expand_around_center(left: int, right: int) -> int:
    # Keep expanding while within bounds and characters match
    while left >= 0 and right < len(s) and s[left] == s[right]:
        left -= 1
        right += 1
    # The loop breaks when the bounds are invalid.
    # The actual length of the valid palindrome is (right - 1) - (left + 1) + 1
    # which simplifies to right - left - 1
    return right - left - 1

for i in range(len(s)):
    # Check for odd-length palindromes (single character center)
    len1 = expand_around_center(i, i)

    # Check for even-length palindromes (center between i and i+1)
    len2 = expand_around_center(i, i + 1)

    # Take the longest of the two types of palindromes found at this center
    longest_at_center = max(len1, len2)

    # If we found a new maximum length, calculate its starting index
    if longest_at_center > max_len:
        max_len = longest_at_center
        # Math to find the exact starting index based on the center 'i'
        # We subtract half the length to step backwards from the center.
        # (longest_at_center - 1) // 2 handles both odd and even lengths correctly.
        start = i - (longest_at_center - 1) // 2

return s[start : start + max_len]

```

JavaScript Snippet

```

function longestPalindrome(s) {
    if (!s || s.length < 2) return s;

    let start = 0;
    let maxLen = 0;

    // Helper function to expand outwards
    function expandAroundCenter(left, right) {
        while (left >= 0 && right < s.length && s[left] === s[right]) {
            left--;
            right++;
        }
    }

```

```

    }
    // Return the length of the valid palindrome found
    return right - left - 1;
}

for (let i = 0; i < s.length; i++) {
    // Odd length center
    let len1 = expandAroundCenter(i, i);
    // Even length center
    let len2 = expandAroundCenter(i, i + 1);

    let longestAtCenter = Math.max(len1, len2);

    if (longestAtCenter > maxLen) {
        maxLen = longestAtCenter;
        // Calculate start index mathematically to avoid storing the whole string
        start = i - Math.floor((longestAtCenter - 1) / 2);
    }
}

return s.substring(start, start + maxLen);
}

```

Secondary Solution: Dynamic Programming (Unoptimized Space)

Why mention this? Because DP is a foundational concept. The logic is: A string is a palindrome if its first and last characters match AND the substring inside them is also a palindrome.

Python Snippet

```

def longestPalindrome_DP(s: str) -> str:
    n = len(s)
    if n < 2:
        return s

    # dp[i][j] will be True if the substring s[i...j] is a palindrome
    dp = [[False] * n for _ in range(n)]

    start = 0
    max_len = 1

    # Base Case 1: Every single character is a palindrome
    for i in range(n):
        dp[i][i] = True

    # Base Case 2: Check 2-character substrings

```

```

for i in range(n - 1):
    if s[i] == s[i+1]:
        dp[i][i+1] = True
        start = i
        max_len = 2

# Step 3: Check substrings of length 3 or greater
for length in range(3, n + 1):
    for i in range(n - length + 1):
        j = i + length - 1 # Ending index

        # It's a palindrome if the outer chars match AND the inner string is a palindrome
        if s[i] == s[j] and dp[i+1][j-1]:
            dp[i][j] = True
            if length > max_len:
                start = i
                max_len = length

return s[start : start + max_len]

```

JavaScript Snippet

```

function longestPalindromeDP(s) {
    const n = s.length;
    if (n < 2) return s;

    // Create a 2D boolean array filled with false
    const dp = Array.from({ length: n }, () => Array(n).fill(false));
    let start = 0;
    let maxLen = 1;

    // Base Case 1: Length 1
    for (let i = 0; i < n; i++) {
        dp[i][i] = true;
    }

    // Base Case 2: Length 2
    for (let i = 0; i < n - 1; i++) {
        if (s[i] === s[i + 1]) {
            dp[i][i + 1] = true;
            start = i;
            maxLen = 2;
        }
    }

    // Step 3: Length 3 and beyond
    for (let len = 3; len <= n; len++) {

```

```

    for (let i = 0; i < n - len + 1; i++) {
        let j = i + len - 1;

        if (s[i] === s[j] && dp[i + 1][j - 1]) {
            dp[i][j] = true;
            if (len > maxLen) {
                start = i;
                maxLen = len;
            }
        }
    }
}

return s.substring(start, start + maxLen);
}

```

Note 1: Terminology and Techniques

- **Expand Around Center:** A two-pointer technique specifically tailored for symmetry. It avoids redundant checks by anchoring a “center” and radiating outward, ensuring we only do work as long as the palindromic property holds.
- **Dynamic Programming (DP):** A method of solving complex problems by breaking them down into simpler subproblems and storing their results (often in a grid or table) to avoid re-computing them. Here, knowing if a smaller inner string is a palindrome prevents us from scanning it again.
- **Manacher’s Algorithm (L6/L7 level knowledge):** It is mathematically possible to solve this in $O(N)$ strict time using Manacher’s Algorithm. It cleverly uses the properties of overlapping palindromes to skip center expansions entirely. However, the logic is highly complex, rarely expected to be memorized or coded perfectly in a 45-minute interview, and mostly serves as a good “trivia” mention to show deep algorithmic awareness.

Note 2: Real-World Interview Variations

1. **Meta: “Longest Symmetric Server Traffic Spike”** * *Prompt:* You are given an array of integer values representing network traffic volume per minute over a day: [10, 20, 50, 100, 50, 20, 10...]. Identify the longest contiguous period of time where the traffic ramp-up perfectly mirrors the ramp-down. * *Solution:* This is the exact same problem, just with an array of integers instead of a string of characters. You would implement the exact same Expand Around Center algorithm, comparing `traffic[left] == traffic[right]`.
2. **Google: “Validating DNA Sequences”** * *Prompt:* Given a massive string of DNA nucleotides (A, C, G, T), find the largest “mirror” sequence.

However, in this variation, 'A' mirrors 'T', and 'C' mirrors 'G'. * *Solution*: You still use Expand Around Center, but you modify the equality check. Instead of `s[left] == s[right]`, you write a helper function `is_mirror(s[left], s[right])` that returns True if the pair is (A,T), (T,A), (C,G), or (G,C).

3. Bloomberg: “Historical Price Pattern Matching” * *Prompt*: Given a chronological list of daily stock closing prices, find the longest window where the price action symmetrically reversed itself (e.g., went up by \$2, then up by \$5, then down by \$5, then down by \$2). * *Solution*: Convert the absolute prices into an array of “deltas” (differences between consecutive days). A symmetric reversal means a delta of +5 mirrors a delta of -5. Apply Expand Around Center on the delta array, where the matching condition is `delta[left] == -1 * delta[right]`.

647. Palindromic Substrings

As a senior engineer, I’ve seen candidates struggle with “Palindromic Substrings” (LeetCode 647) quite a bit. It is a fantastic problem because it tests whether you can identify repeated, redundant work and optimize it away without relying on heavy data structures.

At the L5/L6 level, I am not just looking for a solution; I am looking for an intuition-driven approach. I want to see you discard inefficient methods logically and arrive at an elegant, memory-conscious algorithm.

Let’s break this down end-to-end, completely grounded in visuals.

1. Problem Explanation

The Goal: Given a string `s`, count how many palindromic substrings it contains.

Key Definitions: * **Substring**: A contiguous sequence of characters within a string. You cannot skip letters. * **Palindrome**: A string that reads the exact same forwards and backwards.

Visualizing Substrings vs Subsequences:

String: "t a c o"

Valid Substring: "a c" (Contiguous)
t [a c] o

Invalid Substring: "t c" (Skipped 'a' - this is a subsequence, not a substring)
[t] a [c] o

Visualizing the Target (Palindromes): Input: `s = "aba"`

We must evaluate EVERY possible contiguous block:

Length 1:

```
[a] b a -> "a" (Reads same forwards/backwards? YES)
a [b] a -> "b" (YES)
a b [a] -> "a" (YES)
```

Length 2:

```
[a b] a -> "ab" (NO)
a [b a] -> "ba" (NO)
```

Length 3:

```
[a b a] -> "aba" (YES)
```

Total Palindromic Substrings = 4

2. Solution Explanation

Let's look at the evolution of solving this problem, starting from the brute force approach and moving to the optimized "L5" approach.

Approach 1: The Brute Force (The "Try Everything" Method) The most obvious way is to generate every possible substring, and then write a helper function to check if each one is a palindrome.

- **Why it's bad:** Generating all substrings requires a nested loop, giving us N^2 substrings. For *each* of those substrings, checking if it's a palindrome takes another loop of length N . This means we are doing $N * N * N$ (or N^3) operations. For a string of 1000 characters, that's 1,000,000,000 operations. It's too slow.

Approach 2: Expand Around Center (The Optimized Approach)

How do we get rid of the redundant work? Look at how palindromes are structured. They mirror outward from a central point.

If we know that "aba" is a palindrome, and we want to check "cabac", we *don't* need to re-verify the inner "aba". We just need to check if the new outer letters ('c' and 'c') match.

The Strategy: Instead of generating substrings and checking them, we treat every character (and every space between characters) as a potential "center" of a palindrome. We then expand outwards from that center as long as the left and right characters match.

The "Trick" - Odd vs Even Lengths: Palindromes can have an odd length (e.g., "aba" - center is 'b') or an even length (e.g., "abba" - center is between the two 'b's). We must check both types of centers. For a string of length N , there

are N odd centers (every letter) and $N - 1$ even centers (the spaces between letters). Total centers = $2N - 1$.

Walkthrough Visualization: $s = \text{"aaba"}$

Let's iterate through the string and expand from each center.

Indices: 0 1 2 3

String: a a b a

--- CENTER AT INDEX 0 ('a') ---

Odd Center (L=0, R=0):

[a] a b a

L/R

'a' == 'a' -> Count = 1

Expand (L=-1, R=1): L is out of bounds. Stop.

Even Center (L=0, R=1):

[a a] b a

L R

'a' == 'a' -> Count = 2

Expand (L=-1, R=2): L is out of bounds. Stop.

--- CENTER AT INDEX 1 ('a') ---

Odd Center (L=1, R=1):

a [a] b a

L/R

'a' == 'a' -> Count = 3

Expand (L=0, R=2):

[a] a [b] a

L R

'a' != 'b' -> Stop.

Even Center (L=1, R=2):

a [a b] a

L R

'a' != 'b' -> Stop.

--- CENTER AT INDEX 2 ('b') ---

Odd Center (L=2, R=2):

a a [b] a

L/R

'b' == 'b' -> Count = 4

Expand (L=1, R=3):

a [a] b [a]

L R

'a' == 'a' -> Count = 5

Expand (L=0, R=4): R is out of bounds. Stop.

```

Even Center (L=2, R=3):
  a  a [b  a]
      L  R
  'b' != 'a' -> Stop.

--- CENTER AT INDEX 3 ('a') ---
Odd Center (L=3, R=3):
  a  a  b [a]
      L/R
  'a' == 'a' -> Count = 6
  Expand (L=2, R=4): R out of bounds. Stop.

Even Center (L=3, R=4): R out of bounds. Stop.

FINAL COUNT = 6.

```

3. Time and Space Complexity Analysis

Here is the breakdown of why the Expand Around Center method is the gold standard here.

```

=====
TIME COMPLEXITY: O(N^2) (where N is the length of string s)
=====

```

Derivation Diagram:

```

[ String of Length N ]
|
|--> Total potential centers = (2 * N) - 1
|   (N single characters + N-1 spaces between characters)
|
V
[ For EACH center... ]
|
|--> We expand left and right.
|--> In the absolute worst case (e.g., "aaaaaaa"),
|   we expand outward until we hit the edges.
|--> Max expansions for a center = N / 2 steps.
V
Total Operations = (Number of Centers) * (Max Expansions per Center)
Total Operations = (2 * N) * (N / 2)
Total Operations = N * N = N^2

```

Because the dominant scaling factor is N squared, Time Complexity is $O(N^2)$.

```
=====
SPACE COMPLEXITY: O(1)
=====
Derivation Diagram:

[ Variables Used in Memory ]
|
|--> 'count' integer to track total palindromes.
|--> 'left' integer pointer.
|--> 'right' integer pointer.
V
Total memory used = 3 Integers.
No matter if the string is 10 characters or 10,000 characters long,
we only ever need exactly these few integer variables.
Memory does not scale with input size. Space is Constant, i.e., O(1).
=====
```

4. Solution Code

Here are both approaches. Note how the optimized approach delegates the expansion logic to a clean helper function.

Python Snippets

```
# =====
# APPROACH 1: BRUTE FORCE (Unoptimized)
# Time: O(N^3) | Space: O(1)
# Note: Useful to explain, but don't submit this in an interview.
# =====
def countSubstrings_brute(s: str) -> int:
    count = 0
    n = len(s)

    # Generate all possible start and end points for substrings
    for i in range(n):
        for j in range(i, n):
            # Extract the substring
            sub = s[i:j+1]
            # Check if it reads the same backwards (sub[::-1] reverses it)
            if sub == sub[::-1]:
                count += 1

    return count
```

```

# =====
# APPROACH 2: EXPAND AROUND CENTER (Optimized L5 Solution)
# Time:  $O(N^2)$  | Space:  $O(1)$ 
# =====
def countSubstrings(s: str) -> int:
    total_palindromes = 0

    # Helper function to expand outwards from a given left and right index
    def count_palindromes_from_center(left: int, right: int) -> int:
        count = 0
        # While pointers are in bounds and the characters match
        while left >= 0 and right < len(s) and s[left] == s[right]:
            count += 1 # We found a valid palindrome
            left -= 1 # Expand leftwards
            right += 1 # Expand rightwards
        return count

    # Iterate through every character in the string
    for i in range(len(s)):
        # 1. Count odd-length palindromes (single character center)
        total_palindromes += count_palindromes_from_center(i, i)

        # 2. Count even-length palindromes (center is between i and i+1)
        total_palindromes += count_palindromes_from_center(i, i + 1)

    return total_palindromes

```

JavaScript Snippets

```

// =====
// APPROACH 1: BRUTE FORCE (Unoptimized)
// Time:  $O(N^3)$  | Space:  $O(1)$ 
// =====
function countSubstringsBrute(s) {
    let count = 0;

    // Helper to check if string is palindrome
    const isPalindrome = (str) => {
        let l = 0, r = str.length - 1;
        while (l < r) {
            if (str[l] !== str[r]) return false;
            l++; r--;
        }
        return true;
    };
}

```

```

    for (let i = 0; i < s.length; i++) {
        for (let j = i; j < s.length; j++) {
            let sub = s.substring(i, j + 1);
            if (isPalindrome(sub)) count++;
        }
    }

    return count;
}

// =====
// APPROACH 2: EXPAND AROUND CENTER (Optimized L5 Solution)
// Time: O(N^2) | Space: O(1)
// =====
function countSubstrings(s) {
    let totalPalindromes = 0;

    // Helper function to expand outwards from a given left and right index
    const countPalindromesFromCenter = (left, right) => {
        let count = 0;
        // While pointers are in bounds and the characters match
        while (left >= 0 && right < s.length && s[left] === s[right]) {
            count++; // We found a valid palindrome
            left--; // Expand leftwards
            right++; // Expand rightwards
        }
        return count;
    };

    // Iterate through every character in the string
    for (let i = 0; i < s.length; i++) {
        // 1. Count odd-length palindromes (single character center)
        totalPalindromes += countPalindromesFromCenter(i, i);

        // 2. Count even-length palindromes (center is between i and i+1)
        totalPalindromes += countPalindromesFromCenter(i, i + 1);
    }

    return totalPalindromes;
}

```

Note 1: Terms and Techniques Used

- **Expand Around Center (Two Pointers):** We used two index variables (`left` and `right`) to traverse the array simultaneously. By starting them at the same spot (or adjacent spots) and moving them outward, we effectively evaluate “layers” of the string without having to start over from scratch every time. It converts an $O(N^3)$ string generation problem into an $O(N^2)$ pointer comparison problem.
 - **Dynamic Programming (Alternative Context):** You might read online that this can be solved with a 2D Dynamic Programming matrix. While true (it caches whether `s[i:j]` is a palindrome to determine if `s[i-1:j+1]` is), **Expand Around Center is strictly better for this problem**. DP requires $O(N^2)$ memory to store the matrix, whereas our Two Pointer approach requires $O(1)$ memory. An L5 engineer should point this out to their interviewer.
-

Note 2: Real-World & Interview Variations

Once you write the optimized code above, interviewers at Google, Meta, and Bloomberg rarely stop there. They will twist the problem. Here is how they ask it and how you adapt:

Variation 1: “Longest Palindromic Substring” (LeetCode 5) - *Classic Google & Bloomberg* * **The Prompt:** Instead of counting *how many* palindromic substrings exist, return the *longest* one. * **How to solve:** You use the exact same “Expand Around Center” skeleton. However, inside your helper function, instead of returning a count, you keep track of the maximum length seen so far (`right - left - 1`) and the starting index of that maximum palindrome. At the very end of the main loop, you slice the original string using that max start index and length.

Variation 2: “Valid Palindrome II” (LeetCode 680) - *Meta Heavy-weight* * **The Prompt:** Given a string `s`, return true if it can be a palindrome after deleting **at most one** character from it. * **How to solve:** You use Two Pointers, but instead of expanding from the center out, you squeeze from the outside in (`Left` at 0, `Right` at `N-1`). The moment `s[left] != s[right]`, you have a choice. You can either delete the left character or delete the right character. You write a tiny helper function to check if the remaining string `s[left+1 : right]` is a regular palindrome, OR if `s[left : right-1]` is a regular palindrome. If either is true, you return true!

Variation 3: “Palindromic Substrings of Specific Length / Cost” - *Bloomberg* * **The Prompt:** Count palindromes, but only those of length `K`, or where characters have assigned “weights” and the palindrome must be below a certain weight. * **How to solve:** For length `K`, you wouldn’t expand from center. You would use a fixed **Sliding Window** of size `K` iterating over the

string, checking if the window contents form a palindrome. For the weighted version, you maintain a running sum of weights inside your Expand Around Center loop and break early if the weight exceeds the limit, even if the characters still match.

Encode and Decode Strings

Here is an end-to-end walkthrough for “Encode and Decode Strings,” approached from the perspective of an L5/L6 (Senior/Staff) engineer.

An engineer at this level knows that this isn’t just a puzzle; it is the fundamental basis for how computers communicate over a network (like TCP) or save data to disk. We call this process “Serialization” (encoding objects to a string of bytes) and “Deserialization” (reconstructing the object).

1. Problem Explanation

The Goal: You are given a list of strings (an array). You need to design two functions: 1. **Encode:** Takes the list of strings and squashes them into a single, continuous string. 2. **Decode:** Takes that single string and perfectly reconstructs the original list of strings.

The Catch (Why it is hard): The naïve approach is to just join the strings using a special character, like a comma `,` or a hash `#`. If input is `["eat", "tea"]`, we join with `#` to get `"eat#tea"`. When decoding, we split by `#` and get `["eat", "tea"]`. Perfect!

But what if the strings themselves contain our special character? Input: `["hello#world", "foo"]` Naïve Encode (using `#`): `"hello#world#foo"` Naïve Decode (splitting by `#`): `["hello", "world", "foo"]` -> **FATAL ERROR: Data Loss.** The original array had 2 items; we decoded it into 3.

We cannot guarantee any single character is “safe” to act as a separator because the problem states the strings can contain *any* ASCII characters.

2. Solution Explanation

To solve this bulletproofly, we borrow a concept from computer networking: **Length Prefixing**.

Instead of relying on a separator to tell us where a word *ends*, we tell the decoder exactly how long the word is *before* it begins reading.

The Formula: For every string, we encode it as: `[Length of String] + [Special Delimiter] + [The String Itself]`

Let's say our special delimiter is #. * The word "cat" becomes 3#cat. (Length is 3). * The word "#ab" becomes 3##ab. (Length is 3. Notice how the first # is our delimiter, and the second # is part of the word. This is totally safe now!)

Let's visualize the entire process step-by-step.

```
=====
ENCODING PHASE
=====
Input Array: ["cat", "#ab", "xyz"]

Step 1: Process "cat"
Length: 3
Format: 3 + # + cat
Result String: "3#cat"

Step 2: Process "#ab"
Length: 3
Format: 3 + # + #ab
Result String: "3#cat3##ab"

Step 3: Process "xyz"
Length: 3
Format: 3 + # + xyz
Result String: "3#cat3##ab3#xyz"

FINAL ENCODED STRING: "3#cat3##ab3#xyz"
```

```
=====
DECODING PHASE (The tricky part)
=====
Encoded String: "3#cat3##ab3#xyz"
```

We use a pointer 'i' to traverse the string.

```
-----
Iteration 1:
String: 3#cat3##ab3#xyz
      ^
      i
```

1. Find the delimiter '#'. It is at index 1.
2. Read the number before the '#'. The number is 3.
(This tells us the next 3 characters belong to the string).
3. Extract 3 characters AFTER the '#': "cat"
4. Add "cat" to our result array.

5. Move pointer 'i' to the start of the next chunk.

Iteration 2:

String: 3#cat3##ab3#xyz

^
i

1. Find the delimiter '#'. It is at index 6.
2. Read the number before the '#'. The number is 3.
3. Extract 3 characters AFTER the '#': "#ab"
(Notice how the decoder doesn't get confused by the extra '#'!
It strictly reads 3 characters, no matter what they are).
4. Add "#ab" to our result array.
5. Move pointer 'i' forward.

Iteration 3:

String: 3#cat3##ab3#xyz

^
i

1. Find the delimiter '#'. It is at index 11.
2. Read the number before the '#'. The number is 3.
3. Extract 3 characters AFTER the '#': "xyz"
4. Add "xyz" to our result array.
5. End of string.

FINAL RESULT: ["cat", "#ab", "xyz"]

Why this is L5/L6 level thinking: This approach separates control data (the length and the #) from payload data (the actual string). The decoder never has to guess. It explicitly reads the exact block of memory assigned to the payload, making it 100% immune to edge cases (like empty strings or strings filled with #).

3. Time and Space Complexity Analysis

Let N be the total number of characters across all strings in the input list. Let K be the number of strings in the input list.

TIME COMPLEXITY DERIVATION: $O(N)$

ENCODE OPERATION:

Input: ["a", "b", "c"]

We iterate through each string exactly once.
We calculate its length $\rightarrow O(1)$ time per string.
We append it to our result $\rightarrow O(\text{length of string})$ time.
Total Time to Encode = Sum of all string lengths = $O(N)$.

DECODE OPERATION:

Encoded String: "1#a1#b1#c"
We move our pointer 'i' strictly forward through the string.
Reading the number $\rightarrow$ Touches character once.
Reading the delimiter $\rightarrow$ Touches character once.
Extracting the substring $\rightarrow$ Touches characters once.
Total Time to Decode = Length of encoded string = $O(N)$.

OVERALL TIME COMPLEXITY = $O(N)$

SPACE COMPLEXITY DERIVATION: $O(N)$

ENCODE OPERATION:

We must create a new string to return.
The size of this string is the original characters (N)
plus the length integers (K) plus the '#' delimiters (K).
Total size = $N + 2K$. Since $K \leq N$, this simplifies to $O(N)$.

DECODE OPERATION:

We must create a new array of strings to return.
The total characters stored in this array is exactly N .
 $O(N)$ memory allocated.

OVERALL SPACE COMPLEXITY = $O(N)$

4. Solution Code

Here is the implementation in both Python and JavaScript. Both follow the exact Length Prefixing logic described above.

Python Implementation

```
class Codec:
    def encode(self, strs):
        """
        Encodes a list of strings to a single string.
        """
        encoded_string = ""
        for s in strs:
```

```

        # We append: length of string + '#' + the string itself
        # Example: "apple" -> "5#apple"
        encoded_string += str(len(s)) + "#" + s

    return encoded_string

def decode(self, s):
    """
    Decodes a single string to a list of strings.
    """
    decoded_list = []
    i = 0 # Pointer to track our position in the encoded string

    while i < len(s):
        # Step 1: Find the next '#' delimiter starting from our current position 'i'
        j = i
        while s[j] != '#':
            j += 1

        # Step 2: Extract the length of the string
        # The number is between our start pointer 'i' and the delimiter 'j'
        length_of_str = int(s[i:j])

        # Step 3: Extract the actual string
        # It starts right after the delimiter (j + 1) and goes for 'length_of_str' characters
        start_of_str = j + 1
        end_of_str = start_of_str + length_of_str

        decoded_list.append(s[start_of_str:end_of_str])

        # Step 4: Move the pointer 'i' to the start of the next chunk
        i = end_of_str

    return decoded_list

```

JavaScript Implementation

```

class Codec {
    /**
     * Encodes a list of strings to a single string.
     * @param {string[]} strs
     * @return {string}
     */
    encode(strs) {
        let encodedString = "";
        for (let s of strs) {

```

```

        // Append length + '#' + string
        encodedString += s.length + "#" + s;
    }
    return encodedString;
}

/**
 * Decodes a single string to a list of strings.
 * @param {string} s
 * @return {string[]}
 */
decode(s) {
    const decodedList = [];
    let i = 0; // Pointer to track position

    while (i < s.length) {
        // Step 1: Find the index of the next '#'
        let j = i;
        while (s[j] !== '#') {
            j++;
        }

        // Step 2: Parse the length number
        // substring extracts characters from index 'i' up to (but not including) 'j'
        const lengthOfStr = parseInt(s.substring(i, j), 10);

        // Step 3: Extract the actual payload string
        const startOfStr = j + 1;
        const endOfStr = startOfStr + lengthOfStr;
        const actualString = s.substring(startOfStr, endOfStr);

        decodedList.push(actualString);

        // Step 4: Move pointer to the next chunk
        i = endOfStr;
    }

    return decodedList;
}
}

```

Note 1: Terminology and Techniques Used

- **Length Prefixing / TLV (Type-Length-Value):** This is the core algorithm used here. In network engineering, TLV is a standard way to format data. By putting the “Length” before the “Value”, the receiving system knows exactly how many bytes to allocate and read, preventing buffer overflows and making data extraction incredibly fast. It helps here because it removes the ambiguity of string parsing.
 - **Serialization / Deserialization:** The process of converting complex data structures (like objects or arrays) into a format that can be stored or transmitted (a string or byte array), and vice versa. This is a fundamental concept for saving data to databases or sending data across APIs.
-

Note 2: Real World / Interview Variations (Google, Meta, Bloomberg)

Top companies often test this concept without explicitly calling it “Encode and Decode Strings”. Here is how they mask it and how to solve it:

1. Variation: “Design the framing layer for a custom TCP protocol” (Meta / Google) * *The Setup:* TCP is a continuous “stream” of bytes. If you send “Hello” and then “World”, the receiver might just see “HelloWorld”. How do you separate the messages? * *How to solve:* You explain exactly what we did above: Length Prefixing (Framing). You prepend a 4-byte integer to every message indicating the message size. The receiver reads 4 bytes, figures out the size (N), and then reads the next N bytes as the payload.

2. Variation: “Serialize a complex, nested Dictionary to a text file without using JSON libraries” (Bloomberg) * *The Setup:* You have a dictionary like `{"name": "John", "notes": ["hello, world", "urgent"]}`. Save it to disk. * *How to solve:* You must use serialization. For strings within the dictionary that might contain your file delimiters, you use length prefixing to ensure they decode properly. You would establish rules: D means start dictionary, L means start list, and S means string (followed by our length prefix logic: `S4#John`).

3. Variation: “Chunked Transfer Encoding” (Google) * *The Setup:* You are an HTTP server sending a massive file to a client. You don’t know the total length of the file when you start sending. How do you stream it? * *How to solve:* Instead of sending the length of the *entire* file, you break the file into small pieces. You send the length of Piece 1, then Piece 1. Then the length of Piece 2, then Piece 2. When you are done, you send a length of 0 to signal the end. This is a direct real-world application of the algorithm you just learned!

1768. Merge Strings Alternately

Here is how a senior engineer approaches this problem. While coding platforms often categorize this as an introductory string problem, a senior engineer knows that mismanaging indices here leads to **Out-Of-Bounds** errors, and concatenating strings improperly leads to silent memory bloat. We are going to build a robust, production-grade solution.

1. Problem Explanation

The Goal: You are given two strings, `word1` and `word2`. Your task is to weave them together, one character at a time, strictly starting with `word1`.

The Catch (The Asymmetry): The strings might not be the same length. If you run out of characters in one string, you must take all the leftover characters from the longer string and attach them to the end of your merged result.

Examples:

- **Case 1: Equal Lengths** `word1 = "abc"`, `word2 = "pqr"` Result = "apbqcr" (a->p->b->q->c->r)
 - **Case 2: word1 is longer** `word1 = "abcd"`, `word2 = "pq"` Result = "apbqcd" (a->p->b->q... 'c' and 'd' are leftover and appended)
 - **Case 3: word2 is longer** `word1 = "ab"`, `word2 = "pqrs"` Result = "apbqrs" (a->p->b->q... 'r' and 's' are leftover and appended)
-

2. Solution Explanation

The Intuition (The “Why”): Think of this like a traffic zipper merge. You have two lanes of cars (`word1` and `word2`) converging into a single lane (our result). The traffic light lets one car from the left lane go, then one from the right, alternating perfectly.

However, if the left lane runs out of cars early, the right lane doesn’t stop; all the remaining cars in the right lane just drive straight through to the end.

The Algorithm (The “How”): We will use a single pointer (let’s call it `i`) acting as our index. 1. We start `i` at 0. 2. We check: “Is `i` a valid position in `word1`?” If yes, grab that character and add it to our result. 3. We check: “Is `i` a valid position in `word2`?” If yes, grab that character and add it to our result. 4. We increment `i` by 1 and repeat until `i` has surpassed the length of both strings.

Engineering Excellence Note: In many languages (like Python and JavaScript), strings are “immutable” (unchangeable). If you do `result = result + "a"`, the computer actually destroys the old string and allocates memory for a brand new one. In a loop, this is terribly inefficient. A senior engineer will always use

an Array (or List) to collect the characters, and then glue (join) them together exactly once at the end.

End-to-End Walkthrough with ASCII Visualizations:

Let's trace **Case 3**: word1 = "ab", word2 = "pqrs"

[INITIAL STATE]

word1: a b (length: 2)

word2: p q r s (length: 4)

Index i = 0

Result Array = []

[STEP 1: i = 0]

Check word1: Is 0 < 2? YES. Add 'a'. Result = ['a']

Check word2: Is 0 < 4? YES. Add 'p'. Result = ['a', 'p']

Visual:

word1: [a] b

word2: [p] q r s

Result Array: ['a', 'p']

[STEP 2: i = 1]

Check word1: Is 1 < 2? YES. Add 'b'. Result = ['a', 'p', 'b']

Check word2: Is 1 < 4? YES. Add 'q'. Result = ['a', 'p', 'b', 'q']

Visual:

word1: a [b]

word2: p [q] r s

Result Array: ['a', 'p', 'b', 'q']

[STEP 3: i = 2]

Check word1: Is 2 < 2? NO. Skip word1.

Check word2: Is 2 < 4? YES. Add 'r'. Result = ['a', 'p', 'b', 'q', 'r']

Visual:

word1: a b (exhausted)

word2: p q [r] s

Result Array: ['a', 'p', 'b', 'q', 'r']

[STEP 4: i = 3]

Check word1: Is 3 < 2? NO. Skip word1.

Check word2: Is 3 < 4? YES. Add 's'. Result = ['a', 'p', 'b', 'q', 'r', 's']

```

Visual:
word1:  a  b  (exhausted)
word2:  p  q  r [s]
Result Array: ['a', 'p', 'b', 'q', 'r', 's']

```

```

-----
[ STEP 5: i = 4 ]
Both strings are exhausted. Loop ends.
Final step: Join array into string -> "apbqrs"

```

3. Time and Space Complexity Analysis

Time Complexity: $O(M + N)$ where M is the length of `word1` and N is the length of `word2`. **Space Complexity:** $O(M + N)$ to hold the result array.

TIME COMPLEXITY DERIVATION: $O(M + N)$

We loop until we reach the maximum length between `word1` and `word2`. Let's say max length is K . We do K iterations. Inside the loop, we do constant time $O(1)$ checks and array pushes. Finally, joining an array of size $M+N$ takes $O(M+N)$ time. Overall time scales linearly with the total number of characters.

```

[ Execution Timeline ]
[ Iterating through max(M,N) characters ] ----> [ Joining M+N characters ]
Total Operations     $O(M + N)$ 

```

SPACE COMPLEXITY DERIVATION: $O(M + N)$

We must store every single character from both strings in a new data structure to form the answer.

```

[ Memory Allocation ]
word1 space  : [ a | b ]
word2 space  : [ p | q | r | s ]
Result space : [ a | p | b | q | r | s ]  <-- Requires exactly M + N slots

```

Therefore, extra space needed is exactly proportional to $M + N$.

4. Solution Code

I will present two approaches. The “Standard” one is what we just walked through. The “Optimized” one is what an L5 engineer writes to show they

know how to minimize unnecessary CPU cycles (by avoiding checking if `(i < length)` over and over again once one string is entirely empty).

Python Snippets

```
# Approach 1: Standard Single-Pass (Clean, easy to read)
# Time: O(M + N), Space: O(M + N)
def mergeAlternately_Standard(word1: str, word2: str) -> str:
    result = []
    i = 0
    # Loop until 'i' has passed the end of BOTH strings
    while i < len(word1) or i < len(word2):
        if i < len(word1):
            result.append(word1[i])
        if i < len(word2):
            result.append(word2[i])
        i += 1

    # Join array elements into a single string for O(N) performance
    return "".join(result)

# Approach 2: Optimized Chunking (The L5/L6 Answer)
# Time: O(M + N), Space: O(M + N)
def mergeAlternately_Optimized(word1: str, word2: str) -> str:
    """
    Optimization: Instead of asking "am I out of bounds?" on every single iteration,
    we only loop for the overlapping part. Once we hit the end of the shorter string,
    we just dump the entire remaining chunk of the longer string into the result at once.
    This saves CPU cycles on if-statements.
    """
    result = []
    min_length = min(len(word1), len(word2))

    # 1. Zip the overlapping parts together
    for i in range(min_length):
        result.append(word1[i])
        result.append(word2[i])

    # 2. Dump any remaining characters from word1 (will do nothing if word1 is shorter)
    result.append(word1[min_length:])

    # 3. Dump any remaining characters from word2 (will do nothing if word2 is shorter)
    result.append(word2[min_length:])

    return "".join(result)
```

JavaScript Snippets

```
// Approach 1: Standard Single-Pass (Clean, easy to read)
// Time:  $O(M + N)$ , Space:  $O(M + N)$ 
function mergeAlternately_Standard(word1, word2) {
    const result = [];
    const maxLength = Math.max(word1.length, word2.length);

    for (let i = 0; i < maxLength; i++) {
        // If i is within word1's bounds, push the char
        if (i < word1.length) {
            result.push(word1[i]);
        }
        // If i is within word2's bounds, push the char
        if (i < word2.length) {
            result.push(word2[i]);
        }
    }

    // Join array elements into a single string
    return result.join("");
}

// Approach 2: Optimized Chunking (The L5/L6 Answer)
// Time:  $O(M + N)$ , Space:  $O(M + N)$ 
function mergeAlternately_Optimized(word1, word2) {
    /*
     * Optimization: We avoid checking bounds inside the loop.
     * We iterate exactly up to the length of the shorter string.
     * Then we slice and append the remainder of whichever string is longer.
     */
    const result = [];
    const minLength = Math.min(word1.length, word2.length);

    // 1. Weave the overlapping prefix
    for (let i = 0; i < minLength; i++) {
        result.push(word1[i]);
        result.push(word2[i]);
    }

    // 2. Append the suffixes (substring will just return "" if there's no remainder)
    result.push(word1.substring(minLength));
    result.push(word2.substring(minLength));

    return result.join("");
}
```

Note 1: Terminology Addendum

String Immutability: * **What it is:** In languages like Python, Java, and JavaScript, once a string is created in memory, it cannot be changed. * **Why it matters here:** If you write `result = result + char`, the computer has to find a new spot in memory, copy the whole existing `result` string over, and add the new `char`. Doing this in a loop causes memory allocation to skyrocket (an $O(N^2)$ hidden cost). * **How it applies to this problem:** We completely bypass this issue by pushing characters into an Array/List (which *is* mutable and can dynamically resize efficiently) and then performing a single, optimized string `.join()` at the very end.

Two-Pointer / Index Tracking: * **What it is:** Using integer variables to represent our current reading position in a data structure. * **Why it matters here:** It allows us to track independent progress across multiple data sources simultaneously without needing nested loops or complex data manipulation.

Note 2: Real-World Interview Variations

While “Merge Strings Alternately” is a warmup, tech giants use this exact mental muscle for much harder system-level problems. Here is how Google, Meta, and Bloomberg evolve this logic:

Variation 1: “Merge K Sorted Lists / Arrays” (Very common at Meta/Google) * *Prompt:* Instead of 2 strings, you are given K sorted arrays of varying lengths. Merge them into one massive sorted array. * *How to solve:* You can’t just alternate `i` across K arrays because they need to be sorted by value, not just woven. The L5 solution introduces a **Min-Heap (Priority Queue)**. You put the first element of every array into the heap. You pop the smallest element, add it to your result array, and then push the *next* element from whichever array that popped element originated from.

Variation 2: “Merging Data Streams with Timestamps” (Bloomberg Systems Design flavor) * *Prompt:* You are receiving two separate real-time feeds of market trade data over a network. Sometimes Feed A is faster, sometimes Feed B is faster. Print them to the console strictly in chronological order based on their timestamps. * *How to solve:* This is the same two-pointer logic but applied to streams. You look at the “head” of Stream A and the “head” of Stream B. If `A.time < B.time`, you process A and advance A’s pointer. If `B.time < A.time`, you process B. If one stream temporarily disconnects (runs out of length), you might have to buffer the other stream.

Variation 3: “Merge Intervals” (Google/Meta staple) * *Prompt:* You are given an array of time intervals (e.g., meeting times: `[1, 3]`, `[2, 6]`, `[8, 10]`). Merge all overlapping intervals into a cohesive schedule (Result: `[1,`

6], [8, 10]). * *How to solve:* You first sort the arrays by their start times. Then, you use a single loop (like our optimized string chunking approach). You compare the end time of your last merged interval with the start time of the next interval. If they overlap, you fuse them by taking the `max()` of their end times. If they don't, you push the new interval to your result array.

2663. Lexicographically Smallest Beautiful String

Here is how a Senior Software Engineer (L5/L6) would break down and explain this problem. At this level, the focus shifts from “just hacking together a solution” to deeply understanding the mathematical boundaries of the problem, optimizing for memory, and communicating the “Aha!” moments clearly.

This is a hard problem because it disguises a math/number-system concept inside a string manipulation task. Let's deconstruct it.

1. Problem Explanation

The Premise: You are given a string `s` and an integer `k`. The alphabet you are allowed to use is the first `k` lowercase English letters (e.g., if `k=3`, your universe of characters is strictly `a`, `b`, `c`).

The Goal: Find the **lexicographically next** string that is “beautiful”. * **Lexicographically next:** Think of dictionary order. The “next” word after “apple” is “applf”. It's exactly like adding 1 to a number (e.g., $199 + 1 = 200$), but with letters. * **Beautiful:** The string cannot contain *any* palindromic substring of length 2 or more. (A palindrome reads the same forwards and backwards, like “racecar” or “bb”).

The L5/L6 “Aha!” Insight (The Palindrome Rule): The problem says “no palindromes of length 2 or greater”. This sounds terrifying to check. However, every large palindrome has a smaller palindrome at its core.

EVEN Length Palindrome Core:

```
a  [b  b]  a
   ^    ^
```

Matches at distance 1 (Adjacent)

ODD Length Palindrome Core:

```
r  a  [c  e  c]  a  r
      ^      ^
```

Matches at distance 2 (One char between)

Conclusion: If we strictly prevent any character from matching its immediate left neighbor (`s[i] == s[i-1]`) AND its second-left neighbor (`s[i] ==`

s[i-2]), it is mathematically impossible to form *any* palindrome of *any* length. The problem is now vastly simplified.

2. Solution Explanation

To find the “next” valid string, we treat the string like a number in base- k .

When you add 1 to the number 129 (base-10), you start at the rightmost digit (9). Because $9 + 1 = 10$ (which exceeds the max digit 9), you reset it to 0 and “carry over” to the left, incrementing the 2 to a 3. Result: 130.

We do the exact same thing with our string, starting from the rightmost character:

1. **Increment:** Try to move the character to the next letter in the alphabet.
2. **Validate:** Check if this new letter violates our Palindrome Rule (does it match the 1st or 2nd character to its left?). If it violates, increment it again!
3. **Carry Over:** If the character exceeds our allowed alphabet k , we cannot increment it anymore. We must move our pointer to the left and try incrementing *that* character instead.
4. **Greedy Fill:** Once we successfully increment a character at index i without exceeding k and without violating the Palindrome Rule, the prefix is locked in. Now, to make the string lexicographically *smallest*, every character to the right of i must be filled with the absolute lowest possible valid characters (starting from ‘a’).

Detailed ASCII Walkthrough Let’s trace $s = \text{"abc"}$, $k = 3$. Allowed alphabet: a, b, c.

Initial State:

```
String:  [ a ]  [ b ]  [ c ]
Indices:   0      1      2
Allowed: a, b, c
```

STEP 1: Start at the rightmost index ($i = 2$)

```
[ a ]  [ b ]  [ c ]
           ^
           i=2
```

Action: Increment ‘c’.

‘c’ + 1 = ‘d’.

‘d’ is strictly greater than our allowed $k=3$ (‘c’).

We must CARRY OVER. Move i to the left.

STEP 2: Carry over to $i = 1$

```
[ a ]  [ b ]  [ c ]
       ^
       i=1
```

Action: Increment 'b' -> 'c'.
 Validate 'c':
 - Does it match s[i-1] ('a')? No.
 - Does it match s[i-2] (None)? No.
 Valid!

Our locked prefix is now: [a, c]

 STEP 3: Greedy Fill to the right (j = 2)
 Because we incremented at i=1, everything to the right must be made as small as possible. We iterate j from i+1 to end.

Prefix: [a] [c] [?]
 ^
 j=2

We try the smallest characters starting from 'a':
 Try 'a':
 - Does 'a' match s[j-1] ('c')? No.
 - Does 'a' match s[j-2] ('a')? YES. (Palindrome "aca" formed!). 'a' is INVALID.

Try 'b':
 - Does 'b' match s[j-1] ('c')? No.
 - Does 'b' match s[j-2] ('a')? No.
 Valid!

 FINAL RESULT:
 [a] [c] [b]

Output: "acb"

The Pigeonhole Principle Insight: During the Greedy Fill phase, do we need to check the whole alphabet? **No.** We only need to avoid 2 previous characters (j-1 and j-2). Therefore, out of the first 3 letters in the alphabet (a, b, c), at least ONE will always be valid! We never need to check beyond 'c' during the greedy fill.

3. Time and Space Complexity Analysis

Time Complexity: O(N) Where N is the length of the string.

Visual Derivation of Time Complexity:
 String Length = N

Phase 1: Finding the increment point (Moving Left)
 <--- [Pointer]
 In the worst case, we carry over all the way to index 0.
 Max steps = N.
 At each step, we do at most 26 checks (size of alphabet). 26 is a constant.

Phase 2: Greedy Fill (Moving Right)
 [Pointer] --->
 Once we find the increment point, we move right to the end.
 Max steps = N.
 At each step, we check at most 3 letters ('a', 'b', 'c'). 3 is a constant.

Total Time = $O(\text{Phase 1}) + O(\text{Phase 2})$
 Total Time = $O(N) + O(N) = O(2N)$
 Drop the constant $\rightarrow O(N)$ Time Complexity.

Space Complexity: $O(N)$

Visual Derivation of Space Complexity:
 Strings are immutable in Python and JavaScript.
 To modify characters in-place efficiently, we must convert the string into an Array/List of characters.

Input String: "a b c d e"
 Converted: ['a', 'b', 'c', 'd', 'e'] $\rightarrow$ Size N

This array scales linearly with the input string length.
 Therefore, Space = $O(N)$.

4. Solution Code

Here are the optimal, well-commented implementations.

Python Snippet

```
def smallestBeautifulString(s: str, k: int) -> str:
    # Convert to list for  $O(1)$  character modifications
    chars = list(s)
    n = len(chars)
    i = n - 1

    # Calculate the string representation of our hard limit
    limit = chr(ord('a') + k)

    # PHASE 1: Find the rightmost character we can increment
```

```

while i >= 0:
    # Increment the current character
    chars[i] = chr(ord(chars[i]) + 1)

    # If it exceeds our alphabet limit, reset/carry-over to the left
    if chars[i] == limit:
        i -= 1
    # Check Palindrome rule: matches immediate left or second left?
    elif (i >= 1 and chars[i] == chars[i-1]) or \
        (i >= 2 and chars[i] == chars[i-2]):
        # Invalid. The while loop will continue and increment it AGAIN
        continue
    else:
        # Valid increment found! Break out of Phase 1.
        break

# If we carried over past the first character, no valid string exists
if i < 0:
    return ""

# PHASE 2: Greedy Fill the remaining suffix with the smallest valid chars
for j in range(i + 1, n):
    # By the Pigeonhole Principle, one of 'a', 'b', or 'c' will ALWAYS work
    for candidate in ['a', 'b', 'c']:
        # Ensure we don't create a palindrome with the new prefix
        if (j >= 1 and candidate == chars[j-1]) or \
            (j >= 2 and candidate == chars[j-2]):
            continue

        # Found the smallest valid char, lock it in and move to next j
        chars[j] = candidate
        break

return "".join(chars)

```

JavaScript Snippet

```

var smallestBeautifulString = function(s, k) {
    // Convert string to array of char codes for easy math
    let chars = s.split('').map(c => c.charCodeAt(0));
    let n = chars.length;
    let i = n - 1;

    // ASCII code for 'a' is 97. The limit is 'a' + k.
    const LIMIT = 97 + k;

```

```

// PHASE 1: Find the valid increment point
while (i >= 0) {
    chars[i]++; // Increment character

    if (chars[i] === LIMIT) {
        // Exceeded allowed alphabet. Move left to carry over.
        i--;
    } else if ((i >= 1 && chars[i] === chars[i-1]) ||
        (i >= 2 && chars[i] === chars[i-2])) {
        // Creates a palindrome. Loop repeats to increment again.
        continue;
    } else {
        // Valid character found!
        break;
    }
}

// If we carried over beyond the first index, it's impossible
if (i < 0) return "";

// PHASE 2: Greedy Fill
for (let j = i + 1; j < n; j++) {
    // Test ASCII 97 ('a'), 98 ('b'), 99 ('c')
    for (let cand = 97; cand <= 99; cand++) {
        if ((j >= 1 && cand === chars[j-1]) ||
            (j >= 2 && cand === chars[j-2])) {
            continue;
        }
        chars[j] = cand;
        break;
    }
}

// Convert ASCII codes back to string
return String.fromCharCode(...chars);
};

```

Note 1: Terms and Techniques

- **Greedy Algorithm:** Used in Phase 2. A greedy algorithm makes the locally optimal choice at each step with the hope of finding a global optimum. Here, picking the absolutely smallest valid character ('a', 'b', or 'c') from left-to-right guarantees the whole string remains lexicographically smallest.

- **Pigeonhole Principle:** A math concept stating that if you have N items and put them into M containers (where $N > M$), at least one container must contain more than one item. Applied here: We only need to avoid 2 containers (the previous 2 characters). If we test 3 items ('a', 'b', 'c'), at least one is guaranteed to be safe.
- **Base-K Number System Analogy:** Treating strings like numbers. Instead of digits 0-9 (Base-10), we use letters a-z up to limit k . The “carry over” logic when `chars[i] == limit` perfectly mirrors grade-school addition.

Note 2: Real-World & Indirect Interview Variations

1. Meta (Facebook): “Next Closest Time” or “Next Permutation”

* *The Twist:* Given a string representing a time (e.g., “19:34”), find the next closest time by reusing the same digits. * *How an L5 solves it:* This maps to the same “Increment and Validate” mindset. You start from the rightmost digit, try to increment it using available characters. The validation logic changes from “no palindromes” to “must be a valid clock time” (hours < 24, minutes < 60). The “carry over” logic is identical.

2. Google: “Valid Network Routing Paths” * *The Twist:* You are given an array representing a path through network nodes (e.g., [NodeA, NodeC, NodeF]). You need to find the next lexicographically ordered path of the exact same length. Rule: A path is invalid if a packet bounces between two immediate nodes (A->C->A) or loops back after one hop (A->B->C->A). * *How an L5 solves it:* This is LeetCode 2663 *exactly*, just disguised with networking terminology. A->C->A is a length 2 palindrome. A->B->C->A is a length 3 palindrome check. The nodes are the alphabet. You apply the exact same algorithm.

3. Bloomberg: “Financial Ticker Generation” * *The Twist:* You are generating new 4-letter stock ticker symbols sequentially. However, certain “ugly” patterns are banned by the exchange (e.g., repeating adjacent letters, or letters repeating with one gap, to prevent visual confusion). Given the current ticker, what is the next valid one? * *How an L5 solves it:* Again, this is a 1-to-1 mapping. Tickers are strings. The validation rule is literally the palindrome check. You solve it using the Base-26 array carry-over and greedy fill.

202. Happy Number

As an engineer, approaching a problem like “Happy Number” requires looking past the immediate math and recognizing the underlying data structure it represents. To a senior engineer, this isn’t just an arithmetic problem; it is a cycle detection problem.

Here is the end-to-end walkthrough of how a top-band engineer would break

down, visualize, and solve LeetCode 202.

1. Problem Explanation

The Rule: You are given a positive integer. You must repeatedly replace that number with the sum of the squares of its digits.

The Two Outcomes: 1. **The Happy Case:** The sequence eventually reaches the number 1. Once it hits 1, it stays at 1 forever (since $1^2 = 1$). 2. **The Unhappy Case:** The sequence never reaches 1. Instead, it falls into an endless cycle of repeating numbers.

The Underlying Question (The “Why”): Before writing any code, a senior engineer asks: *“Could the numbers just keep getting bigger and bigger infinitely, meaning it never hits 1 and never loops?”*

We must prove the numbers shrink. Let’s look at the worst-case scenario for a 3-digit number: 999. Sum of squares: $9^2 + 9^2 + 9^2 = 81 + 81 + 81 = 243$. Even for a massive 4-digit number like 9999, the sum is 324.

Any number larger than 3 digits will drastically collapse into a smaller number (specifically, into the 1 to 243 range). Because the pool of numbers it can collapse into is strictly limited, by the Pigeonhole Principle, the sequence **must** eventually repeat a number if it does not hit 1.

Therefore, infinite growth is impossible. It will always either hit 1 or hit a cycle.

2. Solution Explanation

Let’s visualize both cases to understand exactly what we are building.

Case 1 Walkthrough: $n = 19$ (A Happy Number)

Number		Calculation		Result
-----		-----		-----
19	->	$1^2 + 9^2$	->	$1 + 81 \rightarrow 82$
82	->	$8^2 + 2^2$	->	$64 + 4 \rightarrow 68$
68	->	$6^2 + 8^2$	->	$36 + 64 \rightarrow 100$
100	->	$1^2 + 0^2 + 0^2$	->	$1 + 0 + 0 \rightarrow 1$ (Happy!)

Case 2 Walkthrough: $n = 2$ (An Unhappy Number)

Start: 2

[2]	->	4	->	16	->	37	->	58	->	89	->	145	->	42	->	20	->	[4]
		^																

+-----+
 (The sequence loops back to 4 indefinitely)

Approach 1: The HashSet (The Standard Solution) If we know a cycle is the only alternative to reaching 1, we can just use a “memory” to track every number we see. * **How:** We use a Hash Set. We calculate the next number. If it is 1, return true. If it is already in our Hash Set, we have detected a cycle, so return false. If neither, add it to the set and repeat. * **Pros:** Very intuitive. * **Cons:** It requires extra memory to store every step.

Approach 2: Floyd’s Cycle Detection (The L6 Optimization) We can visualize the sequence of numbers exactly like a Linked List. * 19 -> 82 -> 68 -> 100 -> 1 -> 1 -> 1... * 2 -> 4 -> 16 -> 37 -> ... -> 20 -> 4 -> 16...

Instead of keeping a memory of all numbers, we use two pointers: a **Slow** pointer and a **Fast** pointer. * **How:** Slow takes one step (calculates the sum of squares once). Fast takes two steps (calculates the sum of squares, then calculates it again on the result). * If there is a cycle, the Fast pointer will eventually “lap” the Slow pointer and they will land on the exact same number. If there is no cycle, the Fast pointer will hit 1 first. * **Why this is better:** It requires absolutely zero extra memory, regardless of how long the chain is.

3. Time and Space Complexity Analysis

Here is the derivation of the complexity, mapped out sequentially without relying on complex math notations.

```
+-----+
| TIME COMPLEXITY DERIVATION: O(log N) |
+-----+
| N = The starting number |
| |
| 1. Cost of calculating the sum of squares: |
|   To extract digits, we divide by 10 repeatedly. |
|   A number N has roughly (log10 N) digits. |
|   So, one step of calculation takes O(log N) time. |
| |
| 2. Number of steps required: |
|   As proven earlier, numbers collapse very quickly. |
|   Once the number falls below 243, the maximum length of any cycle |
|   is a fixed constant (we know the longest cycle under 243 is short). |
|   Therefore, the number of steps is bound by a constant. |
| |
| TOTAL TIME: Constant steps * O(log N) per step =====> O(log N) |
+-----+
```

+-----+ +-----+ SPACE COMPLEXITY DERIVATION +-----+ Approach 1: Hash Set - We store every number generated in a set. - The number of elements added is bound by a constant once it drops below 243. However, initially, the number is stored, taking $O(\log N)$ space for large numbers. - Space: $O(\log N)$ Approach 2: Fast & Slow Pointers - We only store two integer variables: 'slow' and 'fast'. - No data structures grow with the size of the input. - Space: $O(1)$ +-----+	
--	--

4. Solution Code

I will provide both the HashSet and the Fast/Slow Pointer solutions in Python and JavaScript. An interviewer expects you to code the Fast/Slow pointer method to demonstrate deep algorithmic understanding.

Solution 1: Fast and Slow Pointers (Optimized Space) Python Snippet

```
def isHappy(n: int) -> bool:
    # Helper function to calculate the sum of squares of digits
    def get_next(number):
        total_sum = 0
        while number > 0:
            # Extract the last digit using modulo
            number, digit = divmod(number, 10)
            # Add the square of the digit to the running total
            total_sum += digit ** 2
        return total_sum

    # Initialize both pointers at the starting number
    slow = n
    fast = get_next(n)

    # Loop until the fast pointer reaches 1 (happy)
    # or the fast pointer laps the slow pointer (cycle detected)
    while fast != 1 and slow != fast:
```

```

        slow = get_next(slow)           # Slow takes 1 step
        fast = get_next(get_next(fast)) # Fast takes 2 steps

    # If the loop breaks because fast == 1, it's a happy number.
    return fast == 1

```

JavaScript Snippet

```

function isHappy(n) {
    // Helper function to calculate the sum of squares of digits
    const getNext = (number) => {
        let totalSum = 0;
        while (number > 0) {
            // Extract last digit
            let digit = number % 10;
            totalSum += digit * digit;
            // Remove the last digit from the number
            number = Math.floor(number / 10);
        }
        return totalSum;
    };

    let slow = n;
    let fast = getNext(n);

    // Continue until fast hits 1 or fast catches up to slow
    while (fast !== 1 && slow !== fast) {
        slow = getNext(slow);           // Move 1 step
        fast = getNext(getNext(fast)); // Move 2 steps
    }

    return fast === 1;
}

```

Solution 2: Hash Set (Unoptimized Space, Good for initial thought process) Python Snippet

```

def isHappy_set(n: int) -> bool:
    def get_next(number):
        total_sum = 0
        while number > 0:
            number, digit = divmod(number, 10)
            total_sum += digit ** 2
        return total_sum

    # A set to keep track of all numbers we have seen
    seen = set()

```

```

    # Loop as long as we haven't reached 1 and haven't seen the number before
    while n !== 1 and n not in seen:
        seen.add(n)
        n = get_next(n)

    return n == 1

```

JavaScript Snippet

```

function isHappySet(n) {
    const getNext = (number) => {
        let totalSum = 0;
        while (number > 0) {
            let digit = number % 10;
            totalSum += digit * digit;
            number = Math.floor(number / 10);
        }
        return totalSum;
    };

    const seen = new Set();

    while (n !== 1 && !seen.has(n)) {
        seen.add(n);
        n = getNext(n);
    }

    return n === 1;
}

```

Note 1: Terminology and Techniques

Floyd's Cycle-Finding Algorithm (Tortoise and Hare): This is the algorithm used in Approach 2. It is a pointer algorithm that uses two pointers moving through a sequence at different speeds. * **Why it helps:** It is strictly designed to detect cycles in any sequence (most commonly Linked Lists) without using extra memory (Hash Sets/Maps) to store the history of visited nodes. * **How it applies here:** Because replacing a number with its sum of squares creates a deterministic chain of numbers, that chain is functionally identical to a singly linked list. Applying Floyd's algorithm allows us to achieve $O(1)$ space complexity.

Note 2: Real-World Interview Variations

Because “Happy Number” is heavily algorithmic, top tech companies rarely ask it exactly as written. They abstract the “cycle detection” concept into system design or graph problems.

1. Google: “Detecting Infinite Loops in State Machines”

* *Prompt:* You are given a program state simulator. The function `getNextState(currentState)` returns the next state of the system based on complex internal logic. A state is “terminal” if `getNextState` returns the state itself. Write a system that determines if the machine will eventually terminate or if it is stuck in an infinite loop. * *Solution:* This is the exact same problem. You do not need to understand the internal logic of `getNextState`. You apply Floyd’s Tortoise and Hare algorithm. `slow = getNextState(start)`, `fast = getNextState(getNextState(start))`. If they meet before reaching a terminal state, the machine is stuck in a loop.

2. Meta: “Deadlock Detection in Resource Dependency” * *Prompt:*

In an operating system, processes request resources. Process A is waiting on Resource 1, which is held by Process B, which is waiting on Resource 2, which is held by Process A. Detect if a deadlock (a circular dependency) exists in a chain of processes. * *Solution:* While this can be solved with Depth First Search (DFS) and a “visited” set (similar to our HashSet approach), treating the strict dependency chain with Fast and Slow pointers can detect the cycle with minimal memory overhead, preventing the deadlock detection system itself from causing out-of-memory errors on massive process chains.

3. Bloomberg: “Circular References in Financial Entity Structures”

* *Prompt:* A corporate holding structure is defined where Company A owns Company B, Company B owns C, etc. We need to ensure there are no illegal circular ownership loops (e.g., C owns a stake in A). Given a function `getPrimarySubsidiary(company)`, check for cycles. * *Solution:* Again, this is cycle detection. Since Bloomberg deals with massive datasets of corporate entities, keeping a Hash Set of every company might be memory-heavy during a large batch process. Using Fast/Slow pointers moving through the ownership chain identifies the cycle in $O(1)$ space.

383. Ransom Note

As a senior engineer looking at “Ransom Note” (LeetCode 383), I see a problem that is fundamentally about resource allocation and inventory management. While it is categorized as an “Easy” problem, in a top-tier interview (Google, Meta, Bloomberg), the interviewer isn’t just looking for a working solution. They are evaluating whether you immediately jump to the most optimal data structure and if you can clearly articulate why you chose it over the alternatives.

Here is the detailed, end-to-end walkthrough of how I would break this down,

completely grounded in visuals.

1. Problem Explanation

The Core Question: You are given two strings: `ransomNote` and `magazine`. You need to figure out if you can build the `ransomNote` by cutting out individual letters from the `magazine`. Each letter you cut out from the `magazine` can only be used exactly once. Return `true` if you can, and `false` if you cannot.

The Golden Rules: 1. **Existence:** Every character in the ransom note must exist in the magazine. 2. **Quantity:** If the ransom note needs three 'a's, the magazine must have *at least* three 'a's. Two is not enough.

Visualizing the Cases:

Case A: Valid Construction (True) Input: `ransomNote` = "aa", `magazine` = "aab"

Ransom Note Needs:	Magazine Has:
[a] [a]	[a] [a] [b]

Step 1: Match first 'a'

[a] <----->	[a] (Used!)
[a]	[a] [b]

Step 2: Match second 'a'

(Used!)	(Used!)
[a] <----->	[a] [b]

Result: TRUE. We found all required letters.
(It doesn't matter that 'b' was left over).

Case B: Invalid - Missing Character (False) Input: `ransomNote` = "cat", `magazine` = "car"

Ransom Note Needs:	Magazine Has:
[c] [a] [t]	[c] [a] [r]

Step 1: Match 'c'

[c] <----->	[c] (Used!)
[a] [t]	[a] [r]

Step 2: Match 'a'

(Used!)	(Used!)
[a] <----->	[a] [r]

Step 3: Match 't'

```
[t] <-----/ X /-----> (No 't' found! Only 'r' is left)
```

Result: FALSE. We cannot finish the note.

Case C: Invalid - Not Enough Quantity (False) Input: ransomNote = "aa", magazine = "ab"

Ransom Note Needs:	Magazine Has:
[a] [a]	[a] [b]

Step 1: Match first 'a'

[a] <----->	[a] (Used!)
[a]	[b]

Step 2: Match second 'a'

```
[a] <-----/ X /-----> (No more 'a's! Only 'b' is left)
```

Result: FALSE. We ran out of the letter 'a'.

2. Solution Explanation

Let's discuss how to translate this physical "cutting out" process into code.

Approach 1: The Brute Force "Search and Destroy" (Unoptimized)

A junior approach might be to simulate exactly what a human does. For every letter in the ransom note, scan the magazine to find it. If found, cross it out (delete it from the magazine string) so it can't be used again.

- **Why it's bad:** Strings are immutable in most languages (like Python and JavaScript). To "cross out" a letter, you have to create a whole new copy of the string without that letter. Doing this over and over is incredibly slow and wastes memory.

Approach 2: The Frequency Counter / Inventory Map (The Optimized L5 Approach)

Instead of searching for letters one by one, let's act like a smart warehouse manager. First, we do a full inventory of our warehouse (the `magazine`). We count exactly how many of each letter we have. Then, we look at our packing list (the `ransomNote`). For every letter we need, we check our inventory. If we have it, we take one (decrement the count). If our inventory for that letter is 0 (or it doesn't exist), we immediately know we cannot fulfill the order.

Since the problem guarantees that the inputs only contain lowercase English letters, we don't even need a complex Hash Map. We can use a simple Array of 26 slots, where index 0 represents 'a', index 1 represents 'b', and so on.

Walkthrough Visualization: ransomNote = “aab”, magazine = “baaab”

Step 1: Build the Inventory from Magazine ("baaab")
Create an array of 26 buckets (all starting at 0).
[a:0, b:0, c:0, ... z:0]

Read 'b' -> Add 1 to 'b' bucket: [a:0, b:1, c:0...]
Read 'a' -> Add 1 to 'a' bucket: [a:1, b:1, c:0...]
Read 'a' -> Add 1 to 'a' bucket: [a:2, b:1, c:0...]
Read 'a' -> Add 1 to 'a' bucket: [a:3, b:1, c:0...]
Read 'b' -> Add 1 to 'b' bucket: [a:3, b:2, c:0...]

Final Inventory: 3 'a's, 2 'b's.

Step 2: Process the Ransom Note ("aab")

Read 'a':
Do we have 'a'? Yes, we have 3.
Take one. Inventory 'a' becomes 2.

Read 'a':
Do we have 'a'? Yes, we have 2.
Take one. Inventory 'a' becomes 1.

Read 'b':
Do we have 'b'? Yes, we have 2.
Take one. Inventory 'b' becomes 1.

Did we run out of any needed letter? No.
Result: TRUE.

Note on an immediate optimization: If the length of the ransom note is strictly greater than the length of the magazine, it is physically impossible to construct the note. We can add a one-line check at the very beginning to return **false** instantly and save time.

3. Time and Space Complexity Analysis

Let M be the length of the magazine string. Let N be the length of the ransomNote string.

=====

TIME COMPLEXITY: $O(M + N)$

=====

Derivation Diagram:

```

[ Initial Size Check ] -----> O(1)
    |
    V
[ Loop through Magazine (Length M) ]
    |-- Read char, math operation to find index -> O(1)
    |-- Increment array value -----> O(1)
    Total for this step: M * O(1) -----> O(M)
        |
        V
[ Loop through Ransom Note (Length N) ]
    |-- Read char, math operation to find index -> O(1)
    |-- Check if value == 0 (Fail case) -----> O(1)
    |-- Decrement array value -----> O(1)
    Total for this step: N * O(1) -----> O(N)
        |
        V
Total Time = O(1) + O(M) + O(N) = O(M + N)

```

*Note: In the worst case, we look at every character in both strings exactly once. This is linear time.

```

=====
SPACE COMPLEXITY: O(1) Constant Space
=====
Derivation Diagram:

```

```

[ Memory Allocated ]
    |
    |--> Array of exactly 26 integers.
        V
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ... 0 ] (26 slots)

```

No matter how large M or N gets (whether the magazine is 10 letters or 10 billion letters), the array we create to store the frequencies will always be exactly 26 integers in size.

Because the memory footprint does not grow as the input grows, Space Complexity is strictly O(1).

```

=====

```

4. Solution Code

JavaScript

```

/**
 * Approach 1: Brute Force with String Replacement (Unoptimized)
 * Time:  $O(N * M)$  - Because replace() scans the string
 * Space:  $O(M)$  - Because we create modified copies of the magazine
 */
function canConstructBrute(ransomNote, magazine) {
    let magStr = magazine;
    for (let char of ransomNote) {
        // If the character isn't in our current magazine string, fail.
        if (!magStr.includes(char)) return false;
        // Replace ONLY the first occurrence of the character with empty string
        magStr = magStr.replace(char, "");
    }
    return true;
}

/**
 * Approach 2: Frequency Array (Optimized L5 Approach)
 * Time:  $O(M + N)$ , Space:  $O(1)$ 
 */
function canConstruct(ransomNote, magazine) {
    // Quick Optimization: If note is longer than magazine, it's impossible.
    if (ransomNote.length > magazine.length) return false;

    // Create an array of 26 zeros to represent letter frequencies
    const inventory = new Array(26).fill(0);

    // Step 1: Stock the inventory using the magazine
    for (let i = 0; i < magazine.length; i++) {
        // .charCodeAt(0) gets the ASCII number. Subtracting 97 (ASCII for 'a')
        // maps 'a' to index 0, 'b' to 1, etc.
        const index = magazine.charCodeAt(i) - 97;
        inventory[index]++;
    }

    // Step 2: Fulfill the ransom note
    for (let i = 0; i < ransomNote.length; i++) {
        const index = ransomNote.charCodeAt(i) - 97;

        // If the bucket for this letter is 0, we don't have enough.
        if (inventory[index] === 0) {
            return false;
        }

        // Otherwise, use one letter.
        inventory[index]--;
    }
}

```

```

    }

    return true;
}

```

Python

```

# Approach 1: Brute Force with List Modification (Unoptimized)
# Time: O(N * M)
# Space: O(M) - Converting magazine to a list takes extra memory
def canConstruct_brute(ransomNote: str, magazine: str) -> bool:
    mag_list = list(magazine)

    for char in ransomNote:
        if char in mag_list:
            # remove() finds the first occurrence and deletes it. Takes O(M) time.
            mag_list.remove(char)
        else:
            return False

    return True

# Approach 2: Frequency Array (Optimized L5 Approach)
# Time: O(M + N), Space: O(1)
def canConstruct(ransomNote: str, magazine: str) -> bool:
    # Quick Optimization: If note is longer than magazine, it's impossible.
    if len(ransomNote) > len(magazine):
        return False

    # Create an array of 26 zeros
    inventory = [0] * 26

    # Step 1: Stock the inventory using the magazine
    for char in magazine:
        # ord() gets the ASCII value. ord('a') is 97.
        index = ord(char) - ord('a')
        inventory[index] += 1

    # Step 2: Fulfill the ransom note
    for char in ransomNote:
        index = ord(char) - ord('a')

        # If we have 0 left of this character, we fail.
        if inventory[index] == 0:
            return False

```

```

        # Otherwise, take one from the inventory.
        inventory[index] -= 1

    return True

# Note: In a real Python interview, you could also use collections.Counter:
# return not collections.Counter(ransomNote) - collections.Counter(magazine)
# However, implementing the array logic manually shows a deeper understanding
# of memory management and underlying data structures.

```

Note 1: Terms and Techniques Used

- **Frequency Counting / Hashing:** This is the primary algorithm used here. It is the process of mapping a piece of data (in this case, a character) to a location in memory where we store how many times we've seen it.
 - **Why it helps:** It prevents us from having to do nested loops $O(N * M)$. By spending a tiny, fixed amount of memory (the 26-slot array) to store state, we can process each string in a single, independent pass. It is the classic “trade Space for Time” optimization, though here the space is so small it is considered constant $O(1)$.
-

Note 2: Real-World & Interview Variations

In a top-tier interview, the baseline solution above will take you about 10 minutes. The interviewer will spend the remaining 35 minutes modifying the constraints. Here is what they will ask:

Variation 1: “What if the magazine is not a string, but a massive stream of characters coming over a network that doesn’t fit in memory?” (Classic Google Systems/Algo hybrid question) * The Catch: You cannot do `for char in magazine` because the magazine is too big to hold in RAM, and it might be infinitely long. * **The Solution:** You reverse the logic. First, you build your Frequency Array based on the `ransomNote` (which is short). You keep a counter of `totalLettersNeeded = ransomNote.length`. Then, you start reading the magazine stream one character at a time. If the character is in your array (value > 0), you decrement the array and decrement `totalLettersNeeded`. The moment `totalLettersNeeded` hits 0, you return `true` and close the stream. If the stream ends and `totalLettersNeeded` is > 0 , you return `false`.

Variation 2: “What if the characters are Unicode (emojis, Chinese characters, etc.) instead of lowercase English letters?” (Meta / Bloomberg) * The Catch: An array of 26 slots will immediately crash because

Unicode has over 140,000 characters. You cannot map an emoji to an index using `ord(char) - ord('a')`. * **The Solution:** You must swap the fixed Array for a dynamic **Hash Map** (a Dictionary in Python, or a Map/Object in JS). The character itself becomes the key, and the count becomes the value. The Time Complexity remains $O(M + N)$, but the Space Complexity changes from $O(1)$ to $O(U)$, where U is the number of *unique* characters in the magazine.

Variation 3: “Word-based Ransom Note” (Meta) * **The Catch:** Instead of characters, you must form a sentence using whole words cut out from a magazine paragraph. Case matters, and punctuation might be attached. * **The Solution:** First, sanitize the inputs (e.g., strip punctuation). Then, split both strings by spaces into arrays of words. Apply the exact same Hash Map Frequency Counter technique, but use the whole string words as the keys instead of single characters.

205. Isomorphic Strings

Here is an end-to-end, top-tier (L5/L6) walkthrough for solving “Isomorphic Strings.”

An L5 (Senior) or L6 (Staff) engineer approaches this problem by immediately recognizing it as a “Cipher” or “Mapping” problem. They won’t just rush into coding; they will clearly identify the trap that catches junior engineers: assuming a one-way map is enough.

1. Problem Explanation

The Core Goal: You are given two strings, s and t . You need to determine if you can swap every unique character in s with a character in t (or itself) to perfectly recreate t .

The Rules of the “Secret Cipher”: Think of this like a decoder ring. 1. **Consistency:** If ‘e’ translates to ‘a’ once, every single ‘e’ in the string must translate to ‘a’. You cannot have ‘e’ translate to ‘a’ in one place and ‘x’ in another. 2. **Uniqueness (The Trap):** No two different letters can translate to the same letter. For example, ‘x’ and ‘y’ cannot both translate to ‘z’. Each letter in t must map back to exactly one letter in s . This strict two-way relationship is called a **bijection**. 3. **Preservation of Order:** The positions of the characters cannot change.

2. Solution Explanation

To solve this, we need a way to keep track of the translations as we read through the strings left to right. We will use a Hash Map (Dictionary) to store our

“Decoder Ring” rules.

The Junior Engineer Mistake (One-Way Mapping): A junior engineer will only map **s** to **t**. Let’s look at why this fails with **s = "badc"** and **t = "baba"**. * ‘b’ maps to ‘b’ (Valid) * ‘a’ maps to ‘a’ (Valid) * ‘d’ maps to ‘b’ (Valid in a one-way map, but INVALID overall because ‘b’ in **s** already maps to ‘b’ in **t**!)

The L5/L6 Solution (Two-Way Mapping): To ensure absolute uniqueness, we must maintain TWO maps: 1. Map **S_to_T**: Tracks what a character in **s** translates to in **t**. 2. Map **T_to_S**: Tracks what a character in **t** translates back to in **s**.

As we iterate through the strings, we check if the current characters violate any rules established in our two maps.

Let’s visualize this with two full examples.

Case 1: Valid Isomorphic Strings (**s = "egg", t = "add"**)

```
=====
INITIAL STATE
=====
Strings:  s = "egg", t = "add"
Pointers: i = 0

Map_S_to_T: {} (Empty)
Map_T_to_S: {} (Empty)

=====
STEP 1: Index 0
=====
Characters: s[0] = 'e', t[0] = 'a'

Check:
- Is 'e' in Map_S_to_T? NO.
- Is 'a' in Map_T_to_S? NO.

Action: Record the new mappings.
Map_S_to_T: {'e' -> 'a'}
Map_T_to_S: {'a' -> 'e'}

=====
STEP 2: Index 1
=====
Characters: s[1] = 'g', t[1] = 'd'

Check:
```

- Is 'g' in Map_S_to_T? NO.
- Is 'd' in Map_T_to_S? NO.

Action: Record the new mappings.

Map_S_to_T: {'e' -> 'a', 'g' -> 'd'}

Map_T_to_S: {'a' -> 'e', 'd' -> 'g'}

```
=====
STEP 3: Index 2
=====
Characters: s[2] = 'g', t[2] = 'd'
```

Check:

- Is 'g' in Map_S_to_T? YES. It maps to 'd'. Does this match t[2]? YES ('d' == 'd').
- Is 'd' in Map_T_to_S? YES. It maps to 'g'. Does this match s[2]? YES ('g' == 'g').

Action: Valid mapping confirmed. Continue.

```
=====
CONCLUSION
=====
Reached the end of the strings with no conflicts.
Return TRUE.
```

Case 2: Invalid Isomorphic Strings (s = "foo", t = "bar")

```
=====
INITIAL STATE
=====
Strings: s = "foo", t = "bar"
Pointers: i = 0

Map_S_to_T: {}
Map_T_to_S: {}
```

```
=====
STEP 1: Index 0
=====
Characters: s[0] = 'f', t[0] = 'b'
```

Action: Record mapping.

Map_S_to_T: {'f' -> 'b'}

Map_T_to_S: {'b' -> 'f'}

```
=====
STEP 2: Index 1
```

```
=====
Characters: s[1] = 'o', t[1] = 'a'
```

```
Action: Record mapping.
Map_S_to_T: {'f' -> 'b', 'o' -> 'a'}
Map_T_to_S: {'b' -> 'f', 'a' -> 'o'}
```

```
=====
STEP 3: Index 2
=====
```

```
Characters: s[2] = 'o', t[2] = 'r'
```

Check:

- Is 'o' in Map_S_to_T? YES. It is recorded as mapping to 'a'.
- Look at t[2]. It is 'r'.
- CONFLICT DETECTED! Our map says 'o' MUST become 'a', but the string is asking it to become

Action: Return FALSE immediately.

3. Time and Space Complexity Analysis

Let N be the length of the string s.

TIME COMPLEXITY DERIVATION: Order of N

Input Length: N characters

Operation Loop:

For i from 0 to N-1:	---> Loops N times
Look up s[i] in Map_S_to_T	---> Order of 1 time
Look up t[i] in Map_T_to_S	---> Order of 1 time
Insert s[i] and t[i] into Maps	---> Order of 1 time

Total Time = N loops * (Order of 1 operations)

Total Time = Order of N

SPACE COMPLEXITY DERIVATION: Order of 1 (Constant Space)

Wait, why is it Order of 1 and not Order of N?

This is a detail a Staff Engineer will point out to impress the interviewer.

What are we storing in memory?

We are storing characters in our Hash Maps.

How many unique characters can possibly exist?

The problem usually specifies standard ASCII characters.
There are exactly 256 standard ASCII characters.

Worst Case Scenario: Every character in the string is unique.
Our maps will grow, but they will STOP growing at 256 entries.
Even if the string is 10 million characters long, the maps never exceed a size of 256.

Because the maximum size of the Hash Map is bounded by a fixed constant (256),
the space required does not scale infinitely with N.

Total Space = Order of 1 (Constant)

4. Solution Code

Here is the implementation using the dual-map strategy.

Python Solution

```
def isIsomorphic(s, t):
    # Base case: If lengths are different, they cannot be isomorphic
    if len(s) != len(t):
        return False

    map_s_to_t = {}
    map_t_to_s = {}

    # Iterate through both strings simultaneously
    for i in range(len(s)):
        char_s = s[i]
        char_t = t[i]

        # Check for conflict in the S -> T mapping
        if char_s in map_s_to_t:
            if map_s_to_t[char_s] != char_t:
                return False
        else:
            map_s_to_t[char_s] = char_t

        # Check for conflict in the T -> S mapping
        if char_t in map_t_to_s:
            if map_t_to_s[char_t] != char_s:
                return False
        else:
            map_t_to_s[char_t] = char_s
```

```
# If we made it through the whole string without conflicts, it's valid
return True
```

JavaScript Solution

```
/**
 * @param {string} s
 * @param {string} t
 * @return {boolean}
 */
function isIsomorphic(s, t) {
    // Base case: lengths must match
    if (s.length !== t.length) {
        return false;
    }

    const mapSToT = new Map();
    const mapTToS = new Map();

    for (let i = 0; i < s.length; i++) {
        const charS = s[i];
        const charT = t[i];

        // Check for conflict in the S -> T mapping
        if (mapSToT.has(charS)) {
            if (mapSToT.get(charS) !== charT) {
                return false;
            }
        } else {
            mapSToT.set(charS, charT);
        }

        // Check for conflict in the T -> S mapping
        if (mapTToS.has(charT)) {
            if (mapTToS.get(charT) !== charS) {
                return false;
            }
        } else {
            mapTToS.set(charT, charS);
        }
    }

    return true;
}
```

Note 1: Terminology Used

- **Bijection (Bi-directional mapping):** This is a mathematical term for a mapping between two sets where every element in the first set pairs with exactly one element in the second set, and vice versa. There are no unpaired elements and no two-to-one pairings. Identifying that this problem requires a bijection is the key to knowing you need *two* Hash Maps instead of just one.
-

Note 2: Real-World / Interview Variations (Google, Meta, Bloomberg)

Here are the indirect ways top companies test this exact same underlying logic:

1. Variation: Word Pattern (Meta) * *The Twist:* Instead of two strings of characters like “egg” and “add”, you are given a string pattern and a string of words. For example, pattern = “abba”, words = “dog cat cat dog”. * *How to solve:* The logic is 100% identical. The only difference is data preparation. You split the words string into an array of strings (["dog", "cat", "cat", "dog"]). Then, instead of mapping character-to-character, you map character-to-word. Map a -> dog, b -> cat, etc., using the exact same Two-Map bijection logic.

2. Variation: Group Isomorphic Strings (Google) * *The Twist:* You are given an array of strings: ["aab", "xxy", "xyz", "def", "llm"]. You must group the ones that are isomorphic to each other into lists. Result: [["aab", "xxy", "llm"], ["xyz", "def"]]. * *How to solve:* You cannot compare every string to every other string (that would take too much time). Instead, you must write a function to convert every string into a **Canonical Pattern**. * “aab” -> First unique char becomes ‘0’, second becomes ‘1’. So “aab” becomes “0 0 1”. * “xxy” -> First unique char ‘x’ is ‘0’, ‘y’ is ‘1’. “xxy” becomes “0 0 1”. * You use these numeric patterns (“0 0 1”) as the keys in a Hash Map, and append the original words to an array stored at that key.

3. Variation: Structural Malware Analysis (Bloomberg - Systems/Security) * *The Twist:* In a real-world systems interview, they might ask how to detect polymorphic computer viruses that change their variable names but keep the same code structure. * *How to solve:* You parse the code into an Abstract Syntax Tree (AST) or a sequence of operational codes. You then apply isomorphic mapping. If Virus A uses registers [R1, R2, R1] and a suspicious file uses registers [R8, R9, R8], you check if the arrays are isomorphic. If they are, the suspicious file is likely a clone of the virus with swapped register names.

290. Word Pattern

Here is how a senior engineer (L5/L6) approaches the “Word Pattern” problem. To ace top-tier interviews, you must demonstrate that you understand not just how to store data, but how to enforce strict, two-way relationships between different data sets while handling edge cases (like length mismatches) immediately.

1. Problem Explanation

The Goal: You are given a **pattern** (a sequence of letters) and a string **s** (a sentence of words separated by spaces). You need to determine if there is a strict, one-to-one mapping between every letter in the **pattern** and every word in **s**.

The Rule of “Bijection” (Two-Way Street): 1. Every specific letter must always map to the exact same word. (e.g., ‘a’ -> “dog”). 2. Every specific word must always map to the exact same letter. (e.g., “dog” -> ‘a’). 3. No two letters can map to the same word. 4. No two words can map to the same letter.

Examples: * **Case 1: Valid pattern** = “abba”, **s** = “dog cat cat dog” ‘a’ maps to “dog”, ‘b’ maps to “cat”. Perfect two-way street. Result: **True**

- **Case 2: Invalid (Letter violates rule)** **pattern** = “abba”, **s** = “dog cat cat fish” ‘a’ maps to “dog”. At the end, ‘a’ tries to map to “fish”. Collision! Result: **False**
- **Case 3: Invalid (Word violates rule)** **pattern** = “abba”, **s** = “dog dog dog dog” ‘a’ maps to “dog”. Next, ‘b’ tries to map to “dog”. But “dog” is already claimed by ‘a’. Collision! Result: **False**
- **Case 4: Invalid (Length mismatch)** **pattern** = “abc”, **s** = “dog cat” Not enough words for the pattern. Result: **False**

2. Solution Explanation

The Intuition (The “Why”): A common mistake junior engineers make is using a single Hash Map (Dictionary) to track **letter -> word**. They check if ‘a’ maps to “dog”, and if ‘b’ maps to “cat”.

However, a single map only enforces a **one-way street**. Look at Case 3 (**pattern** = “abba”, **s** = “dog dog dog dog”). If we only map **letter -> word**, ‘a’ becomes “dog”, and ‘b’ becomes “dog”. The map sees nothing wrong with this! It doesn’t know that “dog” was already taken.

To enforce a **two-way street**, we have two choices: 1. Keep **two separate Hash Maps**: One for **letter -> word** and one for **word -> letter**. 2. Keep **one Hash Map** (**letter -> word**) and **one Set** (**seen_words**). Before we

map a new letter to a word, we check if the word is already sitting in our Set of claimed words.

The second approach (Map + Set) is highly preferred by senior engineers because it uses slightly less memory and is conceptually cleaner.

The Algorithm (The “How”): 1. Split the string `s` into an array/list of words. 2. Check for the most obvious edge case: If the length of the pattern array does not equal the length of the words array, return `False` immediately. 3. Iterate through the pattern and the words simultaneously. 4. For each `letter` and `word`: * *Check 1*: Have we seen this `letter` before? * If yes, does it map to the current `word`? If not, return `False`. * *Check 2*: If we haven’t seen the `letter`, is the `word` already claimed by a *different* letter (is it in our `seen_words` set)? * If yes, return `False`. * *Action*: If it passes both checks, save `letter -> word` in the Map, and add `word` to the Set. 5. If we make it to the end without failing, return `True`.

End-to-End Walkthrough with ASCII Visualizations:

Let’s trace the tricky **Case 3**: `pattern = “abba”`, `s = “dog dog dog dog”`

```
[ INITIAL STATE ]
pattern:  a  b  b  a
words:    dog dog dog dog
Index i = 0
charToWord Map: {}
seenWords Set:  {}
```

```
-----
[ STEP 1: i = 0, char = 'a', word = "dog" ]
Check: Is 'a' in charToWord? No.
Check: Is "dog" in seenWords? No.
Action: Add to Map and Set.
```

```
Map:  { 'a' -> "dog" }
Set:  { "dog" }
```

```
Visual:
  [a] b  b  a
  [dog] dog dog dog
```

```
-----
[ STEP 2: i = 1, char = 'b', word = "dog" ]
Check: Is 'b' in charToWord? No.
Check: Is "dog" in seenWords? YES!
```

COLLISION DETECTED!

We are trying to assign 'b' to "dog", but our Set tells us "dog"

has already been claimed by someone else (in this case, 'a').

Visual:

```
  a [b] b  a
    | (wants to point to "dog")
    v
  dog [dog] dog dog
    ^
    |
    a (already claimed "dog!")
```

Action: Return False immediately. We don't even need to finish the string.

3. Time and Space Complexity Analysis

Time Complexity: $O(N)$ where N is the number of characters in the string s .

Space Complexity: $O(M)$ where M is the number of unique words/characters.

TIME COMPLEXITY DERIVATION ($O(N)$):

Step 1: Splitting the string "dog cat cat dog" by spaces.

This requires the computer to read the entire string char by char.

Time = $O(N)$ where N is total string length.

Step 2: Iterating through the words array (length W).

Checking a Map is $O(1)$. Checking a Set is $O(1)$.

Time = $O(W)$ loops * $O(1)$ checks = $O(W)$

Total Time = $O(N) + O(W)$.

Since W (number of words) is always smaller than N (number of characters), the dominant factor is N .

Final Time Complexity = $O(N)$.

[Timeline of execution]

[Splitting string: $O(N)$] -----> [Iterating words: $O(W)$]

SPACE COMPLEXITY DERIVATION ($O(M)$):

We create three things in memory:

1. An array of words from the split string.
2. A Hash Map storing up to 26 key-value pairs (English alphabet).
3. A Set storing up to 26 unique words.

[Memory Footprint]

Words Array: | dog | cat | cat | dog | -> $O(N)$ space

Hash Map:	a:dog b:cat	-> O(26) -> O(1) space
Set:	dog cat	-> O(26) -> O(1) space

Because we split the string into an array, the space scales with the number of characters/words. Space = O(N) or O(M).

4. Solution Code

Python Snippets

```
# Approach 1: One Map + One Set (The Cleanest L5 Standard)
# Time: O(N), Space: O(N)
def wordPattern_MapSet(pattern: str, s: str) -> bool:
    words = s.split(" ")

    # Edge case: If lengths don't match, bijection is impossible
    if len(pattern) != len(words):
        return False

    char_to_word = {}
    seen_words = set()

    # Zip allows us to iterate through both lists simultaneously
    for char, word in zip(pattern, words):
        # Case 1: We have mapped this character before
        if char in char_to_word:
            # Does it match the word we previously assigned to it?
            if char_to_word[char] != word:
                return False

        # Case 2: New character, but word is already claimed by another character
        elif word in seen_words:
            return False

        # Case 3: Completely new character and new word. Create the mapping.
        else:
            char_to_word[char] = word
            seen_words.add(word)

    return True

# Approach 2: Tracking via Indices (The "Clever" single-map trick)
# Note: Useful to know, but Approach 1 is much preferred for readability in an interview.
def wordPattern_Indices(pattern: str, s: str) -> bool:
    words = s.split(" ")
```

```

if len(pattern) != len(words):
    return False

# We use map.setdefault to map both the 'char' and the 'word' to the first index
# they appear at. If their "first seen" indices diverge, they aren't a true pair.
# We differentiate char and word by adding a prefix so 'a' and "a" don't collide.
index_map = {}

for i in range(len(words)):
    char_key = f"char_{pattern[i]}"
    word_key = f"word_{words[i]}"

    # If the first index we saw 'char' is different from the first index we saw 'word'
    if index_map.setdefault(char_key, i) != index_map.setdefault(word_key, i):
        return False

return True

```

JavaScript Snippets

```

// Approach 1: One Map + One Set (The Cleanest L5 Standard)
// Time: O(N), Space: O(N)
function wordPattern_MapSet(pattern, s) {
    const words = s.split(" ");

    // Edge case: If lengths don't match, bijection is impossible
    if (pattern.length !== words.length) {
        return false;
    }

    const charToWord = new Map();
    const seenWords = new Set();

    for (let i = 0; i < pattern.length; i++) {
        const char = pattern[i];
        const word = words[i];

        // Case 1: We have mapped this character before
        if (charToWord.has(char)) {
            // Does it match the word we previously assigned to it?
            if (charToWord.get(char) !== word) {
                return false;
            }
        }

        // Case 2: New character, but the word is already claimed
        else if (seenWords.has(word)) {

```

```

        return false;
    }
    // Case 3: Completely new character and new word. Create the mapping.
    else {
        charToWord.set(char, word);
        seenWords.add(word);
    }
}

return true;
}

// Approach 2: Two Maps (Alternative standard approach, explicit two-way street)
// Time: O(N), Space: O(N)
function wordPattern_TwoMaps(pattern, s) {
    const words = s.split(" ");
    if (pattern.length !== words.length) return false;

    const charToWord = new Map();
    const wordToChar = new Map();

    for (let i = 0; i < pattern.length; i++) {
        const char = pattern[i];
        const word = words[i];

        // Check forward mapping
        if (charToWord.has(char) && charToWord.get(char) !== word) return false;
        // Check backward mapping
        if (wordToChar.has(word) && wordToChar.get(word) !== char) return false;

        // Establish two-way mapping
        charToWord.set(char, word);
        wordToChar.set(word, char);
    }

    return true;
}

```

Note 1: Terminology Addendum

Bijection (Bijective Mapping): * **What it is:** A mathematical term for a strict “two-way street” relationship between two sets of data. Every item in Set A is paired with exactly one item in Set B, and every item in Set B is paired with exactly one item in Set A. No left-overs, no double-dipping. * **Why it**

helps: In programming, ensuring a bijection protects against data collisions where multiple identifiers accidentally map to the same resource. * **How it applies here:** The problem explicitly demands a bijection between characters and strings. The only way to code a true bijection is to track both sides of the relationship (using two maps, or a map and a set).

Hash Map / Dictionary: * **What it is:** A data structure that stores data in Key-Value pairs, allowing for ultra-fast $O(1)$ lookups.

Note 2: Real-World Interview Variations

Because “Word Pattern” is relatively straightforward, top companies rarely ask it exactly as written. They adapt the *concept of bijection* into harder problems.

Variation 1: “Isomorphic Strings” (LeetCode 205) - Very common at Google/Bloomberg * *Prompt:* Given two strings `s` and `t`, determine if they are isomorphic. (e.g., `egg` and `add` are isomorphic. `foo` and `bar` are not). * *How to solve:* This is literally the exact same problem as Word Pattern, but instead of comparing “Characters” to “Words”, you are comparing “Characters” to “Characters”. You can copy-paste the logic of our Map + Set solution above, just removing the `s.split(" ")` step, and it will perfectly solve Isomorphic Strings.

Variation 2: “Word Pattern II” (LeetCode 291) - Meta staple * *Prompt:* Given a `pattern` and a string `s`, find if `s` matches the pattern. **BUT, there are no spaces in `s`.** (e.g., `pattern` = “abab”, `s` = “redblueredblue”). * *How to solve:* This drastically changes the difficulty from Easy to Hard. Because there are no spaces, you don’t know where one word ends and another begins. An L5 engineer solves this using **Backtracking (Recursion)** combined with the Map/Set logic. You try taking 1 character for ‘a’, then 1 character for ‘b’. If that fails later, you backtrack, and try taking 2 characters for ‘a’, and so on, keeping the Map/Set updated as you explore different substring combinations.

Variation 3: “Distributed System Log Correlation” (System Design Context) * *Prompt:* You have an old legacy database that uses Integer IDs, and a new database that uses UUID strings. You are migrating logs. How do you verify that the old logs perfectly translate to the new logs without merging identities? * *How to solve:* This is the real-world application of bijection. You would stream the logs and maintain a distributed cache (like Redis) acting as your Two Maps. If `Legacy_ID_1` maps to `New_UUID_A`, but later `Legacy_ID_2` also tries to map to `New_UUID_A`, your script flags a data corruption error. The core logical constraint is identical to this LeetCode problem.

30. Substring with Concatenation of All Words

Here is how an L5/L6 Senior Software Engineer would break down and explain this problem. At this level, the focus is on identifying the specific mathematical constraints of the problem (in this case, “all words have the same length”) and exploiting them to turn a brutally slow algorithm into a highly optimized one.

1. Problem Explanation

The Premise: You are given a long string `s` and a list of `words`. You need to find the starting index of every substring in `s` that is a perfect, continuous concatenation of ALL the strings in `words`, in any order.

The Crucial Constraint (The “Aha!” Moment): The problem explicitly states: “*All the strings of words are of the **same length**.*” This is the single most important sentence in the prompt. If the words were of different lengths, this would be an incredibly complex dynamic programming or trie backtracking problem. Because they are the same length, we can treat these words as “giant characters” and solve this exactly like a standard Sliding Window problem.

Example: `s = "barfoothefoobarman"`, `words = ["foo", "bar"]` Length of each word (`L`) = 3. Number of words (`M`) = 2. Total length of our target substring = $M * L = 6$.

We are looking for substrings of length 6 that contain exactly one “foo” and exactly one “bar”. - “barfoo” (Starts at index 0) -> Valid! - “foobar” (Starts at index 9) -> Valid!

2. Solution Explanation

I will explain two approaches. It is a massive red flag in an interview to immediately jump to the optimal solution without acknowledging the brute force, as it looks like you memorized the answer.

Approach 1: The Brute Force (Chunking) The most intuitive way is to start at *every single index* `i` in the string `s`. From that index, we extract a substring of length $M * L$. We then chop that substring into `M` pieces of length `L` and check if those pieces match our `words` array. *Why it’s sub-optimal:* If we start at index 0, we check “bar”, “foo”. Then we move to index 1 and check “arf”, “oot”. Then index 2 “rfo”, “oth”. We are doing massive amounts of redundant string slicing and hash map comparisons, taking $O(N * M * L)$ time.

Approach 2: The Multi-Pass Sliding Window (The L5/L6 Way) Instead of shifting our starting point by 1 character every time, what if we shifted by `L` characters? Since every word is length `L`, we can split the string `s` into clean blocks of length `L`.

But wait! What if the valid answer starts at index 1, or index 2? To cover all possible starting offsets, we only need to perform L independent sliding window passes. - Pass 0: Starts at index 0, jumps by L. - Pass 1: Starts at index 1, jumps by L. ... - Pass L-1: Starts at index L-1, jumps by L.

Inside each pass, we maintain a sliding window of words. We keep track of the count of words currently in our window. - If we see a word that belongs in **words**, we add it to our window count. - If our window gets too large (contains more than M words), we drop the leftmost word. - If we see a word that is completely invalid (not in **words** at all), we clear our window entirely and start fresh from the next word.

Detailed ASCII Walkthrough Let's trace `s = "barfoothefoobarman"`, `words = ["foo", "bar"]`. `L = 3` (word length), `M = 2` (number of words). Target Map: { "foo": 1, "bar": 1 }

Because `L=3`, we will do exactly 3 independent passes.

=====

PASS 0 (Starts at index 0)

String chunked by `L=3`:

[bar] [foo] [the] [foo] [bar] [man]

Step 1:

Window sees "bar".

Window Map: {"bar": 1}. Matches found = 1.

Step 2:

Window sees "foo".

Window Map: {"bar": 1, "foo": 1}. Matches found = 2.

Matches (2) == M (2). VALID! Record index 0.

Step 3:

Window sees "the". "the" is not in our target words!

The chain is broken. We must clear the window completely.

Window Map: {}. Matches found = 0.

Step 4:

Window sees "foo".

Window Map: {"foo": 1}. Matches found = 1.

Step 5:

Window sees "bar".

Window Map: {"foo": 1, "bar": 1}. Matches found = 2.

Matches (2) == M (2). VALID!

Calculation for starting index: Current end position minus total length of words.

Record index 9.

```

=====
PASS 1 (Starts at index 1)
String chunked by L=3:
b [arf] [oot] [hef] [oob] [arm] an

Step 1:
"arf" is invalid. Clear window.
Step 2:
"oot" is invalid. Clear window.
(None of these chunks match "foo" or "bar". Pass 1 yields nothing.)

=====
PASS 2 (Starts at index 2)
String chunked by L=3:
ba [rfo] [oth] [efo] [oba] [rma] n

(Again, none of these chunks form valid words. Pass 2 yields nothing.)

=====
Final Result: [0, 9]

```

3. Time and Space Complexity Analysis

Time Complexity: $O(N * L)$ Where N is the length of string s , and L is the length of each word.

Visual Derivation of Time Complexity:

```

We do L passes.
Pass 0:  [---] [---] [---] [---] ... (N/L words processed)
Pass 1:  . [---] [---] [---] [---] ... (N/L words processed)
...
Pass L:  ... [---] [---] [---] ... (N/L words processed)

```

Total words processed across ALL passes = $L * (N / L) = N$ words.

For each word processed, we must extract a substring of length L to check it. String slicing/extraction takes $O(L)$ time.

```

Total Time = (Total words processed) * (Time to slice one word)
Total Time =  $N * O(L)$ 
Total Time =  $O(N * L)$ 

```

Space Complexity: $O(M * L)$ Where M is the number of words, and L is the

length of each word.

Visual Derivation of Space Complexity:

Memory Map 1: Target Word Frequencies

```
{
  "foo": 1,
  "bar": 1
}
```

Max size is M keys. Each key is a string of length L. Space = $O(M * L)$

Memory Map 2: Current Window Frequencies

```
{
  "foo": 1,
  "bar": 1
}
```

Max size is M keys. Each key is a string of length L. Space = $O(M * L)$

We also allocate an array to return results, which in the worst case (e.g. `s="aaaa"`, `words=["a"]`) could be $O(N)$. But typically auxiliary space is governed by the maps.

Total Space = $O(M * L)$

4. Solution Code

Here are both the sub-optimal brute-force (highly readable) and the optimal multi-pass sliding window implementations.

Python Solutions

```
# --- SOLUTION 1: The Brute Force (Chunking) -  $O(N * M * L)$  Time ---
# Excellent to code first in an interview if you have 45 minutes, then optimize.
from collections import Counter

def findSubstring_brute(s: str, words: list[str]) -> list[int]:
    if not s or not words:
        return []

    L = len(words[0])
    M = len(words)
    total_len = L * M
    target_counts = Counter(words)
    res = []

    # Iterate through every single character index up to the last possible starting point
```

```

for i in range(len(s) - total_len + 1):
    # Extract the giant substring
    sub = s[i : i + total_len]

    # Break it into L-sized chunks and count them
    seen = {}
    is_valid = True

    for j in range(0, total_len, L):
        word = sub[j : j + L]
        if word not in target_counts:
            is_valid = False
            break

        seen[word] = seen.get(word, 0) + 1
        if seen[word] > target_counts[word]:
            is_valid = False
            break

    if is_valid:
        res.append(i)

return res

# --- SOLUTION 2: Optimized Multi-Pass Sliding Window - O(N * L) Time ---
# The top-tier expected solution.
def findSubstring_optimal(s: str, words: list[str]) -> list[int]:
    if not s or not words:
        return []

    L = len(words[0])
    M = len(words)
    target_counts = Counter(words)
    res = []

    # We do exactly L passes to cover all possible character offsets
    for offset in range(L):
        left = offset
        matches = 0
        window_counts = {}

        # Jump by L characters at a time
        for right in range(offset, len(s) - L + 1, L):
            word = s[right : right + L]

```

```

if word in target_counts:
    window_counts[word] = window_counts.get(word, 0) + 1
    matches += 1

    # If we have too many of a specific word, shrink window from the left
    while window_counts[word] > target_counts[word]:
        left_word = s[left : left + L]
        window_counts[left_word] -= 1
        matches -= 1
        left += L # Shrink window

    # If we have exactly M valid words in our window, we found an answer!
    if matches == M:
        res.append(left)

else:
    # The word is completely invalid. Chain is broken. Reset everything.
    window_counts.clear()
    matches = 0
    left = right + L # Move the left pointer past this invalid word

return res

```

JavaScript Solutions

```

// --- SOLUTION 1: The Brute Force (Chunking) - O(N * M * L) Time ---
var findSubstringBrute = function(s, words) {
    if (!s || words.length === 0) return [];

    let L = words[0].length;
    let M = words.length;
    let totalLen = L * M;
    let res = [];

    let targetCounts = {};
    for (let word of words) {
        targetCounts[word] = (targetCounts[word] || 0) + 1;
    }

    for (let i = 0; i <= s.length - totalLen; i++) {
        let seen = {};
        let isValid = true;

        for (let j = 0; j < M; j++) {
            let wordIndex = i + j * L;
            let word = s.slice(wordIndex, wordIndex + L);

```

```

        if (!targetCounts[word]) {
            isValid = false;
            break;
        }

        seen[word] = (seen[word] || 0) + 1;
        if (seen[word] > targetCounts[word]) {
            isValid = false;
            break;
        }
    }

    if (isValid) res.push(i);
}

return res;
};

// --- SOLUTION 2: Optimized Multi-Pass Sliding Window - O(N * L) Time ---
var findSubstringOptimal = function(s, words) {
    if (!s || words.length === 0) return [];

    let L = words[0].length;
    let M = words.length;
    let targetCounts = {};
    for (let word of words) {
        targetCounts[word] = (targetCounts[word] || 0) + 1;
    }

    let res = [];

    // We do exactly L independent passes
    for (let offset = 0; offset < L; offset++) {
        let left = offset;
        let matches = 0;
        let windowCounts = {};

        // Jump by L characters at a time
        for (let right = offset; right <= s.length - L; right += L) {
            let word = s.slice(right, right + L);

            if (targetCounts[word]) {
                windowCounts[word] = (windowCounts[word] || 0) + 1;
                matches++;
            }
        }
    }

```

```

        // Shrink from left if we have an excess of a specific word
        while (windowCounts[word] > targetCounts[word]) {
            let leftWord = s.slice(left, left + L);
            windowCounts[leftWord]--;
            matches--;
            left += L;
        }

        // If the window has exactly M valid words
        if (matches === M) {
            res.push(left);
        }
    } else {
        // Invalid word encountered. Reset the entire sliding window.
        windowCounts = {};
        matches = 0;
        left = right + L;
    }
}

return res;
};

```

Note 1: Terms and Techniques Used

- **Sliding Window:** An algorithmic paradigm where you maintain a subset of data (a “window”) over an array or string. As you iterate, the window expands on the right and contracts on the left. It prevents repetitive calculation of overlapping data.
 - **Multi-Pass Sliding Window:** A specialized variation of the Sliding Window. Used when the problem requires chunking the array into specific block sizes (like length L). Instead of one window moving by 1 step, you run L different windows, each moving by L steps, starting at offsets 0, 1, 2... up to $L-1$.
 - **Frequency Map (Hash Map):** A dictionary object used to count the occurrences of items. In Python, `collections.Counter` handles this automatically. In this problem, it’s used to compare “what words we need” vs “what words we currently have in our window”.
-

Note 2: Real-World & Indirect Interview Variations

Top-tier companies rarely ask for generic “words” in a “string”. They frame the exact same algorithm around system design or domain-specific concepts.

1. Meta (Facebook): “Analyzing Server Event Logs” * *The Setup:* You are given an endless stream of text representing server logs. Every event is mapped to a highly specific, standardized 4-character code (e.g., “ERR1”, “SUC2”, “WARN”). An alert needs to trigger if a specific sequence of 5 events occurs consecutively within the log stream, in any order. * *How it maps:* The log stream is `s`. The 4-character constraint is $L = 4$. The 5 events are the `words` array. * *How an L5 solves it:* Implement the Optimal Multi-Pass Sliding Window. Because L is 4, you run 4 concurrent window offsets to monitor the incoming text stream efficiently in real-time without buffering excessive amounts of memory.

2. Google: “Network Packet Reassembly” * *The Setup:* A network node receives fragments of data. Each fragment has an 8-byte header string. You are looking for a complete payload that is made up of a specific set of 10 headers that arrive together, though the network might scramble their exact arrival order. Given the massive incoming string of bytes, find the start index of valid payloads. * *How it maps:* $L = 8$ (header size). The set of 10 headers is `words`. * *How an L5 solves it:* Again, exact same problem. The discussion here would pivot slightly to Time Complexity. An L5 would point out that since L is fixed at 8, we do exactly 8 passes over the string chunking by 8, resulting in incredibly fast $O(N * 8) \rightarrow O(N)$ linear time processing.

3. Bloomberg: “Financial Ticker Sub-sequence Detection” * *The Setup:* You are processing a raw tick-by-tick tape of stock exchanges, represented as a massive string of 3-letter tickers (e.g., “APLGOOAMZMSF...”). An algorithmic trading bot wants to fire when a specific “basket” of 8 specific stocks all trade sequentially, regardless of their order. * *How it maps:* $L = 3$ (ticker length). The basket is `words`. The string is the tape. * *How an L5 solves it:* Same algorithm. However, an L5 engineer at Bloomberg would likely mention that allocating strings in memory inside the inner loop (`s.slice(right, right+L)`) might trigger Garbage Collection pauses. They would suggest optimizing further by using a rolling hash or comparing characters by integer indices directly instead of allocating new strings for every chunk, minimizing memory overhead.

68. Text Justification

As an engineer sitting across the table from you in an interview, I’ll be completely candid: LeetCode 68 is rarely about testing a brilliant algorithmic “aha!” moment. It is a pure string manipulation and simulation problem.

When a top-tier company asks this, they are testing your **engineering rigor**, your ability to translate tedious business logic into clean code, and your disci-

pline in handling edge cases without making a mess of variables.

Let's break down exactly how a senior engineer approaches, visualizes, and codes this.

1. Problem Explanation

The Goal: We are given an array of words and a maximum line width (`maxWidth`). We need to format the text such that every line is exactly `maxWidth` characters long, and the text is fully justified (flush left and flush right).

The Rules (The Business Logic):

1. **Greedy Packing:** Pack as many words as possible into a single line. You must leave at least one space between words.
2. **Even Space Distribution:** If a line has extra spaces left over, distribute them as evenly as possible between words.
3. **Left-Heavy Spaces:** If the spaces cannot be distributed evenly, the empty slots on the left get more spaces than the slots on the right.
4. **Single Word Edge Case:** If a line only contains one word, left-justify it and pad the right with spaces.
5. **Last Line Edge Case:** The very last line of the text must be left-justified, with exactly one space between words, and padded to the right with spaces.

Visualizing the Goal: Let `words = ["This", "is", "an", "example", "of", "text", "justification."]` and `maxWidth = 16`.

Target Output:

```
[
  "This    is    an",
  "example  of text",
  "justification. "
]
```

Let's look at the anatomical breakdown of line 2: "example of text"

`MaxWidth = 16`

Lengths: "example"(7) + "of"(2) + "text"(4) = 13 characters of actual text.

Spaces needed: $16 - 13 = 3$ spaces.

Number of gaps between 3 words: 2 gaps (between example/of, and of/text).

Distribution: $3 \text{ spaces} / 2 \text{ gaps} = 1 \text{ space each}$, with 1 leftover space.

The leftover space goes to the leftmost gap.

Result: "example" + (2 spaces) + "of" + (1 space) + "text"

2. Solution Explanation

A senior engineer avoids writing one massive loop that tries to do everything at once. This problem requires a two-phase approach inside our main loop:

Phase 1: Identification (gathering words for the current line) and **Phase 2: Formatting** (applying the specific justification rules to those gathered words).

Phase 1: Identification (Greedy Packing) We keep a running list of words for the current line and a running count of their characters. Before adding a new word, we check: *If we add this word, plus the mandatory minimum spaces between the current words, will it exceed `maxWidth`?* If yes, we stop gathering and move to Phase 2. If no, we add the word to our current line pool.

Phase 2: Formatting (The Math) Once we have our list of words for the line, we format it based on the three scenarios:

- **Scenario A: It is the last line OR it only has 1 word.** Join the words with a single space. Calculate how much padding is needed to reach `maxWidth`, and add those spaces to the right end.
- **Scenario B: It is a normal, fully justified line.** We calculate the total spaces we need to insert. We divide that by the number of “gaps” between words to find the `base_spaces` every gap gets. We use the modulo operator (remainder) to find how many `extra_spaces` we have. We loop through the gaps, giving everyone the `base_spaces`, and handing out 1 `extra_space` to the leftmost gaps until we run out.

ASCII Diagram Walkthrough Let's trace `words = ["What", "must", "be", "acknowledgment", "shall"]`, and `maxWidth = 16`.

Step 1: Packing Line 1

Trying to pack words...

1. Add "What" (len 4). Total len so far + min spaces = 4. (Under 16)
2. Add "must" (len 4). Total len = "What" (4) + 1 space + "must" (4) = 9. (Under 16)
3. Add "be" (len 2). Total len = 9 + 1 space + "be" (2) = 12. (Under 16)
4. Add "acknowledgment" (len 14). Total len = 12 + 1 space + 14 = 27. (OVER 16! STOP)

Current Line Words: ["What", "must", "be"]

Step 2: Formatting Line 1 (Normal Justification)

Words: ["What", "must", "be"]

Letter Count: 4 + 4 + 2 = 10

Total Spaces Needed: 16 (`maxWidth`) - 10 (Letter Count) = 6 spaces.

Gaps between words: 3 words - 1 = 2 gaps.

Math:

Base spaces per gap = 6 / 2 = 3

Extra spaces (remainder) = 6 % 2 = 0

Assembly:

[What] + (3 spaces) + [must] + (3 spaces) + [be]

Result: "What must be"

Step 3: Packing Line 2

Starting fresh...

1. Add "acknowledgment" (len 14). Total len = 14. (Under 16)
2. Add "shall" (len 5). Total len = 14 + 1 space + 5 = 20. (OVER 16! STOP)

Current Line Words: ["acknowledgment"]

Step 4: Formatting Line 2 (Single Word Edge Case)

Words: ["acknowledgment"]

Gaps: 0. (Can't divide by zero!)

Rule applied: Single word rule.

Join with spaces (none), pad right.

```
Result: "acknowledgment  " (padded with 2 spaces to reach 16)
```

Step 5: Packing and Formatting Line 3 (Last Line Edge Case)

```
Words left: ["shall", "be"]
```

Since we reached the end of the input array, this is the LAST LINE.

Rule applied: Last line rule.

Join with single spaces, pad right.

"shall" + (1 space) + "be" = "shall be" (length 8)

Pad to 16: "shall be"

3. Time and Space Complexity Analysis

```

+-----+
| TIME COMPLEXITY DERIVATION:  $O(N)$  |
+-----+
| N = Total number of characters across all words in the input array. |
| |
| 1. Identification Phase: |
|   We iterate through the array of words exactly once. |
|   Cost:  $O(W)$  where  $W$  is the number of words. |
| |
| 2. Formatting Phase: |
|   For each line, we construct a new string of length `maxWidth`. |
|   Every word is placed into a string exactly once. |
|   The number of spaces added is bound by `maxWidth` * number of lines |
|   which roughly scales linearly with the total characters  $N$ . |
| |
| TOTAL TIME: We touch each word/character a constant number of times. |
|   =>  $O(N)$  Time Complexity |
+-----+

```

SPACE COMPLEXITY DERIVATION: $O(N)$ ----- 1. Result Array: We must store the final formatted strings to return them. The total length of all strings in the result array will equal the number of lines * `maxWidth`. 2. Working Line Buffer: We temporarily store words for the current line in an array. This takes at most $O(\text{maxWidth})$ space at any given time. TOTAL SPACE: Bounded by the size of the output required. => $O(N)$ Space Complexity -----
--

4. Solution Code

Here is the implementation. Notice how the logic is cleanly separated between gathering words and formatting the line. This prevents the dreaded “spaghetti code” that ruins many interviews.

Python Snippet

```
def fullJustify(words: list[str], maxWidth: int) -> list[str]:
    res = []
    current_line_words = []
    current_line_letters_count = 0

    for word in words:
        # Check if adding the next word exceeds maxWidth
        # length of current words + length of new word + mandatory 1 space per existing word
        if current_line_letters_count + len(word) + len(current_line_words) > maxWidth:

            # FORMATTING PHASE
            # We have gathered all words for this line. Time to format it.
            gaps = len(current_line_words) - 1

            # Scenario A: Single word line
            if gaps == 0:
                res.append(current_line_words[0].ljust(maxWidth))

            # Scenario B: Normal line
            else:
                total_spaces_needed = maxWidth - current_line_letters_count
                base_spaces = total_spaces_needed // gaps
```

```

        extra_spaces = total_spaces_needed % gaps

        line = ""
        for i in range(len(current_line_words) - 1):
            line += current_line_words[i]
            line += " " * base_spaces
            # Distribute leftover spaces to the leftmost gaps
            if extra_spaces > 0:
                line += " "
                extra_spaces -= 1

        # Add the final word (which doesn't have spaces after it)
        line += current_line_words[-1]
        res.append(line)

        # Reset the line tracking variables for the next line
        current_line_words = []
        current_line_letters_count = 0

        # IDENTIFICATION PHASE
        # Add the current word to our running pool
        current_line_words.append(word)
        current_line_letters_count += len(word)

        # SCENARIO C: The Last Line
        # Join with single spaces, then pad the right side with remaining spaces
        last_line = " ".join(current_line_words)
        res.append(last_line.ljust(maxWidth))

    return res

```

JavaScript Snippet

```

function fullJustify(words, maxWidth) {
    const res = [];
    let currentLineWords = [];
    let currentLineLettersCount = 0;

    for (let word of words) {
        // Check if adding the current word exceeds the max width
        // Required length = letters so far + length of new word + minimum 1 space per word
        if (currentLineLettersCount + word.length + currentLineWords.length > maxWidth) {

            // FORMATTING PHASE
            let gaps = currentLineWords.length - 1;

            // Scenario A: Single word line

```

```

    if (gaps === 0) {
        // Pad to the right until maxWidth
        res.push(currentLineWords[0].padEnd(maxWidth, ' '));
    }
    // Scenario B: Normal justified line
    else {
        let totalSpacesNeeded = maxWidth - currentLineLettersCount;
        let baseSpaces = Math.floor(totalSpacesNeeded / gaps);
        let extraSpaces = totalSpacesNeeded % gaps;

        let line = "";
        for (let i = 0; i < currentLineWords.length - 1; i++) {
            line += currentLineWords[i];

            // Add base spaces
            line += " ".repeat(baseSpaces);

            // Add 1 extra space if there are leftovers (left-heavy rule)
            if (extraSpaces > 0) {
                line += " ";
                extraSpaces--;
            }
        }
        // Add the final word to the line
        line += currentLineWords[currentLineWords.length - 1];
        res.push(line);
    }

    // Reset trackers for the next line
    currentLineWords = [];
    currentLineLettersCount = 0;
}

// IDENTIFICATION PHASE
// Add the current word
currentLineWords.push(word);
currentLineLettersCount += word.length;
}

// SCENARIO C: Last line edge case
// Left justified, 1 space between words, padded to the right
let lastLine = currentLineWords.join(" ");
res.push(lastLine.padEnd(maxWidth, ' '));

return res;
}

```

Note 1: Terminology and Techniques

- **Greedy Algorithm / Simulation:** We call this a “Greedy” approach because at every step, we pack as many words as we possibly can into the current line without looking ahead to see if it makes future lines look better or worse. We make the locally optimal choice immediately. It is a “Simulation” because the algorithm doesn’t rely on abstract mathematical properties; it literally simulates the real-world process of writing words left-to-right on a constrained piece of paper.
-

Note 2: Real-World Interview Variations

Because pure string simulation can be tedious, top companies often disguise this problem within practical, domain-specific scenarios.

1. Meta: “Newsfeed Post Truncator” * *Prompt:* You are given a user’s post (a long string) and a `maxDisplayLines` limit, along with a `maxWidth` for the mobile screen. Format the string to fit the screen width, but if the post exceeds `maxDisplayLines`, truncate the final line and append “... See More”. * *Solution Details:* You use the exact same grouping logic to measure word lengths and line capacity. However, you maintain a counter for `current_line_number`. When `current_line_number == maxDisplayLines`, instead of formatting the line normally, you greedily add words to that final line until adding the string “... See More” would exceed `maxWidth`, at which point you stop and return the result.

2. Google: “Terminal Command Output Fitter” * *Prompt:* Write a utility for a CLI tool that takes an array of file names and prints them in a grid (like the `ls` command). The output must fit within the terminal’s `columnWidth`. File names should be left-aligned in columns, and the columns must be spaced evenly. * *Solution Details:* Instead of justifying a single string horizontally to a specific width, you are justifying an array into a grid. You first find the length of the longest word to determine the fixed width of your columns. Then, you calculate how many columns can fit into `columnWidth`. Finally, you iterate through the words, padding each word to the max column width using `.padEnd()`, and appending a newline character whenever you reach the calculated column limit.

3. Bloomberg: “Financial Ticker Tape Formatter” * *Prompt:* You are given a stream of stock symbols and prices: `["AAPL:150", "TSLA:900", "MSFT:300"]`. You have a digital billboard that holds exactly `C` characters per frame. You need to group these tickers so they fill the billboard frame as completely as possible, distributing any extra space evenly between the tickers to make it readable. * *Solution Details:* This is a direct 1-to-1 map to LeetCode 68. The “words” are the “TICKER:PRICE” strings. The `maxWidth` is `C`. You

follow the exact same Phase 1 (measuring if the next ticker fits) and Phase 2 (dividing total spaces by gaps and distributing them).

1147. Longest Chunked Palindrome Decomposition

As a senior engineer, when I look at “Longest Chunked Palindrome Decomposition” (LeetCode 1147), I see a problem that tricks many candidates into overcomplicating things. Because it asks for the “longest” decomposition (an optimization problem), the immediate instinct is to use Dynamic Programming (DP).

However, an L5/L6 engineer will pause, observe the properties of the string, and realize this is actually a **Greedy Algorithm** problem in disguise.

Here is the complete end-to-end walkthrough of how to break this down logically and intuitively.

1. Problem Explanation

The Core Question: You are given a string. You need to split it into the maximum possible number of “chunks” (substrings) so that the chunks mirror each other from the outside in, exactly like a palindrome.

Standard Palindrome vs Chunked Palindrome: * Standard (mirrors letters): r a c e c a r * Chunked (mirrors blocks): [vol] [vo] [vol]

Visualizing the Goal: We want to maximize k (the number of chunks).

Case A: A perfect chunked mirror Input: text = "ghiabcdefhelloadamhelloabcdefghi"

We want to peel off matching layers from the outside:

```
Layer 1: [ghi] ... [ghi]
Layer 2:      [abcdef] ... [abcdef]
Layer 3:           [hello] ... [hello]
Center:                [adam]
```

Chunks: [ghi] [abcdef] [hello] [adam] [hello] [abcdef] [ghi]
Total Chunks (k) = 7

Case B: No matching chunks at all Input: text = "merchant"

Prefixes and Suffixes:

```
'm' != 't'
'me' != 'nt'
'mer' != 'ant'
...
```

Since no outer chunks match, the entire word is the only valid chunk.

Chunks: [merchant]

Total Chunks (k) = 1

2. Solution Explanation

To maximize the number of chunks, we must use a **Greedy Approach**.

The “Aha!” Moment (Why Greedy Works): When iterating from the outside in, we should always make a cut the *very first moment* we find a matching prefix and suffix. Why? Because picking the **shortest** matching outer chunks leaves the **longest** possible string in the middle. A longer middle string gives us more opportunities to make further cuts.

Let's prove this with an example. Input: `text = "abaaba"`

Bad Approach (Taking a longer match):

Let's say we wait and match a longer chunk:

[aba] [aba] -> Total Chunks = 2.

We missed out on a better split!

Greedy Approach (Taking the shortest match immediately):

Step 1: Check length 1 chunks.

Left: 'a' | Right: 'a'. They match! Cut immediately.

[a] [baab] [a]

Step 2: Look at the remaining middle -> "baab"

Check length 1 chunks.

Left: 'b' | Right: 'b'. They match! Cut immediately.

[a] [b] [aa] [b] [a]

Step 3: Look at the remaining middle -> "aa"

Check length 1 chunks.

Left: 'a' | Right: 'a'. They match! Cut.

[a] [b] [a] [a] [b] [a] -> Total Chunks = 6.

By being greedy and taking the shortest match, we achieved **k=6** instead of **k=2**.

The “How” (Two Pointers & Slicing): 1. We will use two pointers: `l` (left start) and `r` (right end), initially spanning the whole string. 2. We will check increasing chunk **length** starting from 1 up to half the remaining string. 3. If `left_chunk == right_chunk`, we add 2 to our total, shrink our window (`l` moves right, `r` moves left), and restart our search for the new middle. 4. If we check all possible lengths and find no matches, the entire remaining middle becomes 1 final center chunk.

3. Time and Space Complexity Analysis

Let N be the length of the string `text`.

```
=====
TIME COMPLEXITY:  $O(N^2)$  in the Worst Case
=====
```

Derivation Diagram:

Consider a string with no matches until the very end: "abcdefgh"

```
[ Length = 1 ] compares 'a' to 'h'          (1 operation)
[ Length = 2 ] compares 'ab' to 'gh'         (2 operations)
[ Length = 3 ] compares 'abc' to 'fgh'       (3 operations)
[ Length = 4 ] compares 'abcd' to 'efgh'     (4 operations)
```

|

V

Total Operations = $1 + 2 + 3 + \dots + (N/2)$

Summing an arithmetic progression from 1 to N scales quadratically.

Therefore, the worst-case time complexity is $O(N^2)$.

*Note: In the best case (e.g., "aaaaaaaa"), it perfectly matches at length 1 every time, resulting in $O(N)$ time complexity.

```
=====
SPACE COMPLEXITY:  $O(1)$  Auxiliary Space (Iterative)
=====
```

Derivation Diagram:

```
[ Variables in Memory ]
|-- l (integer pointer)
|-- r (integer pointer)
|-- res (integer counter)
|-- length (integer)
```

Because we use standard pointers and only create temporary substring slices during comparison without building new massive data structures, the extra memory used does not scale with N . Space is Constant $O(1)$.

4. Solution Code

Here are two standard, interview-ready implementations. The first is the highly-readable Iterative approach ($O(1)$ space). The second is a beautifully concise

Recursive approach that is great to show off in Python.

JavaScript Snippets

```
/**
 * Approach 1: Iterative Two Pointers (Greedy)
 * Time:  $O(N^2)$  Worst Case / Space:  $O(1)$ 
 */
function longestDecomposition(text) {
    let res = 0;
    let l = 0;
    let r = text.length;

    while (l < r) {
        let matchFound = false;

        // Try all chunk lengths from 1 up to half of the current remaining window
        for (let length = 1; length <= Math.floor((r - l) / 2); length++) {

            // Extract the prefix and suffix of the current window
            let leftChunk = text.substring(l, l + length);
            let rightChunk = text.substring(r - length, r);

            // If they match, we greedily take this cut
            if (leftChunk === rightChunk) {
                res += 2;           // Add 2 chunks (one from left, one from right)
                l += length;       // Shrink window from the left
                r -= length;       // Shrink window from the right
                matchFound = true;
                break;             // Stop checking longer lengths!
            }
        }

        // If we tried all possible lengths and found no outer match,
        // the entire remaining middle section forms exactly 1 final chunk.
        if (!matchFound) {
            res += 1;
            break;
        }
    }

    return res;
}
```

Python Snippets

```

# Approach 1: Iterative Two Pointers (Greedy)
# Time:  $O(N^2)$  Worst Case / Space:  $O(1)$ 
def longestDecomposition(text: str) -> int:
    res = 0
    l, r = 0, len(text)

    while l < r:
        match_found = False

        # Check all lengths from 1 up to half the window size
        for length in range(1, (r - l) // 2 + 1):

            # Slicing the prefix and suffix
            left_chunk = text[l : l + length]
            right_chunk = text[r - length : r]

            # Greedily cut at the first match found
            if left_chunk == right_chunk:
                res += 2
                l += length
                r -= length
                match_found = True
                break

        # If no outer matches exist, the remainder is the final center piece
        if not match_found:
            res += 1
            break

    return res

# -----
# Approach 2: Recursive (Elegant Pythonic Alternative)
# Time:  $O(N^2)$  Worst Case / Space:  $O(N)$  due to call stack
# -----
def longestDecomposition_recursive(text: str) -> int:
    # Base case: empty string has 0 chunks
    if not text:
        return 0

    # Look for the shortest matching prefix and suffix
    for length in range(1, len(text) // 2 + 1):
        if text[:length] == text[-length:]:
            # Found a match! Cut the ends, add 2, and recurse on the middle
            return 2 + longestDecomposition_recursive(text[length:-length])

```

```
# If no match is found, the whole string is 1 chunk
return 1
```

Note 1: Advanced Concepts and Algorithms

- **Greedy Algorithm:** The strategy of making the locally optimal choice at each stage with the hope of finding a global optimum. In this problem, the “local optimum” is taking the shortest possible matching chunk immediately, which mathematically guarantees the maximum total chunks.
 - **Rabin-Karp (Rolling Hash):** If an interviewer pushes you to make this strictly **$O(N)$ Time in all cases**, you would mention *Rabin-Karp*. It is a string-matching algorithm that converts strings into integers (hashes). Instead of slicing strings taking $O(K)$ time, you maintain a “rolling hash” of the prefix and suffix. Comparing the hashes takes $O(1)$ time. Implementing this flawlessly on a whiteboard is very difficult, so verbally explaining that it drops the TC from $O(N^2)$ to $O(N)$ is usually exactly what an L5/L6 interviewer wants to hear.
-

Note 2: Real Interview Variations (Google, Meta, Bloomberg)

Understanding Palindrome string partitioning opens the door to heavily tested FAANG variations:

Variation 1: “Valid Palindrome” (LeetCode 125) - Meta * **The Problem:** Check if a string is a standard palindrome, ignoring spaces and casing. * **How to Solve:** Very simple Two Pointers. Left starts at 0, Right starts at $N-1$. While $left < right$, step inwards. If the character isn’t alphanumeric, skip it. If they are alphanumeric and don’t match, return false.

Variation 2: “Shortest Palindrome” (LeetCode 214) - Google Heavy-weight * **The Problem:** Given a string s , you can convert it to a palindrome by adding characters *in front of it*. Find the shortest palindrome you can make. * **How to Solve:** This is finding the longest palindromic *prefix* of the string. You reverse the string s to make rev_s . You want to find the longest prefix of s that perfectly matches a suffix of rev_s . Because you need this to be highly optimized, you use the **KMP Algorithm (Knuth-Morris-Pratt)** lookup table on the combined string $s + \# + rev_s$.

Variation 3: “Palindrome Partitioning” (LeetCode 131) - Bloomberg * **The Problem:** Given a string s , partition s such that every substring of the partition is a standard palindrome. Return *all* possible palindrome partitionings (not just the longest count). * **How to Solve:** Because it asks for *all possibilities*, this is no longer a Greedy problem. It is a **Backtracking (Depth First Search)** problem. You iterate through the string, and if the prefix $s[0:i]$ is a palindrome, you add it to your current path, and recursively DFS on the

remainder of the string `s[i:]`. You append the path to an answer array when you reach the end of the string.

438. Find All Anagrams in a String

Here is an end-to-end, top-tier (L5/L6) walkthrough for solving “Find All Anagrams in a String.”

An L5/L6 (Senior/Staff) engineer approaches this problem by immediately recognizing the inefficiency of re-evaluating overlapping data. They will focus on “state maintenance”—how to efficiently update our knowledge as we move through the data, rather than starting from scratch at every step.

1. Problem Explanation

The Core Goal: You are given a long string `s` and a shorter target string `p`. You need to find every starting position in `s` where the next characters form an anagram of `p`.

What is an Anagram? An anagram is a word formed by rearranging the letters of another word. Two words are anagrams if they have the exact same frequency of every letter. Order does not matter. * Example: “abc” and “cba” are anagrams. Both have one ‘a’, one ‘b’, and one ‘c’.

The Challenge: If `s` is “cbaebabacd” and `p` is “abc”, you are looking for all substrings in `s` of length 3 that contain exactly one ‘a’, one ‘b’, and one ‘c’. The naive approach is to look at “cba” (Match!), then “bae” (No match), then “aeb” (No match), etc., by counting the letters from scratch every single time. This is incredibly slow for massive strings.

2. Solution Explanation

To solve this optimally, we use the **Sliding Window** technique combined with **Frequency Maps**.

The “Why” - Avoiding Redundant Work: Imagine looking at the string “cbaeb” with a window size of 3 (the length of “abc”). 1. First window: [c, b, a]. We count the letters. 2. Second window: [b, a, e].

Notice what happened when we moved the window one step to the right? Most of the window stayed the exact same! The characters ‘b’ and ‘a’ are still there. The only things that changed were: 1. The character ‘c’ fell out of the left side of the window. 2. The character ‘e’ entered the right side of the window.

Instead of recounting everything in the second window from scratch, we can just take our counts from the first window, **subtract 1 from 'c'**, and **add 1 to 'e'**. This reduces a complex counting operation into two simple mathematical steps.

Let's visualize this step-by-step.

```
=====
INITIAL STATE
=====
String s: "c b a e b"
String p: "a b c"
Target Length = 3

Target Frequency (What we are looking for):
[a:1, b:1, c:1, d:0, e:0 ...]

Current Window Frequency (Empty to start):
[a:0, b:0, c:0, d:0, e:0 ...]

Result Array: []

=====
STEP 1: Build the First Window
=====
We expand our window to cover the first 3 characters of 's'.

s = [c b a] e b
    ^ ^ ^
    Window

Add 'c', 'b', and 'a' to our Window Frequency:
Current Window Frequency:
[a:1, b:1, c:1, d:0, e:0 ...]

Check: Does Current Window == Target Frequency?
YES! They match perfectly.
Action: Add the starting index (0) to our Result Array.

Result Array: [0]

=====
STEP 2: Slide the Window Right
=====
We slide the window exactly one space to the right.

s = c [b a e] b
    ^ ^ ^ ^
```

Out Window

1. The character 'c' falls OUT of the left side. We SUBTRACT it from our frequency.
2. The character 'e' comes INTO the right side. We ADD it to our frequency.

Current Window Frequency updates:

- Remove 'c': [a:1, b:1, c:0, d:0, e:0 ...]
- Add 'e': [a:1, b:1, c:0, d:0, e:1 ...]

Check: Does Current Window == Target Frequency?

NO. Target has c:1, Window has c:0, e:1.

Action: Do nothing. Move on.

Result Array: [0]

=====

STEP 3: Slide the Window Right Again

=====

Slide one space to the right.

s = c b [a e b]
 ^ ^ ^ ^

Out Window

1. The character 'b' falls OUT of the left side. SUBTRACT 'b'.
 2. The character 'b' comes INTO the right side. ADD 'b'.
- (In this specific step, 'b' is removed and a new 'b' is added, so the count of 'b' remains 1)

Current Window Frequency updates:

- Remove 'b': [a:1, b:0, c:0, d:0, e:1 ...]
- Add 'b': [a:1, b:1, c:0, d:0, e:1 ...]

Check: Does Current Window == Target Frequency?

NO.

Action: Do nothing. Reached end of string.

FINAL RESULT: [0]

By using this sliding technique, we only ever do two tiny updates per step, no matter how large the target string p is!

3. Time and Space Complexity Analysis

Let N be the length of string s. Let M be the length of string p.

TIME COMPLEXITY DERIVATION: $O(N)$

Input size: String 's' of length N.

Phase 1: Build target frequency map for 'p'.
Loops M times. ---> $O(M)$ time

Phase 2: Build the initial window.
Loops M times. ---> $O(M)$ time

Phase 3: Slide the window across the rest of 's'.
The window slides $N - M$ times.

At each slide, we do exactly:

- 1 array lookup/decrement (remove left char) ---> $O(1)$ time
- 1 array lookup/increment (add right char) ---> $O(1)$ time
- 1 comparison of two 26-element arrays ---> $O(26)$ time which is $O(1)$ constant

Total Time = $O(M) + O(M) + ((N - M) * O(1))$

Since M cannot be larger than N (otherwise we return empty),
the dominant factor is scanning string 's'.

OVERALL TIME COMPLEXITY = $O(N)$

SPACE COMPLEXITY DERIVATION: $O(1)$

What are we storing in memory, regardless of how big 's' is?

1. An array for the Target Frequency of 'p'.
Since the problem limits us to lowercase English letters,
this array is ALWAYS exactly size 26.
2. An array for the Current Window Frequency.
This array is also ALWAYS exactly size 26.

Memory Used = 26 integers + 26 integers = 52 integers.

Because this size is fixed and does NOT grow if 's' becomes
10 billion characters long, the space used is constant.

(Note: We do not count the output Result Array in Space Complexity
as per standard algorithmic conventions).

OVERALL SPACE COMPLEXITY = $O(1)$ Constant Space

4. Solution Code

Python Solutions

```
# -----
# OPTIMIZED SOLUTION: Sliding Window with Array (L5/L6 Level)
# -----

def findAnagrams_optimal(s, p):
    # Edge case: If the target is longer than the string, it's impossible
    if len(p) > len(s):
        return []

    # We use lists of size 26 to represent character frequencies (a-z)
    p_count = [0] * 26
    window_count = [0] * 26
    result = []

    # Phase 1: Populate the target frequency array and the initial window array
    for i in range(len(p)):
        # ord(char) - ord('a') converts 'a' to 0, 'b' to 1, etc.
        p_count[ord(p[i]) - ord('a')] += 1
        window_count[ord(s[i]) - ord('a')] += 1

    # Phase 2: Check the very first window
    # In Python, comparing two lists of integers checks if all elements are identical
    if p_count == window_count:
        result.append(0)

    # Phase 3: Slide the window across the rest of the string 's'
    # 'i' represents the index of the character entering the right side of the window
    for i in range(len(p), len(s)):
        # 1. Add the new character entering the window on the right
        window_count[ord(s[i]) - ord('a')] += 1

        # 2. Remove the old character falling out of the window on the left
        # The character falling out is at index (i - len(p))
        left_char = s[i - len(p)]
        window_count[ord(left_char) - ord('a')] -= 1

        # 3. Compare the updated window with the target
        if p_count == window_count:
            # If they match, the start of this anagram is (i - len(p) + 1)
            result.append(i - len(p) + 1)

    return result
```

```

# -----
# NAIVE SOLUTION: Sorting (Acceptable for discussion, but will Time Out)
# -----
def findAnagrams_naive(s, p):
    if len(p) > len(s):
        return []

    result = []
    # Sort the target string once
    sorted_p = sorted(p)
    window_len = len(p)

    # Check every possible substring of length window_len
    for i in range(len(s) - window_len + 1):
        # Extract substring, sort it, and compare
        substring = s[i : i + window_len]
        if sorted(substring) == sorted_p:
            result.append(i)

    return result

```

JavaScript Solutions

```

// -----
// OPTIMIZED SOLUTION: Sliding Window with Array (L5/L6 Level)
// -----
function findAnagramsOptimal(s, p) {
    if (p.length > s.length) return [];

    // Arrays of size 26 filled with 0
    const pCount = new Array(26).fill(0);
    const windowCount = new Array(26).fill(0);
    const result = [];

    // Helper function to convert char to index (0-25)
    const getIndex = (char) => char.charCodeAt(0) - 97; // 'a' is 97

    // Helper function to compare two arrays of size 26
    const arraysMatch = (arr1, arr2) => {
        for (let i = 0; i < 26; i++) {
            if (arr1[i] !== arr2[i]) return false;
        }
        return true;
    };

    // Phase 1: Populate the initial counts

```

```

    for (let i = 0; i < p.length; i++) {
        pCount[getIndex(p[i])]++;
        windowCount[getIndex(s[i])]++;
    }

    // Phase 2: Check the first window
    if (arraysMatch(pCount, windowCount)) {
        result.push(0);
    }

    // Phase 3: Slide the window
    for (let i = p.length; i < s.length; i++) {
        // 1. Add new right character
        windowCount[getIndex(s[i])]++;

        // 2. Remove old left character
        const leftChar = s[i - p.length];
        windowCount[getIndex(leftChar)]--;

        // 3. Compare
        if (arraysMatch(pCount, windowCount)) {
            result.push(i - p.length + 1);
        }
    }

    return result;
}

// -----
// NAIVE SOLUTION: Sorting (Will cause Time Limit Exceeded)
// -----
function findAnagramsNaive(s, p) {
    if (p.length > s.length) return [];

    const result = [];
    const sortedP = p.split('').sort().join('');
    const windowLen = p.length;

    for (let i = 0; i <= s.length - windowLen; i++) {
        const substring = s.substring(i, i + windowLen);
        const sortedSub = substring.split('').sort().join('');

        if (sortedSub === sortedP) {
            result.push(i);
        }
    }
}

```

```

    return result;
}

```

Note 1: Terminology and Techniques Used

- **Sliding Window:** An algorithmic paradigm used on arrays or strings. It involves creating a “window” (a sub-range of data) that slides over the underlying dataset. It helps because it converts nested loops (which multiply time complexity) into a single sequential pass (which keeps time complexity linear). It applies here perfectly because we need to examine contiguous blocks of characters of a fixed size.
 - **Frequency Array / Map:** A data structure used to count the occurrences of elements. By mapping characters to an array index (e.g., ‘a’ -> 0, ‘z’ -> 25), we can instantly increment or decrement counts in $O(1)$ constant time, rather than searching a string.
-

Note 2: Real World / Interview Variations (Google, Meta, Bloomberg)

Here is how top companies disguise or mutate this exact pattern to test you further:

1. Variation: Minimum Window Substring (Meta - Very Frequent)

* *The Twist:* Instead of finding an *exact* anagram (fixed length), you need to find the *shortest* substring in *s* that contains all the characters of *p* (can include extra, unrelated characters). * *How to solve:* You still use a Sliding Window and Frequency Array, but the window size is **dynamic**. You expand the right edge of the window until you have all the required characters. Once you have them, you shrink the left edge of the window as much as possible until you no longer have all the required characters, keeping track of the minimum length achieved.

2. Variation: Substring with Concatenation of All Words (Google - Hard)

* *The Twist:* Instead of characters, you are dealing with words. You are given a string *s* and an array of words of the exact same length (e.g., ["foo", "bar"]). Find all starting indices of substrings in *s* that are a concatenation of all those words in any order. * *How to solve:* This is the exact same logic, but scaled up. Instead of an array of size 26 for characters, you use a Hash Map where the keys are the string words and the values are their frequencies. Your “Sliding Window” jumps forward by the *length of a word* rather than a single character.

3. Variation: Real-Time Stream Anomaly Detection (Bloomberg - Systems)

* *The Twist:* You aren’t given a string *s*. You are receiving a

continuous stream of network packets or stock ticks. You need to trigger an alert if a specific combination of event types occurs within any 5-minute rolling window. * *How to solve:* You implement a Sliding Window over time (a Queue). As new ticks arrive, you push them to the Queue and update a Frequency Map. You then look at the timestamps; if the oldest event in the Queue is older than 5 minutes, you pop it off and decrement its frequency. If your map matches the “anomaly signature,” you fire the alert.

214. Shortest Palindrome

Here is an end-to-end, L5/L6 level walkthrough for solving “Shortest Palindrome”.

A top-tier engineer approaches this problem by first identifying the underlying bottleneck. They will quickly bypass the brute-force method and recognize that this isn’t just a string manipulation problem—it is a **pattern matching** problem in disguise.

1. Problem Explanation

The Core Goal: You are given a string *s*. You can only add characters to the *front* of the string. Your goal is to make the entire string a palindrome (reads the same forwards and backwards) by adding the fewest characters possible.

Reframing the Problem (The L5 Insight): Instead of guessing what to add, look at what is already there. Every string has a palindromic prefix (even if it’s just the first character).

If *s* = “abac”, the longest palindromic prefix is “aba”. The characters left over are “c”. To make the whole string a palindrome, we just take the leftover characters (“c”), reverse them, and paste them to the front: “c” + “abac” = “cabac”.

The Real Problem: How do we find the *longest palindromic prefix* of a string as fast as possible?

2. Solution Explanation

Approach A: The Brute Force (Good to mention, but sub-optimal)

We could check every possible prefix starting from the longest down to the shortest. 1. Check if “abac” is a palindrome. (No) 2. Check if “aba” is a palindrome. (Yes! Stop here.) *Why it’s sub-optimal:* Reversing and checking strings takes time. If the string is 50,000 characters long, checking every prefix takes Time = Order of N^2 .

Approach B: The “Combined String” Trick (Optimal) We can solve this in linear time using a trick derived from the KMP (Knuth-Morris-Pratt) algorithm.

We create a brand new string by combining the original string `s`, a separator `#`, and the reversed string `s`.

Formula: `combined = s + "#" + reverse(s)`

Let’s visualize why this is pure magic using `s = "abac"`.

```
=====
VISUALIZING THE TRICK
=====
Original (s):      "abac"
Reverse (rev_s):   "caba"
Separator:         "#"

Combined String:   "abac#caba"
```

Now, look at the PREFIXES and SUFFIXES of this combined string:

Prefixes of combined	Suffixes of combined	Match?
a	a	YES (Length 1)
ab	ba	NO
aba	aba	YES (Length 3) <--- LONGEST!
abac	caba	NO

The longest prefix of 'combined' that exactly matches a suffix is "aba" (length 3).

The “Why”: * A suffix of `combined` is literally just a reversed piece of `s`. * If a prefix of `s` exactly matches a reversed piece of `s`, **that prefix is a palindrome!** * The `#` separator is absolutely critical. It prevents the suffix and prefix from accidentally overlapping into each other if the string consists of repeated letters (like “aaaa”).

To find this longest matching prefix-suffix without checking every combination, we build an **LPS Array** (Longest Prefix Suffix array). The last value in this array gives us exactly the length we need.

```
=====
BUILDING THE LPS ARRAY FOR "abac#caba"
=====
```

The LPS array tracks the length of the longest matching prefix-suffix up to that index.

```
Index: 0  1  2  3  4  5  6  7  8
Char  : a  b  a  c  #  c  a  b  a
LPS   : 0  0  1  0  0  0  1  2  3
```

Step-by-step breakdown of LPS values:

```
[0]: 'a' -> No proper prefix/suffix. LPS = 0
[1]: 'ab' -> No match. LPS = 0
[2]: 'aba' -> Prefix 'a' matches suffix 'a'. LPS = 1
...
[6]: 'abac#ca' -> Prefix 'a' matches suffix 'a'. LPS = 1
[7]: 'abac#cab' -> Prefix 'ab' matches suffix 'ab'. LPS = 2
[8]: 'abac#caba' -> Prefix 'aba' matches suffix 'aba'. LPS = 3
```

The final value is 3.

The longest palindromic prefix has length 3 ("aba").

Remaining string to reverse and prepend is "c" (from index 3 to the end of original s).

3. Time and Space Complexity Analysis

Let N be the length of the string s.

TIME COMPLEXITY DERIVATION: Order of N

Operation 1: Reverse string 's'

Requires looking at each character once: Order of N time.

Operation 2: Create 'combined' string

Copying s + '#' + rev_s: Order of N time.

Operation 3: Build LPS Array

We iterate through the combined string of length $2N + 1$.

The two pointers in the KMP algorithm only ever move forward,
or fall back without exceeding the total length.

Number of array operations: Order of N time.

Total Time = $O(N) + O(N) + O(N)$

Total Time = Order of N

SPACE COMPLEXITY DERIVATION: Order of N

Memory allocated:

1. Reversed string 'rev_s': Size N

2. Combined string: Size $2N + 1$

3. LPS Array: Array of integers of size $2N + 1$

Total Space = $N + 2N + 2N = 5N$

Ignoring constants, the memory scales linearly with input size.

Total Space = Order of N

4. Solution Code

Here is the implementation in both Python and JavaScript. Both contain the optimal L5/L6 solution.

Python Solution

```
def shortestPalindrome(s):
    # Edge case: Empty string or single character is already a palindrome
    if not s or len(s) == 1:
        return s

    # Step 1: Create the combined string with a separator
    rev_s = s[::-1]
    combined = s + "#" + rev_s

    # Step 2: Initialize the LPS (Longest Prefix Suffix) array
    n = len(combined)
    lps = [0] * n

    # Step 3: Build the LPS array
    # 'length' tracks the length of the previous longest prefix suffix
    length = 0
    i = 1

    while i < n:
        if combined[i] == combined[length]:
            length += 1
            lps[i] = length
            i += 1
        else:
            # If there is a mismatch, we fall back to the previous valid prefix
            if length != 0:
                length = lps[length - 1]
            else:
                # If length is 0, there is no matching prefix, move forward
                lps[i] = 0
                i += 1

    # Step 4: The last value in the LPS array tells us the length
    # of the longest palindromic prefix in 's'
    longest_pal_prefix_len = lps[-1]
```

```

        # Step 5: Extract the remaining suffix, reverse it, and prepend it
        remaining_suffix = s[longest_pal_prefix_len:]
        return remaining_suffix[::-1] + s

# -----
# ACCEPTABLE NAIVE ALTERNATIVE (For explaining tradeoffs)
# -----
def shortestPalindrome_naive(s):
    rev_s = s[::-1]
    for i in range(len(s) + 1):
        # Check if the prefix of 's' matches the suffix of 'rev_s'
        if s.startswith(rev_s[i:]):
            return rev_s[:i] + s
    return ""

```

JavaScript Solution

```

/**
 * @param {string} s
 * @return {string}
 */
function shortestPalindrome(s) {
    if (!s || s.length <= 1) {
        return s;
    }

    // Step 1: Create the combined string
    const revS = s.split('').reverse().join('');
    const combined = s + "#" + revS;

    // Step 2: Initialize the LPS array
    const n = combined.length;
    const lps = new Array(n).fill(0);

    // Step 3: Build the LPS array
    let length = 0;
    let i = 1;

    while (i < n) {
        if (combined[i] === combined[length]) {
            length++;
            lps[i] = length;
            i++;
        } else {
            // Mismatch handling
            if (length !== 0) {

```

```

        // Fall back to the previous valid prefix length
        length = lps[length - 1];
    } else {
        lps[i] = 0;
        i++;
    }
}

// Step 4: Get the length of the longest palindromic prefix
const longestPalPrefixLen = lps[n - 1];

// Step 5: Extract the non-palindromic suffix, reverse it, and prepend
const remainingSuffix = s.substring(longestPalPrefixLen);
const reversedSuffix = remainingSuffix.split('').reverse().join('');

return reversedSuffix + s;
}

// -----
// ACCEPTABLE NAIVE ALTERNATIVE (For explaining tradeoffs)
// -----
function shortestPalindromeNaive(s) {
    const revS = s.split('').reverse().join('');
    for (let i = 0; i <= s.length; i++) {
        if (s.startsWith(revS.substring(i))) {
            return revS.substring(0, i) + s;
        }
    }
    return "";
}

```

Note 1: Terminology and Techniques Used

- **KMP Algorithm (Knuth-Morris-Pratt):** A classic string-searching algorithm used to find occurrences of a “word” within a main “text string”. It achieves linear time by pre-computing a table (the LPS array) that dictates how far to skip forward when a mismatch occurs, avoiding redundant comparisons.
- **LPS Array (Longest Prefix Suffix):** The core component of KMP. For a given string, an array where the value at index *i* represents the length of the longest proper prefix that is also a suffix in the substring from index 0 to *i*. Applying this tool to a custom *s* + # + *reversed(s)* string is a brilliant adaptation to solve palindrome problems.

Note 2: Real World / Interview Variations (Google, Meta, Bloomberg)

Here are the ways top companies pivot this topic to test your adaptability:

1. Variation: Longest Palindromic Substring (Google / Core Real-World) * *The Twist:* Find the longest palindrome anywhere *inside* the string, not just anchored to the front. (e.g., in “babad”, the answer is “bab” or “aba”).

* *How to solve:* The KMP trick doesn’t work here because the palindrome isn’t anchored to an edge. An L5 engineer will use the “Expand Around Center” technique (Order of N^2 time, Order of 1 space). You loop through each character, treat it as the center of a potential palindrome, and expand pointers outwards to the left and right until the characters no longer match. (For extreme optimization, mention Manacher’s Algorithm, which does this in Order of N time, but is rarely required to be fully coded).

2. Variation: Valid Palindrome II (Meta) * *The Twist:* Given a string, can it be a palindrome if you are allowed to delete *at most one* character? *

How to solve: Use two pointers, one at the start and one at the end, moving inward. If `s[left] == s[right]`, keep going. If they don’t match, you have a choice: either delete the character at `left` or delete the character at `right`. You then run a simple palindrome check on the remaining substring for both choices. If either returns true, the answer is true.

3. Variation: Palindrome Data Stream (Bloomberg / Systems Data)

* *The Twist:* Characters are arriving one by one in a continuous stream. At any given moment, you must return `True` if the stream you have seen so far is a palindrome. You cannot store the whole string because memory is limited. * *How to solve:* You cannot use KMP or standard two-pointers here. You must use a **Rolling Hash** (Rabin-Karp concept). You maintain two integer hashes: a `forward_hash` and a `backward_hash`. * As character `c` arrives: * `forward_hash = (forward_hash * base + value(c)) % MOD` * `backward_hash = (backward_hash + value(c) * power) % MOD` * `power = (power * base) % MOD` * If `forward_hash == backward_hash` at any point, the stream is a palindrome up to that point. This uses Order of 1 Space and Order of 1 Time per character.

String Transforms into Another String

Here is an end-to-end walkthrough for “String Transforms Into Another String” (LeetCode 1153) from the perspective of an L5/L6 (Senior/Staff) engineer.

An engineer at this level knows this is a “gotcha” problem. It looks like a simple Hash Map question at first glance, but it secretly hides a fundamental

concept from Graph Theory and System Design: **Dependency Cycles and Temporary Registers.**

1. Problem Explanation

The Core Goal: You are given two strings, `str1` and `str2`, of equal length. You need to determine if you can turn `str1` into `str2` by replacing all occurrences of a specific character in `str1` with another character, in zero or more steps.

The Rule of Transformation: When you pick a letter to change, *every single instance* of that letter in the current string changes at the exact same time.

Why it is tricky (The “Simultaneous” Trap): Suppose you want to turn `str1 = "ab"` into `str2 = "ba"` (basically swapping them). * If you say “change ‘a’ to ‘b’”, the string becomes `"bb"`. * Now you have lost the original ‘b’! If you try to change ‘b’ to ‘a’, the string becomes `"aa"`. * You failed to get `"ba"`.

To perform a swap, you need a **temporary character** (like how you use a temporary variable `temp` to swap two variables in code). * Change ‘a’ to ‘z’ (temp) -> `"zb"` * Change ‘b’ to ‘a’ -> `"za"` * Change ‘z’ to ‘b’ -> `"ba"` (Success!)

2. Solution Explanation

A senior engineer breaks this problem down into three distinct logical checks.

Check 1: The Base Case If `str1` is already exactly equal to `str2`, no transformations are needed. Return `True`.

Check 2: The Mapping Rule (No One-To-Many mappings) A single character in `str1` cannot transform into two different characters in `str2`. Because all instances of a letter change together, they must all end up as the exact same target letter.

```
=====
INVALID MAPPING VISUALIZATION
=====
str1:  "a b a"
str2:  "c d e"

Index 0: 'a' needs to become 'c'.
Index 2: 'a' needs to become 'e'.

If we change 'a' to 'c', str1 becomes "c b c".
```

We can never make the last letter 'e' without also changing the first letter!
This is a ONE-TO-MANY mapping. It is strictly IMPOSSIBLE.

How to solve this check: We iterate through the strings and build a Hash Map (`str1[i] -> str2[i]`). If we ever see a `str1` character trying to map to a *new* `str2` character when it already has an assigned target, we return `False`.

Check 3: The 26-Character Cycle Trap (The L5/L6 Realization) If the strings pass Check 1 and Check 2, there is only one edge case left that can ruin everything: **A closed loop where no temporary characters are available.**

As explained in the “Swap” example above, resolving a cycle (like `a -> b` and `b -> a`) requires at least one “spare” character in the alphabet that we can use as a temporary stepping stone.

Since there are only 26 lowercase English letters, what happens if `str2` contains *all 26 unique characters*? * If `str2` has all 26 characters, there are zero unused characters left in the alphabet. * If there are zero unused characters, we cannot use a temporary character to break a cycle. * Therefore, if `str2` has 26 unique characters (and `str1 != str2`), it is guaranteed to be impossible.

Let's visualize the Graph Theory behind this:

```
=====
THE CYCLE TRAP VISUALIZATION
=====
```

Let's assume a tiny alphabet of only 3 letters: a, b, c.

```
str1 = "a b c"
str2 = "b c a"
```

Mapping required:

```
'a' -> 'b'
'b' -> 'c'
'c' -> 'a'
```

Attempting the transformation directly:

1. Change 'a' to 'b' --> `str1` becomes `"b b c"`
(FATAL ERROR: We just merged the original 'a' and 'b'. We can never separate them again. 'b' maps to 'c', so they will both become 'c'.)

To fix this, we NEED a 4th letter (like 'x') to act as a temp variable:

1. `a -> x` (`"x b c"`)
2. `b -> c` (`"x c c"`) -- wait, 'c' needs to be 'a'. We merged again.

Actual safe path with a temp variable 'x':

1. `a -> x` (`"x b c"`)

```

2. c -> a  ("x b a")
3. b -> c  ("x c a")
4. x -> b  ("b c a") -> SUCCESS!

```

CONCLUSION: To resolve a chain that loops back on itself, you MUST have at least one character that doesn't exist in the target string. If `unique_chars(str2) == 26`, you have no temp variables available.

3. Time and Space Complexity Analysis

Let N be the length of the string `str1` (and `str2`).

TIME COMPLEXITY DERIVATION: $O(N)$

Input: `str1` (Length N), `str2` (Length N)

```

1. Equality Check: str1 == str2          --->  $O(N)$  time
2. Iterating through strings to build Hash Map: --->  $O(N)$  time
   Inside the loop:
       Hash Map lookup                    --->  $O(1)$  time
       Hash Map insertion                 --->  $O(1)$  time
3. Counting unique characters in str2:    --->  $O(N)$  time

```

Total Time = $O(N) + O(N) + O(N) = O(N)$ time overall.

SPACE COMPLEXITY DERIVATION: $O(1)$ Space

Wait, we are using a Hash Map and a Set. Why is it $O(1)$ and not $O(N)$?

```

1. Hash Map (`mapping`):
   Keys are characters from str1.
   Values are characters from str2.
   Maximum number of unique English lowercase letters is 26.
   Max size of Hash Map = 26.

2. Set (`unique_chars_in_str2`):
   Stores unique characters from str2.
   Max size of Set = 26.

```

Because the memory we allocate is strictly bounded by a constant number (26), it does not grow infinitely as N grows.

Total Space = $O(1)$ (Constant memory)

4. Solution Code

Python Solution

```
def canConvert(str1, str2):
    # Check 1: If they are already identical, 0 steps are needed.
    if str1 == str2:
        return True

    # Check 3 (Early Exit): If target has all 26 chars, we can't break cycles.
    # We use a set to find the number of unique characters in str2.
    if len(set(str2)) == 26:
        return False

    # Check 2: Verify one-to-one mapping
    mapping = {}

    for i in range(len(str1)):
        char1 = str1[i]
        char2 = str2[i]

        # If char1 has been seen before, it MUST map to the same char2
        if char1 in mapping:
            if mapping[char1] != char2:
                # One-to-many mapping detected!
                return False
        else:
            # First time seeing char1, record its target
            mapping[char1] = char2

    # If we mapped everything without conflicts, and target doesn't take
    # up all 26 characters, it is always mathematically possible.
    return True
```

JavaScript Solution

```
/**
 * @param {string} str1
 * @param {string} str2
 * @return {boolean}
 */
function canConvert(str1, str2) {
    // Check 1: If they are identical, 0 transformations needed.
    if (str1 === str2) {
        return true;
    }
}
```

```

// Check 3 (Early Exit): If target uses all 26 chars, no temp space available.
// Set in JS automatically removes duplicates.
const uniqueCharsInStr2 = new Set(str2);
if (uniqueCharsInStr2.size === 26) {
    return false;
}

// Check 2: Verify one-to-one (or many-to-one) mapping
const mapping = new Map();

for (let i = 0; i < str1.length; i++) {
    const char1 = str1[i];
    const char2 = str2[i];

    if (mapping.has(char1)) {
        // If it's already mapped to something else, it's impossible
        if (mapping.get(char1) !== char2) {
            return false;
        }
    } else {
        // Create the new mapping
        mapping.set(char1, char2);
    }
}

return true;
}

```

Note 1: Terminology and Techniques Used

- **Graph Cycles / Directed Graph:** A conceptual way to visualize relationships. If a becomes b, draw an arrow a -> b. If b -> c and c -> a, you have drawn a circle (a cycle).
 - **Temporary Variables (Temp Registers):** In computer science, whenever you swap two pieces of data in memory, you need a third empty space to hold one of the values momentarily so it doesn't get overwritten. This algorithm relies on the fact that an "empty space" is defined as a character not present in the target string.
-

Note 2: Real World / Interview Variations (Google, Meta, Bloomberg)

Top companies test this concept under different disguises. They want to see if you understand **Dependency Resolution**.

1. Variation: Compiler Register Allocation / Graph Coloring (Google) * *The Twist*: You have variables in CPU registers (R1, R2, R3). You want to move data from old registers to new registers simultaneously (e.g., R1→R2, R2→R1). How many temporary registers do you need? * *How to solve*: This is the exact same problem. You build a directed graph of the dependencies. If there are no cycles, you need 0 temporary registers (just do it in reverse topological order). If there is a cycle, you need exactly 1 temporary register (like our spare alphabet character) to break the cycle.

2. Variation: Financial Currency Arbitrage / Exchange Routing (Bloomberg) * *The Twist*: You have an initial portfolio of currencies ["USD", "EUR", "GBP"] and a target portfolio ["EUR", "GBP", "USD"]. You can execute batched exchange contracts, but you can't hold two different originating currencies in the same target currency account simultaneously during transit. * *How to solve*: You apply the one-to-many mapping check (you can't route USD to both EUR and CAD in the same batch command). Then you check for cycles. To resolve the circular swap between USD/EUR/GBP, you must route through a "temporary" 4th currency (like JPY) that isn't currently being utilized in the target portfolio.

3. Variation: Keyboard Remapping / State Machine Transitions (Meta) * *The Twist*: You are writing firmware for a customizable keyboard. A user submits a batch script to remap keys (e.g., "Change A to S, change S to D, change D to A"). Can the firmware execute this if it only has memory to hold the state of the standard 104 keys? * *How to solve*: It's the 26-character rule scaled up to 104. If the target state utilizes all 104 keys, the firmware will overwrite memory during the cycle. The firmware must either have a 105th hidden "buffer" key in memory, or the software must reject the user's input if it requires all keys and contains a cycle.

1653. Minimum Deletions to Make String Balanced

Here is an end-to-end, top-tier (L5/L6) walkthrough for solving "Minimum Deletions to Make String Balanced".

An L5/L6 engineer will immediately recognize that while this problem can be solved with a brute-force approach, it is secretly a classic **Dynamic Programming** or **Prefix/Suffix** optimization problem. An interviewer at this level wants to see you start with a solid mental model and logically compress it down to an incredibly efficient, $O(1)$ space solution.

1. Problem Explanation

The Core Goal: You are given a string made entirely of the letters 'a' and 'b'. Your goal is to delete the minimum number of characters so that the string becomes "balanced".

What does "Balanced" mean? A string is balanced if there is **never** a 'b' that comes before an 'a'. In other words, once you see a 'b', you can never see an 'a' again. The string must look like zero or more 'a's, followed by zero or more 'b's (e.g., "aaabbb", "aaaa", "bbbb", or empty).

The Trap: The string might have multiple overlapping conflicts, like "bababa". You can't just delete all 'b's or all 'a's. You have to find the cheapest combination of deletions.

2. Solution Explanation

Let's look at how a Senior Engineer builds up to the optimal solution.

Approach 1: The "Split Point" Mental Model (Good, but uses extra memory) Imagine drawing a vertical line anywhere in the string. We will call everything to the left "Team A" and everything to the right "Team B". To make the string balanced at this specific line, we must: 1. Delete any 'b's on the left side of the line. 2. Delete any 'a's on the right side of the line.

If we calculate this cost for *every possible line position*, the lowest cost is our answer.

Example: `s = "a b b a b"`

Let's test a split line after index 2:

	Left Side		Right Side
String:	a b b		a b
	-----		-----
Rule:	Must be		Must be
	all 'a's		all 'b's

Deletions needed:

- Left side has two 'b's -> delete them (Cost: 2)
- Right side has one 'a' -> delete it (Cost: 1)

Total Cost for this line = 3

This is a great start, but storing the counts of 'a's and 'b's for every position requires extra memory. We can do better.

Approach 2: The Optimal “One-Pass” State Tracker (L6 Level) Instead of looking at the whole string at once, let’s read it from left to right, character by character.

As we read, we keep track of two variables: 1. **b_count**: How many ‘b’s we have seen so far. 2. **deletions**: The minimum deletions needed to keep the string we have processed so far balanced.

The Golden Rule: * If we see a ‘b’, it doesn’t break any rules because ‘b’s are allowed at the end. We just do **b_count + 1**. * If we see an ‘a’, we have a potential conflict! If we have seen ‘b’s previously, this ‘a’ is appearing *after* a ‘b’. We have two choices to fix this: **Choice 1:** Delete this new ‘a’ (Cost: **deletions + 1**) **Choice 2:** Keep this ‘a’, which means we **MUST** delete ALL the ‘b’s we have seen so far (Cost: **b_count**). We always take the minimum of these two choices!

Let’s visualize this step-by-step for the string "a b b a b a"

```
=====
INITIAL STATE
=====
String: "a b b a b a"
b_count = 0
deletions = 0

=====
STEP 1: Process 'a'
=====
Current char: 'a'
Rule: It's an 'a'. Are there any 'b's to conflict with? b_count is 0. No conflict.
b_count = 0
deletions = 0

=====
STEP 2: Process 'b'
=====
Current char: 'b'
Rule: It's a 'b'. Just count it.
b_count = 1
deletions = 0

=====
STEP 3: Process 'b'
=====
Current char: 'b'
Rule: It's a 'b'. Just count it.
b_count = 2
deletions = 0
```

```

=====
STEP 4: Process 'a' (THE FIRST CONFLICT!)
=====

Current char: 'a'
Rule: We have an 'a', but we have seen 'b's! (b_count is 2).
We must choose the minimum cost to fix this:
- Choice 1: Delete this 'a' -> cost is current deletions (0) + 1 = 1
- Choice 2: Delete all 'b's seen so far -> cost is b_count = 2
Minimum is 1.

b_count = 2
deletions = 1  <-- We decided it's cheaper to drop this 'a'.

=====

STEP 5: Process 'b'
=====

Current char: 'b'
Rule: It's a 'b'. Just count it.
b_count = 3
deletions = 1

=====

STEP 6: Process 'a' (SECOND CONFLICT!)
=====

Current char: 'a'
Rule: We have an 'a', but we have seen 'b's! (b_count is 3).
Choose the minimum cost:
- Choice 1: Delete this 'a' -> cost is current deletions (1) + 1 = 2
- Choice 2: Delete all 'b's seen so far -> cost is b_count = 3
Minimum is 2.

b_count = 3
deletions = 2

=====

FINAL RESULT: deletions = 2
=====

```

3. Time and Space Complexity Analysis

Let N be the total number of characters in the string.

TIME COMPLEXITY DERIVATION: $O(N)$

Input String Length = N

Operation Loop:

```
For i from 0 to N-1:           ---> Loops exactly N times
    If 'b':
        Add 1 to b_count       ---> O(1) time
    If 'a':
        Calculate min(del+1, b_count) ---> O(1) time
```

Total Time = N loops * (O(1) operations)

Overall Time Complexity = O(N)

SPACE COMPLEXITY DERIVATION: O(1)

What memory are we allocating?

1. An integer for 'b_count'
2. An integer for 'deletions'

Regardless of whether the string is 10 characters or 10 billion characters, we only ever need those two integer variables to track our state. We do not create any arrays or copies of the string.

Total Space Complexity = O(1) Constant Space

4. Solution Code

Here is the implementation in both Python and JavaScript. I am providing the Optimal One-Pass approach, as it is the most impressive to write in an interview setting.

Python Solution

```
def minimumDeletions(s):
    """
    Calculates the minimum deletions to make the string balanced.
    """
    b_count = 0
    deletions = 0

    for char in s:
        if char == 'b':
            # Seeing a 'b' doesn't cause an immediate violation.
            # We just note that we have seen it.
            b_count += 1
```

```

else:
    # We found an 'a'. We must decide:
    # 1. Delete this new 'a' (deletions + 1)
    # 2. Keep this 'a' and delete ALL preceding 'b's (b_count)
    # We greedily take the cheaper option.
    deletions = min(deletions + 1, b_count)

return deletions

```

JavaScript Solution

```

/**
 * Calculates the minimum deletions to make the string balanced.
 * @param {string} s
 * @return {number}
 */
function minimumDeletions(s) {
    let bCount = 0;
    let deletions = 0;

    for (let i = 0; i < s.length; i++) {
        const char = s[i];

        if (char === 'b') {
            // No violation yet, just increment our count of 'b's
            bCount++;
        } else {
            // Conflict found (an 'a' appearing).
            // Math.min checks if it's cheaper to delete this 'a'
            // or to delete all 'b's we've encountered so far.
            deletions = Math.min(deletions + 1, bCount);
        }
    }

    return deletions;
}

```

Note 1: Terminology and Techniques Used

- **Dynamic Programming (State Tracking):** The optimal solution is essentially a compressed dynamic programming approach. In traditional DP, we might maintain an array `dp[i]` storing the minimum deletions for the substring up to index `i`. However, because the answer for the current character only depends on the *immediately previous state*, we can optimize

away the array and just use two variables (`b_count` and `deletions`). This is known as **State Space Optimization**.

Note 2: Real World / Interview Variations (Google, Meta, Bloomberg)

Top companies often hide this exact logic behind real-world scenarios:

1. Variation: Validating Deployment Logs (Google) * *The Twist:* You are parsing a continuous server log of “Success” (‘a’) and “Failure” (‘b’) signals. A server is considered “Degraded” if a Success is logged *after* a Failure (indicating an unstable flap). Find the minimum number of logs to delete to make the timeline show a perfectly stable phase (only successes) followed by a completely failed phase (only failures), or purely one or the other. * *How to solve:* This is identical. Map “Success” to ‘a’ and “Failure” to ‘b’. Use the exact same state-tracking algorithm.

2. Variation: Priority Rendering in Newsfeeds (Meta) * *The Twist:* You have an array of content IDs where Type 1 is “Organic Post” (‘a’) and Type 2 is “Sponsored Ad” (‘b’). A strict UX rule states that in a specific high-engagement cluster, organic posts must precede sponsored ads. How many items must be hidden to enforce this rule? * *How to solve:* Map Organic to ‘a’, Sponsored to ‘b’. Use the `b_count` to track how many ads you’ve seen, and `deletions` to track the minimum items to hide when an Organic post appears out of order.

3. Variation: Batching Order Execution (Bloomberg) * *The Twist:* You receive an incoming stream of trade requests: “BUY” (‘a’) and “SELL” (‘b’). A specific dark pool algorithm requires all BUY orders to be processed *before* any SELL orders in a given batch. What is the minimum number of orders you must drop from the batch to satisfy this sequence? * *How to solve:* Again, exact same logic. Every time a “BUY” comes after “SELL”s have started, you must decide if it is cheaper to drop the new “BUY” or drop all the preceding “SELL”s. Minimum cost DP solves it in $O(N)$ time and $O(1)$ space.

777. Swap Adjacent in LR String

Here is an end-to-end, Staff-level (L5/L6) walkthrough for solving “Swap Adjacent in LR String.”

An L5/L6 engineer won’t immediately start writing a Breadth-First Search (BFS) to try every possible swap. A simulation or BFS would take exponential time because the number of possible states explodes. Instead, a senior engineer looks for **Invariants** — fundamental rules of the system that never change, no matter how many swaps you make.

1. Problem Explanation

The Core Goal: You are given a **start** string and an **end** string. Both strings only contain three types of characters: * **L**: A piece that can only move **Left**. * **R**: A piece that can only move **Right**. * **X**: An empty space.

The Rules of Movement: 1. **Move Left (XL -> LX)**: An L can swap with an X to its left. (Think of it as walking left into an empty space). 2. **Move Right (RX -> XR)**: An R can swap with an X to its right. (Think of it as walking right into an empty space). 3. **No Crossing**: An L cannot swap with an R. They block each other. 4. **No Backwards**: An L can NEVER move right. An R can NEVER move left.

You need to determine if it is mathematically possible to transform the **start** string into the **end** string by following these rules.

2. Solution Explanation

To solve this efficiently, we must establish three “Invariants” (unbreakable rules).

Invariant 1: The Relative Order of Ls and Rs Because L and R cannot swap with each other, they can never leapfrog one another. If you remove all the empty spaces (X), the sequence of Ls and Rs in the **start** string MUST be exactly identical to the sequence in the **end** string. *If the sequences don't match, it is physically impossible to transform the string.*

Invariant 2: The Direction of L An L can only move left. Therefore, the starting position of any L must be at the same index or a *higher* index than its final destination in the **end** string. *If an L needs to move to a higher index (move right), it is impossible.*

Invariant 3: The Direction of R An R can only move right. Therefore, the starting position of any R must be at the same index or a *lower* index than its final destination in the **end** string. *If an R needs to move to a lower index (move left), it is impossible.*

The Two-Pointer Strategy: We will use two pointers, i for **start** and j for **end**. We will scan through the strings from left to right, skipping all Xs, and solely comparing the positions of the Ls and Rs to check if our invariants hold true.

Let's visualize this with different cases.

Case 1: Valid Transformation

```
=====
INITIAL STATE
=====
start: "R X X L"
```

end: "X R L X"

We set pointer 'i' at the start of 'start', and 'j' at the start of 'end'.

```
=====
STEP 1: Find the first non-X characters
=====
```

start: R X X L
 ^
 i (Index 0)

end: X R L X
 ^
 j (Index 1)

Check Invariant 1 (Identity):
start[i] is 'R'. end[j] is 'R'. They match! The sequence is intact.

Check Invariant 3 (R moves Right):
We are looking at 'R'.
Its start position 'i' is 0.
Its end position 'j' is 1.
Can 'R' move from index 0 to 1? YES, because $0 \leq 1$.

Move both pointers forward to find the next non-X characters.

```
=====
STEP 2: Find the next non-X characters
=====
```

start: R X X L
 ^
 i (Index 3)

end: X R L X
 ^
 j (Index 2)

Check Invariant 1 (Identity):
start[i] is 'L'. end[j] is 'L'. They match!

Check Invariant 2 (L moves Left):
We are looking at 'L'.
Its start position 'i' is 3.
Its end position 'j' is 2.
Can 'L' move from index 3 to 2? YES, because $3 \geq 2$.

Move both pointers forward.

```
=====
CONCLUSION
=====
Both pointers reach the end of the strings. All rules were followed.
Return TRUE.
```

Case 2: Invalid Transformation (Blocking / Leapfrogging)

```
=====
INITIAL STATE
=====
start: "R X L"
end:   "L X R"

=====
STEP 1: Check Invariants
=====
start: R X L
      ^
      i (Index 0)

end:   L X R
      ^
      j (Index 0)

Check Invariant 1 (Identity):
start[i] is 'R'.
end[j] is 'L'.
CONFLICT! The relative ordering of pieces is different.
An 'R' cannot turn into an 'L', and they cannot cross each other.

Action: Return FALSE immediately.
```

Case 3: Invalid Transformation (Wrong Direction)

```
=====
INITIAL STATE
=====
start: "L X X"
end:   "X X L"

=====
STEP 1: Check Invariants
=====
```

start: L X X
 [^]
 i (Index 0)

end: X X L
 [^]
 j (Index 2)

Check Invariant 1 (Identity):
start[i] is 'L'. end[j] is 'L'. They match.

Check Invariant 2 (L moves Left):
We are looking at 'L'.
Start position 'i' is 0.
End position 'j' is 2.
Can 'L' move from index 0 to index 2?
NO! That requires moving right. 'L' can only move left.

Action: Return FALSE immediately.

3. Time and Space Complexity Analysis

Let N be the length of the string.

TIME COMPLEXITY DERIVATION: Order of N, $O(N)$

Pointers: i and j

Both pointers start at index 0 and move towards index N.
They only ever move FORWARD. They never backtrack.
Pointer 'i' will visit each character in `start` at most once.
Pointer 'j' will visit each character in `end` at most once.

Maximum total operations = N steps for i + N steps for j = 2N
Ignoring constants, the time scales linearly with the input size.

Total Time Complexity = $O(N)$

SPACE COMPLEXITY DERIVATION: Order of 1, $O(1)$

Memory allocated:
- Integer variable 'i'
- Integer variable 'j'
- Length variable 'N'

We are NOT creating any new strings.
We are NOT using recursion (which would consume call stack memory).
No matter if the string is 10 characters or 10 million characters long,
we only ever use exactly two integer pointers.

Total Space Complexity = $O(1)$ (Constant Space)

4. Solution Code

Here is the implementation of the Two-Pointer strategy.

Python Solution

```
def canTransform(start: str, end: str) -> bool:
    # If the lengths don't match, transformation is impossible
    if len(start) != len(end):
        return False

    n = len(start)
    i = 0
    j = 0

    while i < n or j < n:
        # Advance pointer i until it points to a non-'X' character or goes out of bounds
        while i < n and start[i] == 'X':
            i += 1

        # Advance pointer j until it points to a non-'X' character or goes out of bounds
        while j < n and end[j] == 'X':
            j += 1

        # If both pointers went out of bounds at the same time, we successfully checked every character
        if i == n and j == n:
            return True

        # If only ONE pointer went out of bounds, there's a mismatch in the number of L's and R's
        if i == n or j == n:
            return False

        # Invariant 1: The characters must be identical. L cannot cross R.
        if start[i] != end[j]:
            return False

        # Invariant 2: 'L' can only move left. Start index must be >= end index.
```

```

    if start[i] == 'L' and i < j:
        return False

    # Invariant 3: 'R' can only move right. Start index must be <= end index.
    if start[i] == 'R' and i > j:
        return False

    # Characters are valid, move both pointers forward to check the next pair
    i += 1
    j += 1

return True

```

JavaScript Solution

```

/**
 * @param {string} start
 * @param {string} end
 * @return {boolean}
 */
function canTransform(start, end) {
    if (start.length !== end.length) {
        return false;
    }

    const n = start.length;
    let i = 0;
    let j = 0;

    while (i < n || j < n) {
        // Skip 'X' in start string
        while (i < n && start[i] === 'X') {
            i++;
        }

        // Skip 'X' in end string
        while (j < n && end[j] === 'X') {
            j++;
        }

        // Both reached the end successfully
        if (i === n && j === n) {
            return true;
        }

        // One string has more L/R characters than the other

```

```

    if (i === n || j === n) {
        return false;
    }

    // Check relative ordering (Invariant 1)
    if (start[i] !== end[j]) {
        return false;
    }

    // Check movement constraints (Invariants 2 & 3)
    // 'L' cannot move right (i must be >= j)
    if (start[i] === 'L' && i < j) {
        return false;
    }

    // 'R' cannot move left (i must be <= j)
    if (start[i] === 'R' && i > j) {
        return false;
    }

    i++;
    j++;
}

return true;
}

```

Note 1: Terminology Used

- **Two-Pointer Technique:** An algorithmic pattern where two variables (pointers) iterate through a data structure at different speeds or under different conditions. In this problem, it helps us compare the N^{th} valid character in **start** directly against the N^{th} valid character in **end** without allocating extra memory to build new arrays without Xs.
 - **Invariants / State-Space Reduction:** Instead of brute-forcing all possible states (State-Space Exploration), we define rules that the final state *must* adhere to (Invariants). If the rules are broken, we mathematically prove the final state is unreachable, reducing an $O(2^N)$ exponential time problem down to an $O(N)$ linear time verification problem.
-

Note 2: Real-World / Interview Variations (Google, Meta, Bloomberg)

Here are ways top companies disguise or expand on this problem:

1. Variation: One-Dimensional Traffic Jam / Frogs on Lilies (Google)

* *The Twist*: You are given an array representing a narrow road. > represents cars going right, < represents cars going left, and _ is an empty space. Cars cannot pass each other. Can state A become state B? * *How to solve*: This is the exact same problem disguised with different characters. > is R, < is L, and _ is X. You solve it using the exact same two-pointer invariant logic.

2. Variation: Minimum Swaps to reach End State (Meta) * *The Twist*:

Instead of just returning `True` or `False`, return the *minimum number of swaps* required to go from `start` to `end`. * *How to solve*: You first run the validation logic above. If it returns `False`, return -1 (impossible). If it returns `True`, you calculate the distance. Because pieces cannot leapfrog, piece `k` in `start` *must* map directly to piece `k` in `end`. You sum the absolute difference of their indices: `Total Swaps = Sum of Math.abs(start_index_of_piece - end_index_of_piece)` for all non-X pieces.

3. Variation: 2D Grid Movement / Warehouse Robots (Bloomberg)

* *The Twist*: You have a 2D grid. Robots can only move South (Down) or East (Right) into empty spaces. They cannot pass each other. Can grid A become grid B? * *How to solve*: This is the 2D evolution of the problem. It requires checking constraints row by row and column by column. The invariants remain similar: relative ordering must be preserved horizontally and vertically, and starting coordinates (`row`, `col`) must be $\leq$ ending coordinates since they can only move positively on the axes.

833. Find And Replace in String

As an AI assistant channeling the mindset of a Senior/Staff Engineer (L5/L6), I can tell you that this problem is a classic test of **state management and mental models**.

A junior engineer will often rush into manipulating the string directly and fall into the “Index Shifting Trap.” A top-tier engineer steps back, recognizes the trap, and designs a clean, deterministic system that avoids mutating data mid-flight.

Here is a comprehensive, end-to-end walkthrough of how to master “Find And Replace in String.”

1. Problem Explanation

The Goal: You are given an original string **s**, and a list of “replacement operations”. Each operation tells you: 1. **Index:** Where to look in the string. 2. **Source:** The exact substring you expect to find there. 3. **Target:** What you should replace it with.

The Two Golden Rules: 1. **The Verification Rule:** You must check if the source string actually exists at the given **index**. If it doesn’t match perfectly, you **ignore** that specific operation entirely. 2. **The Simultaneous Rule:** All operations are meant to happen “at the same time.” This is the hardest part of the problem. If you start replacing parts of the string left-to-right, the string changes its length. When the string length changes, all the remaining **indices** you were given are now pointing to the wrong letters!

Let’s visualize the “Index Shifting Trap” to understand why naive solutions fail.

```
=====
THE INDEX SHIFTING TRAP (Why naive left-to-right fails)
=====

String: "abcd"
Operations:
Op 0: At index 0, replace "a" with "eee"
Op 1: At index 2, replace "cd" with "ffff"

Let's do Op 0 first:
Index:  0  1  2  3
String: [a][b][c][d]
        |
        Replace "a" with "eee"

New String: "eeebcd"

Now let's do Op 1 (At index 2, replace "cd"):
Index:  0  1  2  3  4  5
String: [e][e][e][b][c][d]
        |
        Look at index 2. Is it "cd"?
        No! It is "e". The operation fails!

Result: "eeebcd" (INCORRECT. The "cd" got shifted away from index 2).
```

2. Solution Explanation

To solve this like an L5/L6, we separate the logic into two distinct phases: **Validation** and **Construction**. We never modify the original string. Instead,

we use a “Read Pointer” to scan the old string and build a brand new string from scratch.

Phase 1: The Validation Map We create an array the exact same size as our original string. We’ll call it the `match` array. We evaluate every operation up front. If an operation is valid (the source text matches the original string), we record the *Operation ID* at that starting index in our `match` array.

Phase 2: The Construction Phase (Read and Jump) We use a pointer to read through the original string from left to right: * If our `match` array says there is NO operation at our current index, we copy the current letter to our new string, and move 1 step forward. * If our `match` array says there IS a valid operation here, we append the `target` word to our new string, and we “jump” our pointer forward by the length of the `source` word (skipping over the old text).

Let’s visualize the winning strategy:

```
=====
PHASE 1: VALIDATION MAP
=====

s = "abcd"
indices = [0, 2], sources = ["a", "cd"], targets = ["eee", "ffff"]

Original String:  [a]  [b]  [c]  [d]
Index:           0   1   2   3

Create a Match Array filled with -1 (meaning "No Operation"):
Match Array:      [-1] [-1] [-1] [-1]

Check Op 0: Index 0, Source "a". Does s[0] == "a"? YES.
-> Write 'Op 0' at MatchArray[0]
Match Array:      [ 0] [-1] [-1] [-1]

Check Op 1: Index 2, Source "cd". Does s[2:4] == "cd"? YES.
-> Write 'Op 1' at MatchArray[2]
Match Array:      [ 0] [-1] [ 1] [-1]

=====
PHASE 2: CONSTRUCTION (Read and Jump)
=====

New String Builder: ""
Pointer (ptr) = 0

-----
STEP 1: ptr = 0
```

Match Array at ptr 0 is [0]. This means execute Op 0!
Op 0 Target is "eee". Op 0 Source ("a") length is 1.

Action: Append "eee" to New String.
Action: Jump ptr forward by 1.

New String: "eee"
Next ptr: 1

STEP 2: ptr = 1
Match Array at ptr 1 is [-1]. No operation here.

Action: Append original char s[1] ('b') to New String.
Action: Move ptr forward by 1.

New String: "eeeb"
Next ptr: 2

STEP 3: ptr = 2
Match Array at ptr 2 is [1]. This means execute Op 1!
Op 1 Target is "ffff". Op 1 Source ("cd") length is 2.

Action: Append "ffff" to New String.
Action: Jump ptr forward by 2.

New String: "eeebffff"
Next ptr: 4

STEP 4: ptr = 4
Pointer has reached the end of the string (length is 4).
We are done!

Final Result: "eeebffff"

3. Time and Space Complexity Analysis

Let **N** be the length of the string **s**. Let **K** be the number of replacement operations. Let **M** be the maximum length of a **source** string.

TIME COMPLEXITY DERIVATION: $O(N + K * M)$

Phase 1: Validation

We loop through K operations.

For each operation, we compare a substring of length up to M.

Substring comparison takes $O(M)$ time.

Validation Time = K operations * M characters = $O(K * M)$

Phase 2: Construction

We use a pointer to traverse the string of length N exactly once.

Appending strings is generally optimized to $O(1)$ in a list builder.

Construction Time = $O(N)$

Total Time = $O(K * M) + O(N) = O(N + K * M)$

SPACE COMPLEXITY DERIVATION: $O(N)$

What memory are we allocating?

1. The 'Match Array' is exactly size N. $\rightarrow O(N)$ Space.

2. The 'New String Builder' will hold the final string.

In the worst case, it scales linearly with the input and targets.

We consider the result string storage as $O(N)$ relative to the growth of the system.

Total Space = $O(N)$

4. Solution Code

Python Snippet

```
def findReplaceString(s, indices, sources, targets):  
    """  
    Builds a new string from left to right using a lookup map to  
    ensure simultaneous replacement rules are respected.  
    """  
    n = len(s)  
    # Step 1: Create a match array to store the operation ID for valid replacements.  
    # We initialize with -1 to indicate "no operation starts here".  
    match = [-1] * n  
  
    # Evaluate all operations  
    for i in range(len(indices)):  
        idx = indices[i]  
        src = sources[i]  
  
        # Verify if the source string exists at the given index  
        # s[idx : idx + len(src)] safely slices the string without out-of-bounds errors
```

```

        if s[idx : idx + len(src)] == src:
            match[idx] = i # Record the operation ID at this index

# Step 2: Build the new string
result = []
ptr = 0

while ptr < n:
    if match[ptr] != -1:
        # We found a valid operation starting at this index
        op_idx = match[ptr]

        # Append the target string
        result.append(targets[op_idx])

        # "Jump" the pointer past the original source string
        ptr += len(sources[op_idx])
    else:
        # No operation here, keep the original character
        result.append(s[ptr])
        ptr += 1

# Join the array of strings into a single final string
return "".join(result)

```

JavaScript Snippet

```

/**
 * @param {string} s
 * @param {number[]} indices
 * @param {string[]} sources
 * @param {string[]} targets
 * @return {string}
 */
function findReplaceString(s, indices, sources, targets) {
    const n = s.length;

    // Step 1: Create a match array filled with -1
    const match = new Array(n).fill(-1);

    // Evaluate all operations
    for (let i = 0; i < indices.length; i++) {
        const idx = indices[i];
        const src = sources[i];

        // Verify if the source string exactly matches the substring in 's'

```

```

        // substring(start, end) safely handles out-of-bounds in JS
        if (s.substring(idx, idx + src.length) === src) {
            match[idx] = i; // Record the operation ID
        }
    }

    // Step 2: Build the new string
    // In JS, string concatenation is heavily optimized by modern engines (V8),
    // but using an array and joining is also a very safe, performant pattern.
    const result = [];
    let ptr = 0;

    while (ptr < n) {
        if (match[ptr] !== -1) {
            // Valid operation found
            const opIdx = match[ptr];

            result.push(targets[opIdx]);

            // Jump the pointer forward by the length of the original text we are replacing
            ptr += sources[opIdx].length;
        } else {
            // No operation, push the original character
            result.push(s[ptr]);
            ptr++;
        }
    }

    return result.join("");
}

```

Note 1: Terms and Techniques

- **Parallel Arrays:** This problem uses `indices`, `sources`, and `targets` where the items at index `i` in all three arrays belong to the same logical “Operation Object”. Identifying this helps you realize that mapping them together via an index `i` is the key to tracking them.
- **Two-Pass Algorithm:** The solution above is a classic Two-Pass approach. Pass 1 validates and records state. Pass 2 executes based on that state. This technique is incredibly common in systems engineering because it separates “Business Logic/Validation” from “Data Mutation,” resulting in highly predictable and bug-free code.
- **Read Head & Write Tape Mental Model:** We treated the `ptr` like the read-head of a cassette player moving over the old string, and the

`result` array as a new blank tape we are writing to. This prevents us from corrupting our own data.

Note 2: Real-World / Interview Variations (Google, Meta, Bloomberg)

Top companies use the core principles of this problem to test your ability to handle data streams, text editors, and state tracking.

1. Variation: Design the backend for a Collaborative Text Editor like Google Docs (Google) * *The Twist*: Instead of a static string, you have a live document. User A sends an operation: (Insert "hello" at index 5). User B sends an operation simultaneously: (Delete 3 characters at index 2). How do you apply them without corrupting the document? * *How to solve*: You must implement **Operational Transformation (OT)**. This is the real-world evolution of our Leetcode problem. If User B deletes characters *before* User A's target index, the server must mathematically intercept User A's operation and shift its target index to $5 - 3 = 2$ before applying it. You build a system that detects index collisions and shifts pending operations dynamically.

2. Variation: Text Editor Undo/Redo Engine (Bloomberg) * *The Twist*: You need to implement an undo/redo stack. You are given a string and a sequence of replacement operations. How do you roll them back? * *How to solve*: Using the exact logic from our solution, you don't just execute operations; you save the *inverse* operation to a Stack. If the operation is "Replace 'cd' at index 2 with 'ffff'", you push the inverse operation "Replace 'ffff' at index 2 with 'cd'" to the Undo Stack. Because you validated the state beforehand (Phase 1 of our solution), you guarantee the Undo Stack contains perfectly executable, non-corrupted operations.

3. Variation: Genome Sequence Splicing (Meta / Healthcare Data Teams) * *The Twist*: You are given a massive DNA string (billions of characters) and thousands of mutations that need to be applied. The memory is too tight to just create a copy of the whole string. * *How to solve*: You cannot use our $O(N)$ space array duplication method. Instead, you use a **Rope Data Structure** or a **Doubly Linked List** of string chunks. You sort the operations right-to-left (descending by index). You then traverse the chunks, splitting them at the indices, removing the target nodes, and inserting the new mutation nodes. By going right-to-left, the indices of the un-mutated chunks on the left remain perfectly accurate.

Longest Substring with at most K Distinct Characters

For a senior engineer (L5) or staff engineer (L6) at Google, solving this isn't just about getting the code to pass. It's about demonstrating a mastery of **Sliding Window dynamics**, managing state efficiently, and ensuring the code handles edge cases gracefully.

1. Problem Explanation

The Goal: Find the length of the longest contiguous “substring” within a given string that contains no more than **k** unique characters.

The Constraints: * **Contiguous:** You cannot skip characters. * **Distinct:** The number of unique symbols (letters, numbers, etc.) must be less than or equal to **k**.

Why is it tricky? As you move through the string, adding a new character might push your “distinct count” over the limit **k**. When that happens, you have to figure out exactly how much of the beginning of your string to “chop off” to make the remaining part valid again.

Example: String: “eceba”, $k = 2$ * “ece” has two distinct characters (‘e’, ‘c’). Length 3. * “eceb” has three distinct characters (‘e’, ‘c’, ‘b’). This is **INVALID** because $3 > 2$. * We must shrink the start until we only have 2 distinct characters left.

2. Solution Explanation

The Intuition (The “Why”): We use a **Dynamic Sliding Window**. Imagine a magnifying glass that starts at the beginning of the string. We stretch the right side of the glass as far as we can. Once we see a $(k+1)$ -th unique character, we start pulling the left side of the glass inward until the view inside the glass is valid again.

The Mechanism (The “How”): 1. **Two Pointers:** **left** (the start of the window) and **right** (the end). 2. **Frequency Map:** A Hash Map to keep track of how many times each character appears *inside* our current window. 3. **Expansion:** Move **right** forward and add the character to the map. 4. **Contraction:** If the size of our map (number of unique keys) is greater than **k**, move **left** forward. For every character the **left** pointer passes, decrement its count in the map. If a count hits 0, delete that character from the map entirely. 5. **Record:** At every step where the map size is $\leq k$, calculate **right** - **left** + 1 and keep the maximum.

Walkthrough with ASCII Visualization: Input: `s = "eceba"`, `k = 2`

```
[ INITIAL STATE ]  
s = "e c e b a"  
k = 2  
L = 0, R = 0  
Map = {}  
Max = 0
```

```
-----  
[ STEP 1: R moves to index 0 ('e') ]  
Map = {'e': 1}  
Distinct count (1) <= k (2). Valid!  
Window: [e] c e b a  
Max = max(0, 0-0+1) = 1
```

```
-----  
[ STEP 2: R moves to index 1 ('c') ]  
Map = {'e': 1, 'c': 1}  
Distinct count (2) <= k (2). Valid!  
Window: [e c] e b a  
Max = max(1, 1-0+1) = 2
```

```
-----  
[ STEP 3: R moves to index 2 ('e') ]  
Map = {'e': 2, 'c': 1}  
Distinct count (2) <= k (2). Valid!  
Window: [e c e] b a  
Max = max(2, 2-0+1) = 3
```

```
-----  
[ STEP 4: R moves to index 3 ('b') ]  
Map = {'e': 2, 'c': 1, 'b': 1}  
Distinct count (3) > k (2). INVALID!
```

```
Action: Move L forward until Map size is 2.  
- L was at 0 ('e'). Decr 'e': Map = {'e': 1, 'c': 1, 'b': 1}  
- L moves to 1 ('c'). Decr 'c': Map = {'e': 1, 'c': 0, 'b': 1}  
  Wait, 'c' is now 0. Remove it! Map = {'e': 1, 'b': 1}  
Now Distinct count (2) == k (2). Valid!
```

```
Window: e c [e b] a  
Max = max(3, 3-2+1) = 3 (Max stays 3)
```

```
-----  
[ STEP 5: R moves to index 4 ('a') ]
```

Map = {'e': 1, 'b': 1, 'a': 1}
Distinct count (3) > k (2). INVALID!

Action: Move L forward.
- L was at 2 ('e'). Decr 'e': Map = {'e': 0, 'b': 1, 'a': 1}
 Remove 'e'. Map = {'b': 1, 'a': 1}
Now Distinct count (2) == k (2). Valid!

Window: e c e b [b a]
Max = max(3, 4-3+1) = 3

FINAL MAX LENGTH: 3

3. Time and Space Complexity Analysis

TIME COMPLEXITY: $O(N)$

-
- The 'right' pointer travels from 0 to N-1 exactly once.
 - The 'left' pointer also travels from 0 up to N-1, but never goes backward.
 - Even though there is a 'while' loop inside a 'for' loop, each character is "visited" by the left and right pointers a maximum of two times total.
 - Hash Map operations (add, remove, lookup) are $O(1)$ on average.

Total Operations: N (right pointer) + N (left pointer)
Result: $O(N)$

SPACE COMPLEXITY: $O(K)$

-
- We store characters in a Hash Map.
 - The window shrinks as soon as the map size hits $K + 1$.
 - Therefore, the Map will never contain more than $K + 1$ key-value pairs at any given moment.
 - If K is 2, the map stores 3 items briefly, then drops to 2.

Result: $O(K)$
(Note: In a worst case where K is greater than the total unique characters in the alphabet, it could be $O(26)$ or $O(128)$, which is effectively constant).

4. Solution Code

Python Snippet (Clean & Optimized)

```
def lengthOfLongestSubstringKDistinct(s, k):  
    # Edge Case: If k is 0 or string is empty, max length is 0  
    if not s or k == 0:  
        return 0  
  
    char_freq = {}  
    left = 0  
    max_len = 0  
  
    for right in range(len(s)):  
        # 1. Expand the window  
        char = s[right]  
        char_freq[char] = char_freq.get(char, 0) + 1  
  
        # 2. Shrink the window if we exceed k distinct characters  
        # While the number of unique keys in our map is > k  
        while len(char_freq) > k:  
            left_char = s[left]  
            char_freq[left_char] -= 1  
  
            # If a character's count hits zero, it's no longer in the window  
            if char_freq[left_char] == 0:  
                del char_freq[left_char]  
  
            # Move the start of the window forward  
            left += 1  
  
        # 3. Capture the maximum valid window size found  
        max_len = max(max_len, right - left + 1)  
  
    return max_len
```

JavaScript Snippet

```
function lengthOfLongestSubstringKDistinct(s, k) {  
    if (!s || k === 0) return 0;  
  
    // A Map is often more performant than an Object for frequent additions/deletions  
    const charFreq = new Map();  
    let left = 0;  
    let maxLen = 0;  
  
    for (let right = 0; right < s.length; right++) {
```

```

// 1. Expand
const char = s[right];
charFreq.set(char, (charFreq.get(char) || 0) + 1);

// 2. Shrink
// Map.size gives the number of unique keys in O(1) time
while (charFreq.size > k) {
    const leftChar = s[left];
    charFreq.set(leftChar, charFreq.get(leftChar) - 1);

    if (charFreq.get(leftChar) === 0) {
        charFreq.delete(leftChar);
    }
    left++;
}

// 3. Record
maxLen = Math.max(maxLen, right - left + 1);
}

return maxLen;
}

```

Note 1: Terminology Addendum

Dynamic Sliding Window: * **What it is:** A technique for tracking a subset of data where the start and end of the subset move independently based on logic constraints. * **Why it helps:** It prevents “Restarting from Zero.” Instead of checking every possible substring (which is $O(N^2)$ or $O(N^3)$), we “slide” the valid range forward, preserving the work we did in previous steps. * **How it applies:** Here, it ensures we only traverse the string once to find the longest valid segment.

Note 2: Real-World Interview Variations

1. Google: “Max Video Stream Buffer” * **The Prompt:** You are streaming video chunks. Each chunk has a ‘ContentID’. Your local cache can only hold k unique contents at a time. What is the longest continuous segment of chunks you can play without needing to clear a unique content from the cache? * **The L5 Solve:** This is the same problem. ContentID replaces Characters. You solve it using the same Sliding Window + Frequency Map.

2. Bloomberg: “Unique Ticker Window” * **The Prompt:** Given a stream of stock tickers (AAPL, TSLA, AAPL, MSFT...), find the longest time window

where you observed at most k different companies. * **The L5 Solve:** Again, the same logic. Bloomberg often focuses on the “Streaming” aspect—meaning you might be asked to solve this if the data is too large to fit in memory. In that case, you emphasize that the Sliding Window only needs to know the current map and two pointers, keeping space at $O(k)$.

3. Meta: “Longest Subarray with at most K zeros” * **The Prompt:** In a binary array of 0s and 1s, find the longest subarray with at most k zeros. * **The L5 Solve:** This is a simplified version. Since there are only two types of data (0 and 1), you don’t need a full Map. You just need a single integer variable `zero_count`. If `zero_count > k`, move the left pointer. This shows you can optimize for specific data types.

Shortest Way to Form String

Here is how a Senior Software Engineer (L5/L6) would approach and explain this problem. This problem is a classic “Greedy vs. Optimized Search” challenge often used to distinguish between candidates who can find a working solution and those who can find the most efficient one for large-scale data.

1. Problem Explanation

The Premise: You have two strings: `source` and `target`. You want to construct `target` by concatenating **subsequences** of `source`.

The Goal: Find the **minimum number of subsequences** of `source` needed to form `target`. If it’s impossible, return -1.

What is a Subsequence? A subsequence is derived by deleting zero or more characters from a string without changing the order of the remaining characters. *Example:* If `source` is “abc”, its subsequences include “a”, “b”, “c”, “ab”, “ac”, “bc”, and “abc”.

The Intuition (The “Why”): Think of the `source` string as a reusable stencil. You want to “stamp” characters onto a page to spell the `target` word. Each “stamp” involves one full pass through the `source` string, picking up whatever letters you need in the order they appear.

Example: `source` = “abc”, `target` = “abcbc” 1. First pass of “abc”: We can pick up “a”, “b”, and “c”. (Target remaining: “bc”) 2. Second pass of “abc”: We can pick up “b” and “c”. (Target remaining: “”) Total passes = 2.

2. Solution Explanation

I will explain the two ways an L5/L6 would approach this: the **Greedy approach** (for small strings) and the **Inverted Index + Binary Search approach** (for large-scale production performance).

Approach 1: The Greedy Two-Pointer (Good for an initial pass) We iterate through `target` and for each character, we try to find it in `source` by moving a pointer forward. If we reach the end of `source`, we increment our “count” and reset the `source` pointer to the beginning.

Approach 2: The Optimized Inverted Index (The L5/L6 Choice) If `source` is very long and we have to process many targets, the Greedy approach is slow because it re-scans `source` over and over. Instead, we pre-process `source` into a dictionary (Map) where every character points to a list of indices where it appears.

The “Aha!” Moment: When we are at index `i` in `source` and need the character ‘x’, we don’t need to scan. We just look at our list of ‘x’ positions and find the first index that is **greater than i** using **Binary Search**.

Detailed ASCII Walkthrough (Optimized) `source = "abacaba", target = "aaca"`

Step 1: Pre-process source into an Index Map

```
'a': [0, 2, 4, 6]
'b': [1, 5]
'c': [3]
```

Step 2: Process Target “aaca”

```
Current Source Pointer (p) = -1
Count = 1 (We start our first "pass")
```

1. Target needs 'a':
Look at 'a' list: [0, 2, 4, 6].
Smallest index > p(-1) is 0.
Update p = 0.
2. Target needs 'a':
Look at 'a' list: [0, 2, 4, 6].
Smallest index > p(0) is 2.
Update p = 2.
3. Target needs 'c':
Look at 'c' list: [3].
Smallest index > p(2) is 3.

Update $p = 3$.

4. Target needs 'a':
Look at 'a' list: [0, 2, 4, 6].
Smallest index $> p(3)$ is 4.
Update $p = 4$.

Target finished in 1 pass. Count = 1.

What if we needed “aaaaa”?

- ... (after step 4, $p = 4$)
5. Target needs 'a':
Look at 'a' list: [0, 2, 4, 6].
Smallest index $> p(4)$ is 6.
Update $p = 6$.
 6. Target needs 'a':
Look at 'a' list: [0, 2, 4, 6].
Smallest index $> p(6)$ is... NONE.
Action: Increment Count to 2. Reset $p = -1$.
Now, Smallest index $> p(-1)$ is 0.
Update $p = 0$.
-

3. Time and Space Complexity Analysis

Time Complexity: $O(M * \log N)$ Where M is the length of `target` and N is the length of `source`.

TC Derivation:

1. Pre-processing: Scan source once to build the map.
Work: $O(N)$
2. Processing Target:
For every character in target (M characters):
Perform Binary Search on an index list (max size N).
Work: $O(\log N)$

Total Time: $O(N + (M * \log N))$

In competitive terms, we simplify this to $O(M * \log N)$.

Space Complexity: $O(N)$

SC Derivation:

We store every index of every character of 'source' in our map.

Map Structure:

```
{
    'a': [index1, index2, ...],
    'b': [index1, ...],
    ...
}
```

Total number of indices stored across all lists = Length of source = N.
 Total Space: $O(N)$

4. Solution Code

Python Implementation

```
import collections
import bisect

def shortestWay(source: str, target: str) -> int:
    # Build the inverted index map
    # char -> list of indices where it appears in source
    char_to_indices = collections.defaultdict(list)
    for i, char in enumerate(source):
        char_to_indices[char].append(i)

    num_subsequences = 1
    # source_ptr tracks our current position in the current source "pass"
    source_ptr = -1

    for char in target:
        if char not in char_to_indices:
            # If a character in target doesn't exist in source, it's impossible
            return -1

        # Find the list of indices for the current target character
        indices = char_to_indices[char]

        # Binary Search (bisect_right) to find the smallest index > source_ptr
        # bisect_right returns the position where source_ptr would be inserted
        idx_in_list = bisect.bisect_right(indices, source_ptr)

        if idx_in_list == len(indices):
            # No index found > source_ptr in this pass.
            # Start a new pass from the beginning of source.
            num_subsequences += 1
            source_ptr = indices[0]
        else:
```

```

        # Move the pointer to the found index
        source_ptr = indices[idx_in_list]

    return num_subsequences

```

JavaScript Implementation

```

/**
 * Optimized solution using an Inverted Index and Binary Search
 */
var shortestWay = function(source, target) {
    // 1. Pre-process source into a map of indices
    const charToIndices = new Map();
    for (let i = 0; i < source.length; i++) {
        if (!charToIndices.has(source[i])) {
            charToIndices.set(source[i], []);
        }
        charToIndices.get(source[i]).push(i);
    }

    let numSubsequences = 1;
    let sourcePtr = -1;

    for (let char of target) {
        if (!charToIndices.has(char)) return -1;

        const indices = charToIndices.get(char);

        // 2. Binary Search to find the next available index
        // This is the manual version of 'bisect_right'
        let foundIdx = binarySearchNext(indices, sourcePtr);

        if (foundIdx === -1) {
            // Must start a new pass
            numSubsequences++;
            sourcePtr = indices[0];
        } else {
            sourcePtr = foundIdx;
        }
    }

    return numSubsequences;
};

/**
 * Helper: Find the smallest value in the array strictly greater than 'target'

```

```

*/
function binarySearchNext(arr, target) {
    let left = 0;
    let right = arr.length - 1;
    let res = -1;

    while (left <= right) {
        let mid = Math.floor((left + right) / 2);
        if (arr[mid] > target) {
            res = arr[mid];
            right = mid - 1; // Try to find even smaller
        } else {
            left = mid + 1;
        }
    }
    return res;
}

```

Note 1: New Terms/Techniques

- **Inverted Index:** Instead of searching the raw text, we map characters to their positions. This is how search engines like Google work—they don’t scan the internet for your keyword; they look up your keyword in a pre-built index of websites.
 - **Greedy Algorithm:** At each step, we make the choice that seems best right now (picking the earliest possible character in **source**). In this specific problem, the Greedy choice is guaranteed to lead to the global optimum.
 - **Binary Search (on indices):** Because our index lists are naturally sorted (we scanned **source** from left to right), we can use Binary Search to jump to the next valid position in $\log N$ time instead of scanning.
-

Note 2: Real-World Interview Variations

1. Google: “Multiple Targets with One Source” * Variation: You have one massive **source** string but 10,000 different **target** strings. How do you optimize? * **L5 Solution:** This is exactly why we use the Inverted Index. You build the index **once** $O(N)$. Then for each of the 10,000 targets, you solve in $O(M \log N)$. If you used the Greedy Two-Pointer, you would scan the massive source 10,000 times, which would be extremely slow.

2. Bloomberg: “Shortest Way with Character Cost” * Variation: Each pass through **source** costs \$5, but skipping a character in **source** during a pass costs \$1. Find the minimum cost. * **L5 Solution:** This is no longer purely

Greedy; it's now a **Dynamic Programming** problem. You would maintain a DP array where `dp[i]` is the minimum cost to form `target[0...i]`.

3. Meta: “Subsequence Match with Wildcards” * **Variation:** source contains wildcards ('?') that can match any character. * **L5 Solution:** You would modify your Inverted Index. The '?' character's index list would contain **every** index in source. When you need a character and it's not found in its specific list, you check the '?' list to see if a wildcard is available sooner.

71. Simplify Path

When an L5 or L6 engineer at Google looks at “Simplify Path,” they don't see a string problem; they see a **State Machine** and a **Stack** problem. They look for the simplest way to transform a noisy, redundant input into a “canonical” form while handling edge cases (like going “up” from the root directory) gracefully.

Here is the engineering breakdown of LeetCode 71.

1. Problem Explanation

The Goal: You are given an absolute path for a file or directory in a Unix-style file system (which always begins with a slash /). You need to convert it to the **canonical path**.

The Unix Rules (The “Why”): * **.** (**Period**): Refers to the current directory. It does nothing. `/home/./user` is just `/home/user`. * **..** (**Double Period**): Refers to the directory up one level. `/home/user/..` takes you back to `/home`. * **//** (**Multiple Slashes**): These should be treated as a single slash / . * **Root Edge Case:** If you are at the root / and try to go up (`..`), you just stay at the root. You can't go higher than the beginning.

The Constraints for the Canonical Path: 1. Must start with a single slash /. 2. Directories must be separated by exactly one slash /. 3. Must not end with a trailing slash (unless it's just the root /). 4. Must only contain the “actual” directory names (no `.` or `..`).

2. Solution Explanation

A senior engineer recognizes that path navigation is a **Last-In, First-Out (LIFO)** process. When you see a directory name, you “enter” it (Push). When you see `..`, you “exit” the most recent directory (Pop).

The Strategy: 1. **Splitting:** Instead of scanning character by character and getting lost in slashes, we split the string by the / character. This gives us a list of “components.” 2. **Filtering:** After splitting, we get three types of

components: * **Empty strings or .:** These happen when there are multiple slashes (//) or a single dot. We **ignore** these. * **..:** We **pop** the top element from our stack (if the stack isn't empty). * **Names (e.g., "home", "user"):** We **push** these onto our stack. 3. **Joining:** Finally, we join the stack elements with a / and add a leading /.

ASCII Diagram Walkthrough Let's trace the path: path = "/home//foo/../bar/./"

Step 1: Split by /

Original: / home / / foo / .. / bar / . /
Components: ["", "home", "", "foo", "..", "bar", ".", ""]

Step 2: Process Components with a Stack

1. Component: "" (Empty)
Action: Ignore.
Stack: []
2. Component: "home"
Action: Push "home"
Stack: ["home"]
Visual: [/home]
3. Component: "" (Empty)
Action: Ignore.
Stack: ["home"]
4. Component: "foo"
Action: Push "foo"
Stack: ["home", "foo"]
Visual: [/home] -> [/foo]
5. Component: ".."
Action: Pop! (Go up one level)
Stack: ["home"]
Visual: [/home] (The "foo" is gone)
6. Component: "bar"
Action: Push "bar"
Stack: ["home", "bar"]
Visual: [/home] -> [/bar]
7. Component: "."
Action: Ignore (Current directory)
Stack: ["home", "bar"]

8. Component: "" (Empty)
Action: Ignore.
Stack: ["home", "bar"]

Step 3: Final Assembly

Join stack with "/": "home/bar"
Add leading "/": "/home/bar"
Result: "/home/bar"

3. Time and Space Complexity Analysis

+-----+ TIME COMPLEXITY DERIVATION: $O(N)$ +-----+	
N = Length of the input string 'path'	
1. Splitting the string:	
Scanning the string once to find delimiters.	
Cost: $O(N)$	
2. Processing Components:	
Each component is pushed or popped at most once.	
Total components is less than N.	
Cost: $O(N)$	
3. Joining the result:	
Concatenating the strings in the stack.	
Cost: $O(N)$	
TOTAL TIME: $O(N) + O(N) + O(N) \implies O(N)$	
+-----+	
+-----+ SPACE COMPLEXITY DERIVATION: $O(N)$ +-----+	
1. Split List:	
In the worst case (e.g., "/a/b/c/d"), we store all characters	
in a new list of strings.	
Cost: $O(N)$	
2. Stack Storage:	
In the worst case, the stack stores all directory names.	
Cost: $O(N)$	

| TOTAL SPACE: O(N) auxiliary space to hold the parsed path. |
+-----+
+-----+

4. Solution Code

Python Snippet

```
def simplifyPath(path: str) -> str:
    # Split by slash and filter out empty segments or single dots
    # This handles "/" and "/" automatically
    segments = path.split("/")
    stack = []

    for segment in segments:
        # If segment is "..", we need to go up one level
        if segment == "..":
            # Only pop if there is something to pop (Root check)
            if stack:
                stack.pop()
            # Ignore empty strings (from "/") and current directory "."
        elif segment == "." or segment == "":
            continue
        # Otherwise, it's a valid directory name
        else:
            stack.append(segment)

    # Join with a single slash and ensure it starts with /
    return "/" + "/".join(stack)
```

JavaScript Snippet

```
/**
 * @param {string} path
 * @return {string}
 */
var simplifyPath = function(path) {
    // Split the path into an array of components
    const segments = path.split("/");
    const stack = [];

    for (let segment of segments) {
        // If we encounter '..', we move one directory up by popping
        if (segment === "..") {
            if (stack.length > 0) {
                stack.pop();
            }
        }
    }
}
```

```

    }
    // If segment is empty (from //) or '.', we do nothing
    else if (segment === "." || segment === "") {
        continue;
    }
    // For any other name, push it to our stack
    else {
        stack.push(segment);
    }
}

// Join the stack with '/' and prepend the root slash
return "/" + stack.join("/");
};

```

Note 1: Terminology and Techniques

- **Stack Data Structure:** A stack is a “First-In, Last-Out” structure. It is perfect for navigation (browser history, undo buttons, file paths) because the most recent “room” you entered is the first one you leave when you go back.
 - **Canonicalization:** This is the process of converting data that has more than one possible representation into a “standard” or “canonical” format. It’s critical in security (to prevent directory traversal attacks) and in databases.
-

Note 2: Real-World Interview Variations

1. **Google: “Cloud Storage Path Resolver” * Prompt:** Given a base path (e.g., `gs://bucket/logs/`) and a relative user input (e.g., `../../configs/secret.json`), resolve the full absolute path. *** Solution:** Treat the protocol (`gs://`) and bucket as the “protected” root. Split the base path and the user input. Push the base path onto the stack first, then process the user input components. Ensure the user’s `..` cannot pop the bucket name (security boundary).
2. **Meta: “URL Cleaner” * Prompt:** In a web crawler, you find URLs like `https://fb.com/profile/../../settings/./privacy`. Clean this URL for the database. *** Solution:** Parse the URL to separate the domain and the path. Apply the stack-based simplification to the path string. Re-attach the cleaned path to the domain.
3. **Bloomberg: “Linux Shell Emulator” * Prompt:** You are building a terminal. The current directory is `pwd`. The user types `cd <path>`. What is

the new `pwd`? * **Solution:** This is dynamic. If the `<path>` starts with `/`, it's an absolute path (reset stack). If it doesn't, it's a relative path (start with the current `pwd` in the stack). Process the new components using the same stack logic. Final stack state becomes the new `pwd`.

65. Valid Number

When a senior engineer (L5+) at Google or Meta looks at “Valid Number” (LeetCode 65), they don't see a string parsing problem. They see a **State Machine** problem.

This problem is notoriously “Hard” because of the sheer number of edge cases (signs, dots, exponents, trailing/leading rules). If you try to solve this with a giant pile of `if/else` statements (often called “Spaghetti Code”), you will fail because it is impossible to debug. A senior engineer will instead model this as a **Deterministic Finite Automaton (DFA)**.

1. Problem Explanation

The Core Question: Given a string, determine if it follows the formal rules of a “Valid Number.”

The Components of a Valid Number: 1. **A Decimal or Integer:** (e.g., “2”, “0.1”, “-.9”, “2.”) 2. **Optional Exponent:** (e.g., “e” or “E”, followed by an integer like “e10” or “E-5”)

Visualizing the Anatomy of a Number:

[SIGN]	[DIGITS]	[DOT]	[DIGITS]	[EXPONENT]	[EXP-SIGN]	[EXP-DIGITS]
<code>+</code> ^	<code>12</code> ^	<code>.</code> ^	<code>5</code> ^	<code>e</code> ^	<code>-</code> ^	<code>10</code> ^
Optional	Must have digits if no dot	Opt but only 1	Opt if digits exist before dot	Optional	Optional	Must have digits if 'e' exists

Common “Trap” Examples: * `“.”`: Invalid (Need at least one digit). * `“e9”`: Invalid (Need a number before the ‘e’). * `“1e”`: Invalid (Need a number after the ‘e’). * `“1.2.3”`: Invalid (Only one dot allowed). * `“+-3”`: Invalid (Signs only at the start of a number or exponent).

2. Solution Explanation

To solve this cleanly, we use a **DFA (Deterministic Finite Automaton)**. Imagine you are on a game board. Every character you read tells you which

square to move to. If you end up on a “Winning Square” (Valid State) after reading the whole string, the number is valid.

The State Map:

We define the following states: 1. **Start:** Nothing read yet. 2. **Sign:** Just read + or -. 3. **Integer:** Reading digits before a dot. 4. **Dot (No leading digits):** Just read . but no digits came before it (e.g., "."). 5. **Decimal:** Reading digits after a dot. 6. **Exponent:** Just read e or E. 7. **Exp-Sign:** Just read + or - after an e. 8. **Exp-Integer:** Reading digits after the exponent.

The DFA Transition Logic (The “Game Board”):

CURRENT STATE	IF WE SEE [0-9]	IF WE SEE [+/-]	IF WE SEE [.]	IF WE SEE [e/E]
1. Start	-> 3 (Int)	-> 2 (Sign)	-> 4 (Dot)	FAIL
2. Sign	-> 3 (Int)	FAIL	-> 4 (Dot)	FAIL
3. Integer	-> 3 (Int)	FAIL	-> 5 (Decimal)	-> 6 (Exp)
4. Dot	-> 5 (Decimal)	FAIL	FAIL	FAIL
5. Decimal	-> 5 (Decimal)	FAIL	FAIL	-> 6 (Exp)
6. Exponent	-> 8 (Exp-Int)	-> 7 (Exp-Sign)	FAIL	FAIL
7. Exp-Sign	-> 8 (Exp-Int)	FAIL	FAIL	FAIL
8. Exp-Integer	-> 8 (Exp-Int)	FAIL	FAIL	FAIL

Why this is better: Each character transition is predictable. We don’t need complex look-aheads. We just look at the current character and the current state.

3. Time and Space Complexity Analysis

=====

TIME COMPLEXITY: $O(N)$

=====

Derivation Diagram:

Input String: [" - 1 2 . 5 e + 7 "] (Length N)

| | | | | | | |
v v v v v v v v

DFA Transitions: 1-2-3-3-5-5-6-7-8 (N transitions)

Each character in the string is visited exactly once.
Inside the loop, we perform a single Hash Map or Array lookup
to find the next state, which takes $O(1)$ time.

Total Time = $N * O(1) = O(N)$.

=====

SPACE COMPLEXITY: $O(1)$

=====

Derivation Diagram:

[DFA State Table]

The table is a fixed size (roughly 8 states by 4 character types).
This table is built once and stays the same size regardless of
how long the input string is.

[Current State Variable]

A single integer/string to keep track of where we are.

Total Space = Fixed Table + Variable = Constant = $O(1)$.

=====

4. Solution Code

A senior engineer avoids nested if statements. We define the DFA as a list of dictionaries (a transition table).

Python

```
def isNumber(s: str) -> bool:
    # Define the DFA table. Each index represents a state.
    # The dictionary keys are character types, and values are the next state.
    dfa = [
        # State 0: Start
        {"digit": 1, "sign": 2, "dot": 3},
        # State 1: Integer (Valid End)
        {"digit": 1, "dot": 4, "exp": 5},
        # State 2: Sign
        {"digit": 1, "dot": 3},
        # State 3: Dot (No leading digits)
        {"digit": 4},
        # State 4: Decimal (Valid End)
        {"digit": 4, "exp": 5},
        # State 5: Exponent
        {"digit": 7, "sign": 6},
        # State 6: Exp-Sign
        {"digit": 7},
        # State 7: Exp-Integer (Valid End)
        {"digit": 7}
    ]

    current_state = 0
```

```

for char in s:
    # Determine the category of the current character
    if char.isdigit():
        group = "digit"
    elif char in "+-":
        group = "sign"
    elif char in "eE":
        group = "exp"
    elif char == ".":
        group = "dot"
    else:
        return False # Invalid character

    # Check if a transition exists for the current state and character group
    if group not in dfa[current_state]:
        return False

    current_state = dfa[current_state][group]

# Valid ending states: 1 (Integer), 4 (Decimal), 7 (Exp-Integer)
return current_state in [1, 4, 7]

```

JavaScript

```

/**
 * L5 Approach: DFA Implementation
 * This avoids "if-else hell" and makes the code maintainable and extensible.
 */
var isNumber = function(s) {
    const dfa = [
        { "digit": 1, "sign": 2, "dot": 3 }, // 0: Start
        { "digit": 1, "dot": 4, "exp": 5 }, // 1: Integer (Valid)
        { "digit": 1, "dot": 3 }, // 2: Sign
        { "digit": 4 }, // 3: Dot (no leading)
        { "digit": 4, "exp": 5 }, // 4: Decimal (Valid)
        { "digit": 7, "sign": 6 }, // 5: Exponent
        { "digit": 7 }, // 6: Exp-Sign
        { "digit": 7 } // 7: Exp-Integer (Valid)
    ];

    let currentState = 0;

    for (let char of s) {
        let group;
        if (char >= '0' && char <= '9') group = "digit";

```

```

else if (char === '+' || char === '-') group = "sign";
else if (char === 'e' || char === 'E') group = "exp";
else if (char === '.') group = "dot";
else return false;

// Transition to next state
if (dfa[currentState][group] === undefined) {
    return false;
}
currentState = dfa[currentState][group];
}

// Define which states are "Winning States"
const validEndStates = new Set([1, 4, 7]);
return validEndStates.has(currentState);
};

```

Note 1: Terms and Techniques

DFA (Deterministic Finite Automaton): * **What it is:** A mathematical model of computation. It consists of a finite set of states and transitions. * **Why it helps:** In string parsing, it ensures that you don't have to keep track of a dozen boolean flags (like `hasDot`, `hasExp`, `hasDigit`). The *state itself* represents exactly what has been seen so far. * **How it applies:** It turns a complex logic problem into a simple table-lookup problem.

Note 2: Real-World & Interview Variations

Variation 1: "Implement `atof` (ASCII to Float) or `parseInt`" (Google/Bloomberg) * **The Problem:** Instead of just returning `true/false`, you must return the actual numerical value. * **Solution:** Use the DFA. While in states 1, 4, and 7, accumulate the value: `total = total * 10 + digit`. If after a dot, use a divisor: `total += digit / divisor`. This is how compilers actually parse your code.

Variation 2: "Log File Parser" (Meta/Google) * **The Problem:** You have a 100GB log file and need to extract all valid timestamps or IDs that follow a specific, complex format. * **Solution:** This is the real-world version of "Valid Number." You would write a DFA (or a Regex, which compiles into a DFA) to scan the stream efficiently without loading the whole file into memory.

Variation 3: "Expression Evaluator / Calculator" (Bloomberg) * **The Problem:** Validate and solve $(3 + 5) * 2$. * **Solution:** This is a step up from a DFA. You need a **PDA (Pushdown Automaton)** which uses a **Stack**

to handle nested parentheses. The “Valid Number” logic becomes just one small part of the “Tokenizing” phase of the calculator.

1249. Minimum Remove to Make Valid Parentheses

This is a classic problem that tests an engineer’s ability to handle **state management** and **two-pass filtering**. At a Senior (L5) or Staff (L6) level, the focus is on achieving the most efficient time complexity while ensuring the code is readable and handles all edge cases (like strings with no parentheses or strings with only closing ones).

1. Problem Explanation

The Goal: You are given a string containing lowercase letters and parentheses (and). You need to remove the **minimum** number of parentheses so that the remaining string is “valid.”

What makes a string valid? 1. Every opening parenthesis (must have a matching closing parenthesis). 2. The closing parenthesis) must come **after** the opening one. 3. Empty strings or strings with only letters are naturally valid.

The Strategy: To make the string valid with “minimum removals,” we just need to identify which parentheses are “orphans” (they don’t have a partner) and delete them.

- **Case A:** A closing) appears when there is no (before it to close. (Delete it!)
- **Case B:** An opening (appears but the string ends before a) can close it. (Delete it!)

2. Solution Explanation

We use a **Stack** to keep track of the indices of opening parentheses. If we see a closing parenthesis, we check the stack. If the stack is empty, that closing bracket is an orphan. If the stack has something, we “pair” them up and move on.

Step-by-Step Visualization Input: `s = "lee(t(c)o)de"`

```
=====
INITIAL STATE
=====
```

String: l e e (t (c) o) d e)
Index: 0 1 2 3 4 5 6 7 8 9 0 1 2
Stack: [] (Empty)
To_Remove: {} (Set of indices to delete)

=====

PASS 1: Identifying Orphans

=====

Indices 0-2: "lee" (Ignore letters)

Index 3: "("
Action: Push index 3 to stack.
Stack: [3]

Index 4: "t" (Ignore)

Index 5: "("
Action: Push index 5 to stack.
Stack: [3, 5]

Index 6: "c" (Ignore)

Index 7: ")"
Action: Stack is NOT empty. This ")" pairs with index 5.
Pop index 5 from stack.
Stack: [3]

Index 8: "o" (Ignore)

Index 9: ")"
Action: Stack is NOT empty. This ")" pairs with index 3.
Pop index 3 from stack.
Stack: []

Index 10-11: "de" (Ignore)

Index 12: ")"
Action: Stack is EMPTY. This ")" has no "(" to pair with.
It is an ORPHAN. Add index 12 to To_Remove.
Stack: []
To_Remove: {12}

=====

AFTER PASS 1: Check Stack

=====

If the stack still has indices (e.g., [1, 4]), those are "(" that never got closed. We must add those indices to To_Remove.

In our example, Stack is empty.
Final To_Remove: {12}

=====

PASS 2: Reconstruct String

=====

We build a new string by looping through the original and skipping any index found in To_Remove.

Result: "lee(t(c)o)de" (The ")" at index 12 was skipped)

3. Time and Space Complexity Analysis

Let N be the length of the string.

TIME COMPLEXITY DERIVATION: Order of N

-
1. Iterating through string once:
Each character is visited once. ---> Order of N
Stack operations (push/pop) are O(1).
 2. Cleanup (adding remaining stack to set):
In worst case, string is all "(((" ---> Order of N
 3. Building the final string:
We loop through string one last time. ---> Order of N

Total Time = N + N + N = 3N

In Big O notation, constants are dropped: Order of N

SPACE COMPLEXITY DERIVATION: Order of N

-
1. The Stack:
Worst case (all "("), it stores N indices. ---> Order of N
 2. The To_Remove Set:
Worst case, stores N indices. ---> Order of N
 3. Result storage (String/Array):
Stores N characters. ---> Order of N

Total Space = $N + N + N = 3N$
In Big O notation: Order of N

4. Solution Code

Python Implementation

```
def minRemoveToMakeValid(s):
    # We use a list for the stack to store indices of '('
    stack = []
    # indices_to_remove stores indices of orphan parentheses
    indices_to_remove = set()

    # PASS 1: Find all orphans
    for i, char in enumerate(s):
        if char not in "()":
            continue
        if char == "(":
            # Record the index of the opening bracket
            stack.append(i)
        elif char == ")":
            if not stack:
                # No matching '(' found, this ')' is an orphan
                indices_to_remove.add(i)
            else:
                # Matching '(' found, pair it and remove from stack
                stack.pop()

    # Add any remaining '(' in stack to removal set (they never closed)
    # This is done by converting the list to a set and merging
    indices_to_remove = indices_to_remove.union(set(stack))

    # PASS 2: Build the resulting string
    result = []
    for i, char in enumerate(s):
        if i not in indices_to_remove:
            result.append(char)

    return "".join(result)
```

JavaScript Implementation

```
/**
 * @param {string} s
 * @return {string}
```

```

*/
function minRemoveToMakeValid(s) {
  const stack = [];
  const indicesToRemove = new Set();

  // PASS 1: Identify indices of unbalanced parentheses
  for (let i = 0; i < s.length; i++) {
    if (s[i] === "(") {
      stack.push(i);
    } else if (s[i] === ")") {
      if (stack.length === 0) {
        // Orphan closing bracket
        indicesToRemove.add(i);
      } else {
        // Valid pair found
        stack.pop();
      }
    }
  }

  // Any remaining items in stack are orphan opening brackets
  while (stack.length > 0) {
    indicesToRemove.add(stack.pop());
  }

  // PASS 2: Reconstruct the string
  let result = "";
  for (let i = 0; i < s.length; i++) {
    if (!indicesToRemove.has(i)) {
      result += s[i];
    }
  }

  return result;
}

```

Note 1: Terminology & Techniques

- **Stack (LIFO):** “Last-In, First-Out.” This is the perfect tool for parentheses because the *last* opening bracket seen is always the *first* one that needs to be closed by a matching bracket.
- **Two-Pass Filtering:** This refers to looking at the data twice. The first pass gathers information (which indices are bad), and the second pass performs the action (building the clean string).

- **Set Lookup:** Using a **Set** for `indices_to_remove` is crucial. Checking if an index exists in a **Set** is **Order of 1**, whereas checking in a **List** is **Order of N**. This keeps the overall time complexity at **Order of N**.
-

Note 2: Real World / Interview Variations

1. **Variation: Check if valid (Google/Meta)** * *The Twist:* Instead of removing, just return true/false if it's valid. * *Solution:* Use a simple counter. Increment for (, decrement for). If the counter ever goes below 0, return False. If it's 0 at the end, return True.
2. **Variation: Multiple types of brackets [], {}, () (Bloomberg)** * *The Twist:* You must ensure ([)] is invalid. * *Solution:* You **must** use a stack of characters (not just a counter). When you see], the top of the stack must be [. If it's anything else, it's invalid.
3. **Variation: Memory Constraint / Zero Space (L6 System Design Question)** * *The Twist:* You have a 100GB string and very little RAM. * *Solution:* You cannot use a **Set** for indices. You perform one pass from **left-to-right** to remove extra). Then, you take that result and perform a second pass from **right-to-left** to remove extra (. This uses two counters and zero extra space for a "Set."

Valid Word Abbreviation

Solving "Valid Word Abbreviation" at an L5/L6 level requires more than just making the code work. A Senior or Staff engineer focuses on **edge-case robustness**, **clean pointer arithmetic**, and **preventing integer overflow or malformed inputs** (like leading zeros).

1. Problem Explanation

The Goal: You are given a string `word` and an abbreviation `abbr`. You need to determine if `abbr` is a valid way to shorten `word`.

The Rules of Abbreviation: 1. A number in `abbr` replaces that exact count of characters in `word`. 2. Characters in `abbr` must match characters in `word` exactly. 3. **The "L5" Trap:** A number in the abbreviation cannot have a **leading zero** (e.g., "01" is invalid). 4. The final position of both pointers must reach the exact end of both strings simultaneously.

Example: Word: `internationalization` Abbr: `i12iz4n`

- `i` matches `i`.
- `12` skips 12 letters.

- iz matches iz.
- 4 skips 4 letters.
- n matches n.

2. Solution Explanation

The most efficient way to solve this is using the **Two-Pointer Technique**. We maintain one pointer for the **word** (**p1**) and one for the **abbr** (**p2**).

The Logic Flow: * If **p2** points to a letter: It must match **word[p1]**. * If **p2** points to a digit: We must parse the *entire* number (it could be multiple digits like “12”), check for leading zeros, and then jump **p1** forward by that amount.

ASCII Visualization: A Successful Match Word: apple, Abbr: a2e

STEP 1: INITIAL STATE

```
word: a p p l e      abbr: a 2 e
      ^              ^
      p1             p2
```

- abbr[p2] is 'a' (a letter).
- word[p1] is 'a'. Match!
- Move both: p1++, p2++

STEP 2: HANDLING THE NUMBER

```
word: a p p l e      abbr: a 2 e
      ^              ^
      p1             p2
```

- abbr[p2] is '2' (a digit).
- First digit check: Is it '0'? No. Valid.
- Parse the number: num = 2.
- Jump word pointer: p1 = p1 + 2

STEP 3: POST-JUMP CHECK

```
word: a p p l e      abbr: a 2 e
      ^              ^
      p1             p2 (incremented past '2')
```

- p1 jumped over 'p' and 'p', now points to 'l'.
- abbr[p2] is 'e' (a letter).
- word[p1] is 'l'. (Wait, 'l' is at index 3, but the letter 'e' is index 4).

Wait, let's re-trace the jump correctly:

Index 0: a (matched)
Index 1: p (skipped)
Index 2: p (skipped)
Index 3: l <-- p1 should be here.
Index 4: e

- abbr[p2] is 'e'.
- word[p1] is 'l'.
- RESULT: 'l' != 'e'. Return FALSE.

ASCII Visualization: The Leading Zero Trap Word: apple, Abbr: a01e

STEP 1: Match 'a'
p1 moves to 1, p2 moves to 1.

STEP 2: Check digit at abbr[p2]
abbr[1] is '0'.
- RULE CHECK: Does the number start with 0? YES.
- RESULT: Immediate FALSE.

3. Time and Space Complexity Analysis

Let N be the length of the word and M be the length of the abbreviation.

TIME COMPLEXITY: $O(N + M)$

We iterate through the abbreviation string (M) exactly once.
Inside the loop, we might jump the word pointer (p1).
Even with jumps, p1 never visits a character in the word twice.
Therefore, the total operations are proportional to the
sum of the lengths of both strings.

Total Time = $O(N + M)$

SPACE COMPLEXITY: $O(1)$

We only store:

1. Two pointer variables (p1, p2)
2. A temporary integer for the current parsed number.

No extra data structures (like HashMaps or Arrays) are
created that grow with the input size.

Total Space = $O(1)$

4. Solution Code

Python Solution

```
def validWordAbbreviation(word, abbr):
    p1 = 0 # Pointer for 'word'
    p2 = 0 # Pointer for 'abbr'

    while p1 < len(word) and p2 < len(abbr):
        # Case 1: Current abbr char is a digit
        if abbr[p2].isdigit():
            # Leading zero check: The first digit of a number cannot be '0'
            if abbr[p2] == '0':
                return False

            # Parse the full number (could be multiple digits)
            num = 0
            while p2 < len(abbr) and abbr[p2].isdigit():
                num = num * 10 + int(abbr[p2])
                p2 += 1

            # Move the word pointer forward by the parsed number
            p1 += num

        # Case 2: Current abbr char is a letter
        else:
            if word[p1] != abbr[p2]:
                return False
            p1 += 1
            p2 += 1

    # Success only if BOTH pointers reached the end of their strings
    return p1 == len(word) and p2 == len(abbr)
```

JavaScript Solution

```
/**
 * @param {string} word
 * @param {string} abbr
 * @return {boolean}
 */
function validWordAbbreviation(word, abbr) {
    let p1 = 0; // Pointer for word
```

```

let p2 = 0; // Pointer for abbr

while (p1 < word.length && p2 < abbr.length) {
    const charAbbr = abbr[p2];

    // Check if char is a digit (0-9)
    if (charAbbr >= '0' && charAbbr <= '9') {
        // Leading zero check
        if (charAbbr === '0') return false;

        // Parse full number
        let num = 0;
        while (p2 < abbr.length && abbr[p2] >= '0' && abbr[p2] <= '9') {
            num = num * 10 + Number(abbr[p2]);
            p2++;
        }

        // Jump the word pointer
        p1 += num;
    } else {
        // Character matching case
        if (word[p1] !== abbr[p2]) {
            return false;
        }
        p1++;
        p2++;
    }
}

// Final check: did we consume both strings entirely?
return p1 === word.length && p2 === abbr.length;
}

```

Note 1: Terminology and Techniques

- **Two-Pointer Technique:** A strategy where two indices are used to process linear data structures (like strings or arrays) simultaneously. It is usually chosen to avoid nested loops, keeping the time complexity at $O(N)$.
 - **Integer Parsing in a Loop:** Notice how we don't use a library like `parseInt`. By doing `num = num * 10 + digit`, we handle multi-digit numbers manually. This is a common pattern in competitive programming and technical interviews for high-performance string processing.
-

Note 2: Real-World / Interview Variations

1. Variation: Internationalization (i18n) (Google / Bloomberg) * **The Context:** Engineers often abbreviate long words by keeping the first/last letters and counting the middle (e.g., `i18n` for `internationalization`). * **The Problem:** Given a list of words, find all words that match a specific abbreviation. * **The L5 Solution:** You would iterate through the list and apply the `validWordAbbreviation` logic to each word. If the list is massive (millions of words), you would optimize by first filtering words that don't match the *effective length* of the abbreviation (effective length = count of literal letters + the value of the numbers).

2. Variation: Word Abbreviation Conflict / Shortest Unique Abbr (Meta) * **The Context:** If you have `["apple", "apply"]`, both could be `a3e` or `a3y`. But what if you want the shortest *unique* abbreviation? * **The L5 Solution:** This involves using a **Trie (Prefix Tree)**. You store all words in a Trie. For each word, you find the first position where its prefix becomes unique compared to others. The abbreviation would then be: `unique_prefix + count_of_remaining_minus_one + last_letter`.

3. Variation: Data Compression Decompression (Bloomberg) * **The Context:** This problem is a simplified version of Run-Length Encoding (RLE). * **The Logic:** Bloomberg often asks candidates to “decompress” a string like `a3b2` into `aaabb`. The pointer logic is identical, but instead of just jumping a pointer, you are appending characters to a result string. The L5 engineer would discuss the cost of string concatenation versus using a `StringBuilder` or `Array.join()` in memory-constrained environments.

680. Valid Palindrome II

Here is an end-to-end walkthrough for “Valid Palindrome II” from the perspective of an L5/L6 (Senior/Staff) engineer.

A high-level engineer recognizes that while a standard palindrome check is a “warm-up,” the “II” version introduces a **branching decision**. The core skill here is identifying that once you find a mismatch, you have exactly two paths to validate, and if either works, the whole string is valid.

1. Problem Explanation

The Core Goal: Given a string, determine if it can become a palindrome after deleting **at most one** character.

What is a Palindrome? A word that reads the same backward as forward (e.g., “aba”, “racecar”).

The Twist: Usually, you check a palindrome by comparing the start and the end. If they match, you move inward. If they don't match, it's not a palindrome. In this problem, if they don't match, you get **one "Lifeline"**. You can "skip" one character and see if the rest of the string is a perfect palindrome.

2. Solution Explanation

We use the **Two-Pointer Technique**. We place one pointer (L) at the start and one pointer (R) at the end.

The "Greedy" Decision We move L forward and R backward as long as `string[L] == string[R]`. When we hit a mismatch (`string[L] != string[R]`), we have two choices to use our one-time skip: 1. **Skip the Left character:** Check if the substring from L + 1 to R is a palindrome. 2. **Skip the Right character:** Check if the substring from L to R - 1 is a palindrome.

If **either** of those remaining substrings is a palindrome, the original string is valid.

Visualization: Case 1 - Success with Skip Input: "abca"

STEP 1: Initial Check

```

-----
a b c a
^     ^
L     R
s[L] ('a') == s[R] ('a') -> MATCH.
Action: Move pointers inward.
```

STEP 2: The Mismatch

```

-----
a b c a
  ^ ^
    L R
s[L] ('b') != s[R] ('c') -> MISMATCH!
```

Action: We have used 0 deletes. We must try our two branches.

Branch A (Skip Left):

Check if "c" (the part between L+1 and R) is a palindrome.

```

  c
  ^
L,R -> YES.
```

Branch B (Skip Right):

Check if "b" (the part between L and R-1) is a palindrome.

b
^

L,R -> YES.

Result: Since at least one branch worked, return TRUE.

Visualization: Case 2 - Failure Input: "abcde"

STEP 1: The Mismatch

a b c d e
^ ^
L R
s[L] ('a') != s[R] ('e') -> MISMATCH!

Branch A (Skip Left 'a'):
Check if "bcde" is a palindrome.
b c d e
^ ^ -> 'b' != 'e'. NO.

Branch B (Skip Right 'e'):
Check if "abcd" is a palindrome.
a b c d
^ ^ -> 'a' != 'd'. NO.

Result: Both branches failed. return FALSE.

3. Time and Space Complexity Analysis

Let N be the number of characters in the string.

TIME COMPLEXITY DERIVATION: $O(N)$

-
1. Main Loop:
In the worst case, we compare characters until we hit the middle or a mismatch. This touches at most N chars. -> $O(N)$
 2. Branching:
If we hit a mismatch, we call a helper function.
This helper checks a substring. The total characters remaining to check is less than N. -> $O(N)$

Total Time = $O(N) + O(N)$
In Big O, we drop constants.

Final Time Complexity: $O(N)$

SPACE COMPLEXITY DERIVATION: $O(1)$

1. Pointers:

We only store two integer variables (L and R). $\rightarrow O(1)$

2. Auxiliary Space:

We do not create new strings (if we use slicing carefully or just pointers in the helper).

We are not using recursion that goes deep; we only branch once. $\rightarrow O(1)$

Final Space Complexity: $O(1)$

4. Solution Code

Python Solution

```
def validPalindrome(s):
    # This helper function performs a standard, strict palindrome check
    def is_palindrome_range(left, right):
        while left < right:
            if s[left] != s[right]:
                return False
            left += 1
            right -= 1
        return True

    l, r = 0, len(s) - 1

    while l < r:
        if s[l] != s[r]:
            # If mismatch, try skipping s[l] OR skipping s[r]
            # Choice 1: Check substring from l+1 to r
            # Choice 2: Check substring from l to r-1
            return is_palindrome_range(l + 1, r) or is_palindrome_range(l, r - 1)

        l += 1
        r -= 1

    return True
```

JavaScript Solution

```

/**
 * @param {string} s
 * @return {boolean}
 */
var validPalindrome = function(s) {
    // Helper function for strict palindrome check
    const isPalindromeRange = (left, right) => {
        while (left < right) {
            if (s[left] !== s[right]) {
                return false;
            }
            left++;
            right--;
        }
        return true;
    };

    let l = 0;
    let r = s.length - 1;

    while (l < r) {
        if (s[l] !== s[r]) {
            // Mismatch found! Try both skip options.
            // If either returns true, the string is valid.
            return isPalindromeRange(l + 1, r) || isPalindromeRange(l, r - 1);
        }
        l++;
        r--;
    }

    return true;
};

```

Note 1: Terminology and Techniques

- **Two-Pointer Technique:** A strategy where two indices (usually `left` and `right`) move toward each other or in the same direction to solve a problem with linear time complexity.
 - **Greedy Approach:** In this problem, we are “greedy” because we match as many characters as possible before using our one “delete” credit. We only use the credit when we absolutely have to (at the first mismatch).
-

Note 2: Real World / Interview Variations

1. Variation: Valid Palindrome III (K-Deletions) * The Twist: What if you can delete up to K characters instead of just 1? * **How to solve (Meta/Google):** The two-pointer branching approach becomes slow because it grows exponentially. Instead, use **Dynamic Programming (DP)**. You find the “Longest Palindromic Subsequence” (LPS). If `Length_of_String - LPS <= K`, it’s valid.

2. Variation: Stream Data Palindrome (Bloomberg) * The Twist: You are receiving characters one by one in a stream. At any point, can you tell if the characters received so far could form a palindrome with 1 deletion? * **How to solve:** This requires a more complex state-tracking approach or a rolling hash, though usually, for streams, interviewers focus on the “Is it a palindrome permutation” (Can the letters be rearranged?), which is solved using a character-frequency bitmask.

3. Variation: Clean Data/Regex (General Interview) * The Twist: The string contains spaces, punctuation, and mixed casing. * **How to solve:** A Senior engineer always asks: “Should I ignore non-alphanumeric characters?” You should pre-process the string (or skip invalid chars on the fly) using something like `char.isalnum()` in Python before applying the logic above.

415. Add Strings

This is an end-to-end, L5/L6 level walkthrough for “Add Strings.”

At a top tech company, an L5 (Senior) or L6 (Staff) engineer doesn’t just see this as a math problem. They see it as a **Big Integer** problem. They know that in production systems, numbers can often exceed the 64-bit limits of standard data types (like `int` or `long`). This problem tests your ability to handle overflow by manually implementing the “Elementary School Addition” algorithm using strings and pointers.

1. Problem Explanation

The Goal: You are given two non-negative integers, `num1` and `num2`, represented as strings. You need to return their sum as a string.

The Constraints (The “Why” it’s hard): 1. You **cannot** use any built-in BigInteger library. 2. You **cannot** convert the inputs directly to integers (e.g., `int(num1)` in Python or `Number(num1)` in JS) because the strings could be thousands of characters long, which would cause an overflow error in most languages.

The Core Concept: We must simulate how we add numbers on paper: * Start from the **rightmost** digit (the ones place). * Add the digits together. * If the

sum is 10 or more, we “carry” the 1 over to the next column. * Move left and repeat.

2. Solution Explanation

We use a “Two-Pointer” approach combined with a “Carry” variable. We place one pointer at the end of each string and move backward.

The “Carry” Logic If we add 8 and 7, we get 15. * The **Result Digit** for the current column is $15 \% 10 = 5$. * The **Carry** for the next column is $15 / 10 = 1$ (integer division).

Let’s visualize adding “456” and “77” step-by-step.

```
=====
INITIAL STATE
=====
num1: "4 5 6"      (Pointer p1 at index 2)
num2: " 7 7"       (Pointer p2 at index 1)
Carry: 0
Result: []

=====
STEP 1: Ones Place
=====
p1 points to '6'
p2 points to '7'

Sum = 6 + 7 + Carry(0) = 13
New Digit = 13 % 10 = 3
New Carry = 13 // 10 = 1

Result: [3]
Pointers: Move p1 and p2 left.

Diagram:
  4 5 [6]
+   7 [7]
-----
      (1) <-- Carry
      3  <-- Current Digit

=====
STEP 2: Tens Place
=====
```

p1 points to '5'
p2 points to '7'

Sum = 5 + 7 + Carry(1) = 13
New Digit = 13 % 10 = 3
New Carry = 13 // 10 = 1

Result: [3, 3]
Pointers: Move p1 and p2 left.

Diagram:

```
    4 [5] 6
+   [7] 7
-----
    (1)    <-- Carry
    3 3
```

=====

STEP 3: Hundreds Place

=====

p1 points to '4'
p2 is out of bounds (effectively 0)

Sum = 4 + 0 + Carry(1) = 5
New Digit = 5 % 10 = 5
New Carry = 5 // 10 = 0

Result: [3, 3, 5]
Pointers: Move p1 left.

Diagram:

```
    [4] 5 6
+     7 7
-----
    (0)    <-- Carry
    5 3 3
```

=====

FINAL STEP: Join and Reverse

=====

Our result list is [3, 3, 5].
When we reverse it and join, we get "533".

3. Time and Space Complexity Analysis

Let N be the length of num1 and M be the length of num2.

TIME COMPLEXITY DERIVATION: $O(\max(N, M))$

We iterate through the strings from right to left.
The number of steps is determined by whichever string is longer.

Loop iterations = $\max(N, M)$

Inside each loop:

- Accessing char by index --> $O(1)$
- Character to Int conversion -> $O(1)$
- Simple math ($\%$, $//$) --> $O(1)$
- Array push/append --> $O(1)$

Total Time = $\max(N, M) * O(1) = O(\max(N, M))$

SPACE COMPLEXITY DERIVATION: $O(\max(N, M))$

We store the result in a list or string builder.
The length of the sum of two numbers can be at most
 $\max(N, M) + 1$ (in case there is a final carry).

Example: "99" + "1" = "100" (Length $2 + 1 = 3$)

Since we store roughly $\max(N, M)$ characters:
Overall Space Complexity = $O(\max(N, M))$

4. Solution Code

The following implementation is optimized for readability and avoids unnecessary string concatenations in the loop (which would be $O(N^2)$ in some languages).

Python Solution

```
def addStrings(num1, num2):  
    # Convert result to list for  $O(1)$  append time  
    res = []  
    carry = 0  
  
    # Set pointers to the end of each string  
    p1 = len(num1) - 1  
    p2 = len(num2) - 1
```

```

# Continue as long as there are digits to process or a carry exists
while p1 >= 0 or p2 >= 0 or carry:
    # Get the digit from num1, or 0 if pointer is out of bounds
    digit1 = ord(num1[p1]) - ord('0') if p1 >= 0 else 0
    # Get the digit from num2, or 0 if pointer is out of bounds
    digit2 = ord(num2[p2]) - ord('0') if p2 >= 0 else 0

    # Calculate current sum and the new carry
    current_sum = digit1 + digit2 + carry
    carry = current_sum // 10

    # Append the last digit of the sum to our result
    res.append(str(current_sum % 10))

    # Move pointers left
    p1 -= 1
    p2 -= 1

# The result was built backwards (ones place first), so reverse it
return "".join(res[::-1])

```

JavaScript Solution

```

/**
 * @param {string} num1
 * @param {string} num2
 * @return {string}
 */
function addStrings(num1, num2) {
    let res = [];
    let carry = 0;

    let p1 = num1.length - 1;
    let p2 = num2.length - 1;

    while (p1 >= 0 || p2 >= 0 || carry > 0) {
        // Character to digit conversion using ASCII math
        const digit1 = p1 >= 0 ? num1.charCodeAt(p1) - 48 : 0;
        const digit2 = p2 >= 0 ? num2.charCodeAt(p2) - 48 : 0;

        const sum = digit1 + digit2 + carry;
        carry = Math.floor(sum / 10);

        // Push the digit to the result array
        res.push(sum % 10);
    }
}

```

```

        p1--;
        p2--;
    }

    // Reverse and join to form the final string
    return res.reverse().join('');
}

```

Note 1: Terminology & Techniques

- **Pointers:** Variables (like `p1` and `p2`) that track the index we are currently looking at. This is more efficient than “slicing” or “popping” characters from a string.
 - **ASCII Math:** In computers, the character ‘0’ has a specific numeric value (48). By subtracting `ord('0')` or 48, we convert the character ‘5’ into the actual integer 5 without using high-level library functions.
 - **String Building:** In Python and JS, strings are “immutable” (cannot be changed). Doing `string = char + string` inside a loop creates a brand new string every time, which is slow. Using an array and then `join()` at the end is the industry standard for performance.
-

Note 2: Real World / Interview Variations

1. **Variation: Add Binary (Meta)** * *The Twist:* The input strings are binary (e.g., “1010” + “1011”). * *Solution:* Use the exact same logic, but change the base. Instead of `% 10` and `// 10`, use `% 2` and `// 2`.
2. **Variation: Multiply Strings (Google)** * *The Twist:* Multiply the two strings instead of adding. * *Solution:* This is significantly harder. You simulate “Long Multiplication.” You create an array of size `len(num1) + len(num2)` and add the result of multiplying each digit from `num1` with each digit from `num2` into the corresponding position in the array.
3. **Variation: Linked List Sum (Bloomberg)** * *The Twist:* The numbers are stored in Linked Lists (e.g., 2 -> 4 -> 3 represents 342). * *Solution:* You traverse the lists simultaneously. The “Next” pointer in the list acts as your loop mechanism. You still maintain a `carry` variable across nodes. This tests your understanding of data structures plus the math logic.

791. Custom Sort String

This is a comprehensive, Staff-level (L5/L6) walkthrough for solving “Custom Sort String.”

A Senior Engineer at Google or Meta looks at this and immediately identifies that the “custom order” is just a set of constraints. They don’t just reach for a `sort()` function—which usually takes $O(N \log N)$ time—but instead look for a linear $O(N)$ solution using a frequency map, because the set of possible characters (a-z) is small and fixed.

1. Problem Explanation

The Goal: You are given two strings: `order` and `s`. * **order:** Defines a new “alphabetical order.” For example, if `order = "cba"`, then ‘c’ comes before ‘b’, and ‘b’ comes before ‘a’. * **s:** The string you need to rearrange.

You must permute (rearrange) `s` so that it follows the sequence defined in `order`. Characters in `s` that are not in `order` can be placed anywhere at the end.

The Intuition: Think of `order` as a set of labeled buckets lined up in a specific sequence. You take all the characters from `s`, and if a character has a matching bucket, you drop it in. If it doesn’t have a bucket, you put it in a “miscellaneous” pile. Finally, you walk past the buckets in order and pick up the characters to form your new string.

2. Solution Explanation

To achieve the best possible performance (Linear Time), we use a **Frequency Counting** strategy.

Step 1: Count Frequencies First, we count how many times each character appears in `s`.

Step 2: Build the Result based on order We iterate through each character in the `order` string. For each character, we check if it existed in `s`. If it did, we append all instances of that character to our result.

Step 3: Handle the Remainder Any characters left over in our frequency map (those that weren’t mentioned in `order`) are appended to the very end.

Visualization Walkthrough: `order = "cba"`, `s = "abcd"`

```
=====
STEP 1: COUNT FREQUENCIES OF 's'
=====
String s: "a b c d"

Frequency Map:
{
```

```

    'a': 1,
    'b': 1,
    'c': 1,
    'd': 1
}

```

```

=====
STEP 2: ITERATE THROUGH 'order' TO BUILD RESULT
=====

```

Order: "c b a"

1. Current char in Order: 'c'
 - Does 'c' exist in Map? Yes (count = 1).
 - Add 'c' to Result.
 - Result: "c"
 - Update Map: {'a':1, 'b':1, 'c':0, 'd':1}
2. Current char in Order: 'b'
 - Does 'b' exist in Map? Yes (count = 1).
 - Add 'b' to Result.
 - Result: "cb"
 - Update Map: {'a':1, 'b':0, 'c':0, 'd':1}
3. Current char in Order: 'a'
 - Does 'a' exist in Map? Yes (count = 1).
 - Add 'a' to Result.
 - Result: "cba"
 - Update Map: {'a':0, 'b':0, 'c':0, 'd':1}

```

=====
STEP 3: APPEND REMAINING CHARACTERS
=====

```

Check Map for any remaining counts > 0:

- 'd' has count 1.
- Add 'd' to Result.

FINAL RESULT: "cbad"

```

=====

```

3. Time and Space Complexity Analysis

Let N be the length of string `s` and M be the length of string `order`.

TIME COMPLEXITY DERIVATION: $O(N + M)$

```

-----

```

1. Counting frequencies in 's':
We visit each character in 's' once. ----> $O(N)$
2. Iterating through 'order':
We visit each character in 'order' once. ----> $O(M)$
(Map lookups and string appends are $O(1)$ avg)
3. Final iteration over Map:
In the worst case, we check all 26 lowercase English letters. ----> $O(26)$

Total Time = $O(N + M + 26)$

Since 26 is a constant, we simplify to:

Total Time = $O(N + M)$

SPACE COMPLEXITY DERIVATION: $O(N)$ or $O(1)$

1. Frequency Map:
The map stores at most 26 keys (a-z).
Since this does not grow with input size N,
it is considered Constant Space. ----> $O(1)$
2. Output String/Array:
We must store the result, which has N
characters. ----> $O(N)$

Note: Usually, in interviews, we don't count the output space unless specifically asked.

If we ignore output, Space = $O(1)$.

If we count output, Space = $O(N)$.

4. Solution Code

Python Solution (Optimal Frequency Map)

```
from collections import Counter

def customSortString(order, s):
    # Step 1: Count occurrences of each character in s
    # Counter({'a': 1, 'b': 1, 'c': 1, 'd': 1})
    counts = Counter(s)
    result = []

    # Step 2: Build result based on 'order'
```

```

for char in order:
    if char in counts:
        # Add the character as many times as it appeared in s
        result.append(char * counts[char])
        # Delete so we know it's handled
        del counts[char]

# Step 3: Handle leftover characters not in 'order'
for char, count in counts.items():
    result.append(char * count)

return "".join(result)

```

JavaScript Solution (Optimal Frequency Map)

```

/**
 * @param {string} order
 * @param {string} s
 * @return {string}
 */
var customSortString = function(order, s) {
    // Step 1: Count character frequencies using a Map or Object
    const counts = {};
    for (const char of s) {
        counts[char] = (counts[char] || 0) + 1;
    }

    let result = "";

    // Step 2: Iterate through the custom order
    for (const char of order) {
        if (counts[char]) {
            // repeat() creates a string with N copies of the character
            result += char.repeat(counts[char]);
            // Clear the count once added
            counts[char] = 0;
        }
    }

    // Step 3: Append anything left in the counts object
    for (const char in counts) {
        if (counts[char] > 0) {
            result += char.repeat(counts[char]);
        }
    }
}

```

```
    return result;
};
```

Note 1: Terminology and Techniques

- **Frequency Map (Counting Sort variant):** Instead of comparing elements against each other (Comparison Sort), we count how many of each item exist. This is the foundation of Counting Sort. It is extremely powerful when the “range” of values (a-z) is much smaller than the number of items to sort.
 - **Bucket Sort Intuition:** This problem is a variation of Bucket Sort where the “buckets” are predefined by the `order` string.
-

Note 2: Real-World / Interview Variations

1. Variation: Custom Sort with Multiple Criteria (Bloomberg) * *The Twist:* Sort a list of items based on a custom priority list, but if priorities are equal, sort alphabetically. * *How to solve:* Instead of frequency maps, you would use a standard Sort function with a **Custom Comparator**. You’d store the order in a Map {char: index} for O(1) priority lookups.

2. Variation: Log File Reordering (Amazon/Google) * *The Twist:* You have logs. Some start with letters, some with numbers. Letter-logs must be sorted lexicographically; digit-logs must remain in their original relative order. * *How to solve:* This requires a **Stable Sort** or a two-pass approach (filter and sort letters, then append digits).

3. Variation: Large Character Sets (Unicode) * *The Twist:* What if the input isn’t just a-z, but any Unicode character? * *How to solve:* The logic remains the same, but the “Space = O(1)” argument becomes weaker because the number of possible characters is now 1 million+. You would still use a Hash Map rather than a fixed-size array to save space.

Group Shifted Strings

To solve “Group Shifted Strings,” a Senior (L5) or Staff (L6) engineer looks for the **invariant property**. In system design and high-level coding, the goal is to find a “signature” that remains the same regardless of how the data is shifted.

1. Problem Explanation

The Core Goal: You are given a list of strings. You need to group strings that belong to the same “shifting sequence.”

What is a Shifting Sequence? Imagine the alphabet is a circular loop (after 'z' comes 'a'). You can shift a string by moving every letter forward by the same amount. * "abc" shifted by 1 becomes "bcd". * "abc" shifted by 2 becomes "cde". * "abc" shifted by 25 becomes "zab".

All of these ("abc", "bcd", "cde", "zab") belong to the same group.

The Challenge: How do we know "abc" and "zab" are the same without testing all 26 possible shifts for every single word? We need a **Canonical Form**—a universal way to represent a shift pattern.

2. Solution Explanation

The "Why" of the solution: The absolute letters change, but the **relative distance** between the letters stays the same.

The Signature Technique: 1. Take a word like "acf". 2. Calculate the distance between adjacent characters: * 'c' - 'a' = 2 * 'f' - 'c' = 3 * Signature: (2, 3) 3. Take a shifted version "bdg": * 'd' - 'b' = 2 * 'g' - 'd' = 3 * Signature: (2, 3) 4. **Wait! What about the circular wrap?** If we have "za": * 'a' - 'z' = -25. * Since it's a circle of 26 letters, we add 26 to any negative result: -25 + 26 = 1. * Signature: (1)

The Strategy: Use a Hash Map where the **Key** is the tuple of distances, and the **Value** is a list of strings that share that distance pattern.

```
=====
VISUALIZING THE SIGNATURE (The "Why")
=====
```

```
String:  A --- B --- C
Distance: +1    +1          -> Signature: (1, 1)

String:  X --- Y --- Z
Distance: +1    +1          -> Signature: (1, 1)

String:  Z --- A --- B
Distance: +1    +1          -> Signature: (1, 1)
(Note: Z to A is 1 because: (0 - 25 + 26) % 26 = 1)
```

```
=====
STEP-BY-STEP WALKTHROUGH
=====
```

```
Input: ["abc", "bcd", "acef", "xyz", "az", "ba", "a", "z"]
```

```
1. Process "abc":
   'b'-'a'=1, 'c'-'b'=1. Key: "1,1".
```

```

Map: {"1,1": ["abc"]}

2. Process "bcd":
   'c'-'b'=1, 'd'-'c'=1. Key: "1,1".
   Map: {"1,1": ["abc", "bcd"]}

3. Process "acef":
   'c'-'a'=2, 'e'-'c'=2, 'f'-'e'=1. Key: "2,2,1".
   Map: {"1,1": ["abc", "bcd"], "2,2,1": ["acef"]}

4. Process "az":
   'z'-'a'=25. Key: "25".
   Map: {..., "25": ["az"]}

5. Process "ba":
   'a'-'b' = -1. (-1 + 26) % 26 = 25. Key: "25".
   Map: {..., "25": ["az", "ba"]}

6. Process "a":
   No adjacent chars. Key: "empty".
   Map: {..., "empty": ["a"]}

7. Process "z":
   No adjacent chars. Key: "empty".
   Map: {..., "empty": ["a", "z"]}

```

3. Time and Space Complexity Analysis

Let N be the number of strings and K be the maximum length of a string.

TIME COMPLEXITY: $O(N * K)$

For every string (N strings):

 We iterate through its characters (K characters).

 We calculate the distance: $O(1)$ per character.

 We create a string/tuple key: $O(K)$.

 We insert into Hash Map: $O(K)$ to hash the key.

Total = $N * (K + K + K)$

Total = $O(N * K)$

SPACE COMPLEXITY: $O(N * K)$

The Hash Map stores every character of every string.

1. The Keys: In the worst case, N keys of length K.
2. The Values: Every original string is stored once.

Total = $O(N * K)$

4. Solution Code

Python Implementation

```
import collections

def groupStrings(strings):
    # Use a dictionary to group strings by their distance signature
    groups = collections.defaultdict(list)

    for s in strings:
        # A single character string has an empty signature
        if len(s) == 1:
            signature = ("single",)
        else:
            diffs = []
            for i in range(len(s) - 1):
                # Calculate circular distance: (char2 - char1 + 26) % 26
                # ord() gets the integer ASCII value
                d = (ord(s[i+1]) - ord(s[i]) + 26) % 26
                diffs.append(d)
            signature = tuple(diffs)

        # Add original string to the list of its signature
        groups[signature].append(s)

    return list(groups.values())
```

JavaScript Implementation

```
/**
 * @param {string[]} strings
 * @return {string[][]}
 */
function groupStrings(strings) {
    const groups = new Map();

    for (let s of strings) {
        let signature = "";
```

```

    if (s.length === 1) {
      signature = "single";
    } else {
      const diffs = [];
      for (let i = 0; i < s.length - 1; i++) {
        // Calculate distance and handle the wrap-around
        let d = (s.charCodeAt(i + 1) - s.charCodeAt(i) + 26) % 26;
        diffs.push(d);
      }
      // Use a comma-separated string as a key for the Map
      signature = diffs.join(',');
    }

    if (!groups.has(signature)) {
      groups.set(signature, []);
    }
    groups.get(signature).push(s);
  }

  return Array.from(groups.values());
}

```

Note 1: Terms & Techniques

- **Canonical Form:** This is a way to represent data such that different but “equivalent” inputs produce the same output. In this problem, the sequence of differences is the canonical form of all possible shifts.
- **Modulo Arithmetic (% 26):** This is the mathematical way to create a “loop.” It ensures that if we “fall off” the end of the alphabet, we wrap back to the beginning.

Note 2: Real-World Interview Variations

1. Variation: Caesar Cipher Detection (Bloomberg) * Problem: Given a list of encrypted messages, group those that were encrypted using the same Caesar Cipher but with different shift keys. *** Solution:** This is the exact same problem. You find the relative distance between letters in the message to identify the underlying pattern.

2. Variation: Image Pattern Recognition (Meta) * Problem: You have a grid of pixels (0-255 brightness). Find patterns that are identical but just shifted in brightness (e.g., one is just a “lighter” version of the other). *** Solution:** Treat the brightness sequence like a string. Instead of (char2 -

`char1) % 26`, you use `(pixel2 - pixel1)`. The “Signature” of the relative changes allows you to match images regardless of lighting.

3. Variation: Genomic Sequence Shifts (Google) * **Problem:** DNA sequences can sometimes be read starting from different offsets. Group sequences that are essentially the same but started at different points in a repeating cycle.
* **Solution:** This uses a variation of the signature. You can use the “Smallest Lexicographical Shift” as the key. For a sequence “GAT”, you check “GAT”, “ATG”, “TGA” and choose the one that comes first alphabetically as your “Signature” (Canonical Form) for the Hash Map.